

**Минобрнауки России**

**Юго-Западный государственный университет**

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА  
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Программно-информационная система для управления проектами

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

Н. Р. Марин

(инициалы, фамилия)

Группа ПО-226

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

# **Минобрнауки России**

## **Юго-Западный государственный университет**

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

---

(подпись, инициалы, фамилия)

« \_\_\_\_ » \_\_\_\_\_ 20 \_\_\_\_ г.

### **ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА**

Студента Марин Н. Р., шифр 22-06-0067, группа ПО-226

1. Тема «Программно-информационная система для управления проектами» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1585-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) проанализировать предметную область управления проектами и сопоставить функциональные возможности существующих ИС УП, доступных российским пользователям;

2) сформировать техническое задание на разработку веб-ориентированной программно-информационной системы для управления проектами;

3) спроектировать архитектуру программной системы, схему базы данных и программный интерфейс взаимодействия клиентской и серверной частей;

4) сконструировать клиентскую и серверную части программной системы, реализовать диаграмму Ганта с поддержкой четырех типов зависимостей задач и разграничение прав участников;

5) провести системное тестирование программной системы.

### 3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: учетные данные пользователей (логин, адрес электронной почты, ФИО, пароль), параметры проектов (название, описание, даты начала и завершения), параметры задач (название, описание, даты начала и завершения), сведения о зависимостях между задачами (предшественник, преемник, тип связи), состав участников проектов с указанием ролей, настройки цветовой темы интерфейса.

2) Выходными данными для программной системы являются: реестр проектов пользователя, диаграмма Ганта проекта со списком задач, стрелками зависимостей, линией текущей даты и индикацией истекшей части полос, перечень участников проекта с их ролями, цветовая палитра интерфейса, сообщения о результатах операций над данными.

### 4. Содержание работы (по разделам):

#### 4.1. Введение.

4.2. Анализ предметной области: управление проектами как область деятельности, методологии управления проектами, обзор информационных систем для управления проектами, сравнительный анализ существующих решений.

4.3. Техническое задание: основание для разработки, цель и назначение разработки, требования к функциональности, требования к интерфейсу, моделирование вариантов использования, требования к надежности и информационной безопасности, требования к программной и технической совместимости, требования к оформлению документации.

4.4. Технический проект: общая характеристика проектного решения, выбор средств и технологий реализации, архитектурное построение системы, проект базы данных.

4.5. Рабочий проект: спецификация компонентов и классов программной системы, системное тестирование программной системы.

#### 4.6. Заключение.

#### 4.7. Список использованных источников.

### 5. Перечень графического материала:

- Лист 1. Сведения о ВКРБ.
- Лист 2. Цель и задачи разработки.
- Лист 3. Диаграмма прецедентов.
- Лист 4. Концептуальная модель данных.
- Лист 5. Архитектура системы.
- Лист 6. Схема базы данных.
- Лист 7. Страница "Диаграмма Ганта".
- Лист 8. Страница "Проект".
- Лист 9. Карточка задачи.
- Лист 10. Страница настроек.
- Лист 11. Заключение.

Руководитель ВКР

---

(подпись, дата)

А. А. Чаплыгин

---

(инициалы, фамилия)

Задание принял к исполнению

---

(подпись, дата)

Н. Р. Марин

---

(инициалы, фамилия)

## РЕФЕРАТ

Объем работы равен 145 страницам. Работа содержит 26 иллюстраций, 24 таблицы, 56 библиографических источников и 11 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: управление проектами, информационная система, диаграмма Ганта, зависимости задач, веб-приложение, REST, Django, React, MySQL, ролевая модель, аутентификация, цветовая палитра.

Объектом разработки является программно-информационная система для управления проектами, рассчитанная на небольшие команды.

Система реализована как веб-приложение с клиент-серверной архитектурой. Серверная часть написана на языке Python с применением фреймворка Django, программный интерфейс построен в стиле REST, данные хранятся в реляционной СУБД MySQL 8.0. Клиентская часть выполнена как одностраничное приложение на библиотеке React. Диаграмма Ганта реализована собственным компонентом и поддерживает четыре типа зависимостей задач, перетаскивание полос, изменение порядка строк и масштабирование.

Разработанная система пригодна для использования небольшими командами без установки клиентского программного обеспечения.

## ABSTRACT

The volume of work is 145 pages. The work contains 26 illustrations, 24 tables, 56 bibliographic sources and 11 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The source code fragments are presented in annex B.

List of keywords: project management, information system, Gantt chart, task dependencies, web application, REST, Django, React, MySQL, role model, authentication, color palette.

The object of the development is a project management information system intended for small teams.

The system is implemented as a web application with a client-server architecture. The server side is written in Python using the Django framework with a REST API and MySQL 8.0 as persistent storage. The client side is a single-page application built with React. The Gantt chart is implemented as a custom component supporting four dependency types, bar dragging, row reordering and scaling.

The developed system can be used by small teams without installing any client-side software.

## СОДЕРЖАНИЕ

|                                                      |    |
|------------------------------------------------------|----|
| ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ                             | 12 |
| ВВЕДЕНИЕ                                             | 15 |
| 1 Анализ предметной области                          | 18 |
| 1.1 Управление проектами как область деятельности    | 18 |
| 1.2 Методологии управления проектами                 | 19 |
| 1.2.1 Классический подход                            | 20 |
| 1.2.2 Диаграмма Ганта                                | 21 |
| 1.2.3 Agile и его частные методы                     | 22 |
| 1.2.4 Гибридные подходы                              | 22 |
| 1.3 Информационные системы для управления проектами  | 24 |
| 1.3.1 Ключевые сущности предметной области           | 25 |
| 1.3.2 Классификация систем управления проектами      | 25 |
| 1.3.3 Типовая функциональность                       | 26 |
| 1.3.4 Проблемы применения ИС УП в небольших командах | 27 |
| 1.3.5 Технологические тенденции                      | 27 |
| 1.4 Сравнительный анализ существующих решений        | 28 |
| 1.4.1 Краткая характеристика рассматриваемых систем  | 28 |
| 1.4.2 Сравнение по ключевым критериям                | 30 |
| 1.4.3 Выводы по сравнению                            | 32 |
| 1.5 Выводы по анализу предметной области             | 33 |
| 2 Техническое задание                                | 35 |
| 2.1 Основание для разработки                         | 35 |
| 2.2 Цель и назначение разработки                     | 35 |
| 2.3 Требования к функциональности                    | 36 |
| 2.4 Требования к интерфейсу                          | 39 |
| 2.5 Моделирование вариантов использования            | 44 |
| 2.5.1 Прецедент «Регистрация пользователя»           | 47 |
| 2.5.2 Прецедент «Вход в систему»                     | 48 |
| 2.5.3 Прецедент «Выход из системы»                   | 49 |

|        |                                                              |    |
|--------|--------------------------------------------------------------|----|
| 2.5.4  | Прецедент «Просмотр профиля пользователя»                    | 49 |
| 2.5.5  | Прецедент «Редактирование профиля пользователя»              | 50 |
| 2.5.6  | Прецедент «Настройка цветовой темы интерфейса»               | 50 |
| 2.5.7  | Прецедент «Просмотр списка проектов»                         | 51 |
| 2.5.8  | Прецедент «Создание проекта»                                 | 51 |
| 2.5.9  | Прецедент «Редактирование параметров проекта»                | 52 |
| 2.5.10 | Прецедент «Удаление проекта»                                 | 53 |
| 2.5.11 | Прецедент «Добавление участника в проект»                    | 53 |
| 2.5.12 | Прецедент «Изменение роли участника»                         | 54 |
| 2.5.13 | Прецедент «Удаление участника из проекта»                    | 54 |
| 2.5.14 | Прецедент «Выход из проекта»                                 | 55 |
| 2.5.15 | Прецедент «Просмотр диаграммы Ганта»                         | 55 |
| 2.5.16 | Прецедент «Создание задачи»                                  | 56 |
| 2.5.17 | Прецедент «Редактирование задачи»                            | 57 |
| 2.5.18 | Прецедент «Удаление задачи»                                  | 58 |
| 2.5.19 | Прецедент «Изменение порядка задач»                          | 59 |
| 2.5.20 | Прецедент «Создание зависимости между задачами»              | 59 |
| 2.5.21 | Прецедент «Удаление зависимости между задачами»              | 60 |
| 2.6    | Требования к надежности                                      | 61 |
| 2.7    | Требования к информационной безопасности                     | 61 |
| 2.8    | Требования к программной и технической совместимости         | 62 |
| 2.9    | Требования к оформлению документации                         | 63 |
| 3      | Технический проект                                           | 64 |
| 3.1    | Общая характеристика проектного решения                      | 64 |
| 3.2    | Выбор средств и технологий реализации                        | 66 |
| 3.2.1  | Состав применяемого технологического стека                   | 66 |
| 3.2.2  | Серверный язык программирования и фреймворк Django           | 67 |
| 3.2.3  | Django REST Framework для построения программного интерфейса | 67 |
| 3.2.4  | Клиентская библиотека React                                  | 68 |
| 3.2.5  | СУБД MySQL                                                   | 69 |



|        |                                                        |     |
|--------|--------------------------------------------------------|-----|
| 3.3    | Архитектурное построение системы                       | 69  |
| 3.3.1  | Уровневая модель архитектуры                           | 69  |
| 3.3.2  | Компоненты клиентской части                            | 70  |
| 3.3.3  | Компоненты серверной части                             | 71  |
| 3.3.4  | Компоненты уровня доступа к данным                     | 72  |
| 3.3.5  | Диаграмма последовательности обработки запроса         | 72  |
| 3.3.6  | Форматы обмена данными                                 | 74  |
| 3.3.7  | Аутентификация и авторизация                           | 75  |
| 3.3.8  | Описание программного интерфейса                       | 76  |
| 3.4    | Проект базы данных                                     | 84  |
| 3.4.1  | ER-модель данных                                       | 84  |
| 3.4.2  | Нормализация данных                                    | 85  |
| 3.4.3  | Описание структуры базы данных                         | 86  |
| 4      | Рабочий проект                                         | 96  |
| 4.1    | Спецификация классов и компонентов программной системы | 96  |
| 4.1.1  | Класс User                                             | 96  |
| 4.1.2  | Класс Project                                          | 97  |
| 4.1.3  | Класс Task                                             | 97  |
| 4.1.4  | Класс TaskDependency                                   | 97  |
| 4.1.5  | Класс ProjectMember                                    | 98  |
| 4.1.6  | Класс UserColorPalette                                 | 98  |
| 4.1.7  | Класс AuthViewSet                                      | 99  |
| 4.1.8  | Класс UserViewSet                                      | 99  |
| 4.1.9  | Класс ProjectViewSet                                   | 100 |
| 4.1.10 | Класс ProjectTaskViewSet                               | 101 |
| 4.1.11 | Класс TaskViewSet                                      | 102 |
| 4.1.12 | Класс ProjectDependencyViewSet                         | 102 |
| 4.1.13 | Класс ProjectMemberViewSet                             | 102 |
| 4.1.14 | Класс PasswordStrengthValidator                        | 103 |
| 4.1.15 | Модуль проверки зависимостей задач                     | 104 |
| 4.1.16 | Модуль проверки прав доступа                           | 104 |

|                                                                                        |        |
|----------------------------------------------------------------------------------------|--------|
| 4.1.17 Компонент GanttChart                                                            | 105    |
| 4.1.18 Компонент TaskCard                                                              | 105    |
| 4.1.19 Компонент ProjectMembers                                                        | 106    |
| 4.1.20 Компонент SettingsPanel                                                         | 106    |
| 4.1.21 Контексты клиентского приложения                                                | 106    |
| 4.1.22 Модули обмена с сервером                                                        | 107    |
| 4.2 Системное тестирование                                                             | 108    |
| 4.2.1 Сводная таблица тест-кейсов                                                      | 108    |
| 4.2.2 ТС-01. Регистрация и вход пользователя в систему                                 | 109    |
| 4.2.3 ТС-02. Создание проекта                                                          | 110    |
| 4.2.4 ТС-03. Добавление задач и просмотр диаграммы Ганта                               | 111    |
| 4.2.5 ТС-04. Создание зависимости между задачами                                       | 112    |
| 4.2.6 ТС-05. Редактирование задачи через карточку задачи                               | 113    |
| 4.2.7 ТС-06. Управление составом участников проекта                                    | 114    |
| 4.2.8 ТС-07. Настройка цветовой темы интерфейса                                        | 115    |
| 4.2.9 ТС-08. Адаптивное отображение на мобильном устройстве                            | 116    |
| ЗАКЛЮЧЕНИЕ                                                                             | 118    |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ                                                       | 120    |
| ПРИЛОЖЕНИЕ А Представление графического материала                                      | 129    |
| ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы                                        | 141    |
| Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)                           | Лист 1 |
| Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)         | Лист 2 |
| Диаграмма прецедентов (Графический материал / Диаграмма прецедентов.png)               | Лист 3 |
| Концептуальная модель данных (Графический материал / Концептуальная модель данных.png) | Лист 4 |
| Архитектура системы (Графический материал / Архитектура системы.png)                   | Лист 5 |
| Схема базы данных (Графический материал / Схема базы данных.png)                       | Лист 6 |
| Страница "Диаграмма Ганта"(Графический материал / Страница "Диаграм-                   |        |

|                                                                  |         |
|------------------------------------------------------------------|---------|
| ма Ганта”.png)                                                   | Лист 7  |
| Страница ”Проект”(Графический материал / Страница ”Проект”.png)  | Лист 8  |
| Карточка задачи (Графический материал / Карточка задачи.png)     | Лист 9  |
| Страница настроек (Графический материал / Страница настроек.png) | Лист 10 |
| Заключение (Графический материал / Заключение.png)               | Лист 11 |

## ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

БД – база данных.

ИС – информационная система.

ИС УП – информационная система управления проектами.

ИТ – информационные технологии.

ПО – программное обеспечение.

РФ – Российская Федерация.

СУБД – система управления базами данных.

ТЗ – техническое задание.

ТП – технический проект.

API (Application Programming Interface) – программный интерфейс приложения, набор соглашений и средств взаимодействия одной программной системы с другой.

CPM (Critical Path Method) – метод критического пути, метод сетевого планирования работ проекта.

CRM (Customer Relationship Management) – система управления взаимоотношениями с клиентами.

CSRF (Cross-Site Request Forgery) – межсайтовая подделка запроса, разновидность атаки на веб-приложения, при которой запрос от имени пользователя выполняется со стороннего сайта.

CSS (Cascading Style Sheets) – каскадные таблицы стилей, язык описания внешнего вида веб-страниц.

DRF (Django REST Framework) – библиотека для построения программных интерфейсов в формате REST поверх фреймворка Django.

ER (Entity-Relationship) – модель «сущность – связь», графический способ описания структуры данных.

ERP (Enterprise Resource Planning) – система планирования ресурсов предприятия.

HTML (HyperText Markup Language) – язык гипертекстовой разметки веб-страниц.

HTTP (HyperText Transfer Protocol) – протокол прикладного уровня для передачи гипертекстовых документов.

HTTPS (HyperText Transfer Protocol Secure) – расширение протокола HTTP с шифрованием передаваемых данных средствами TLS.

JSON (JavaScript Object Notation) – текстовый формат обмена структурированными данными.

ORM (Object-Relational Mapping) – объектно-реляционное отображение, технология связывания объектов программы со строками реляционной базы данных.

PBKDF2 (Password-Based Key Derivation Function 2) – функция формирования криптографического ключа из пароля с применением соли и многократного хеширования.

PERT (Program Evaluation and Review Technique) – техника анализа и оценки программ, метод сетевого планирования.

PMBOK (Project Management Body of Knowledge) – свод знаний по управлению проектами, выпускаемый Project Management Institute.

PMI (Project Management Institute) – Институт управления проектами, профессиональная организация в области проектного менеджмента.

REST (Representational State Transfer) – архитектурный стиль построения распределенных систем, основанный на стандартных операциях протокола HTTP над ресурсами.

SaaS (Software as a Service) – модель распространения программного обеспечения, при которой приложение размещается на инфраструктуре поставщика и предоставляется пользователям через сеть Интернет.

SAFe (Scaled Agile Framework) – масштабированный гибкий фреймворк, набор принципов и практик для применения гибких методологий в крупных организациях с несколькими командами разработки.

SPA (Single Page Application) – одностраничное веб-приложение.

SQL (Structured Query Language) – язык структурированных запросов к реляционным базам данных.

UML (Unified Modeling Language) – унифицированный язык моделирования, применяемый при разработке программного обеспечения.

URL (Uniform Resource Locator) – единый указатель ресурса в сети Интернет.

UTF-8 (Unicode Transformation Format, 8-bit) – кодировка символов Юникода переменной длины, обратно совместимая с ASCII.

XP (eXtreme Programming) – экстремальное программирование, гибкая методология разработки программного обеспечения.

XSS (Cross-Site Scripting) – межсайтовый скриптинг, разновидность атаки на веб-приложения через внедрение исполняемого кода в страницы, отображаемые другим пользователям.

## ВВЕДЕНИЕ

Управление проектами как самостоятельная инженерная дисциплина оформилось во второй половине XX века. К 2000-м годам ее принципы вышли за пределы строительства и оборонных программ – сегодня по проектным методикам работают команды разработки программного обеспечения, маркетинга, научных исследований и государственных программ. Численность проектных команд растет, география распределенная, а планы все чаще ведутся параллельно по нескольким направлениям одновременно.

С увеличением числа задач и участников держать план в почте и электронных таблицах становится невозможно. Сроки расходятся между файлами, ответственные забываются, изменения в зависимостях задач не доходят до тех, кого они касаются. Отсюда возникла отдельная категория программных средств – информационные системы управления проектами. Они хранят данные о проектах, задачах, сроках и составе участников в одной базе, и команда работает с единой версией плана.

К началу 2024 года ситуация с такими системами в России заметно изменилась. Atlassian свернул деятельность в стране и принудительно отключил облачные сервисы Jira и Trello для российских аккаунтов. У серверных версий с февраля 2024 года прекращена поддержка. Asana ограничила бесплатный тариф двумя пользователями. Российские альтернативы развиваются, но ни одна из проанализированных в работе систем не сочетает в одной точке нативную диаграмму Ганта, поддержку всех четырех типов зависимостей сетевого планирования, русскоязычный интерфейс, бесплатный тариф для небольшой команды и устойчивый доступ из РФ. Системы, закрывающие функциональную часть, либо ушли с рынка, либо превратились в универсальные корпоративные платформы, где управление проектами оказалось среди десятков других модулей.

Небольшим командам, ведущим несколько параллельных программных проектов, такие платформы избыточны. Запуск занимает недели, обучение пользователей – дни, и большую часть функций команда не использует.

Простые же канбан-доски быстро упрутся в потолок: как только в проекте появляются взаимосвязанные работы, сроки и разные уровни доступа, доски перестают покрывать задачу планирования. Эти соображения и определили направление работы.

Цель настоящей работы – разработка веб-ориентированной программно-информационной системы для управления проектами, рассчитанной на небольшие команды и объединяющей в одном инструменте реестр проектов, диаграмму Ганта с поддержкой четырех типов зависимостей задач и разграничение прав участников по трем ролям.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и сопоставить функциональные возможности существующих ИС УП, доступных российским пользователям;
- сформировать техническое задание на разработку, описать прецеденты использования системы;
- спроектировать архитектуру системы, схему базы данных и программный интерфейс взаимодействия клиентской и серверной частей;
- реализовать клиентскую и серверную части, диаграмму Ганта с интерактивным изменением дат и порядка задач, режим добавления связей и разграничение прав участников;
- провести системное тестирование программной системы.

Структура и объем работы. Работа состоит из введения, четырех разделов основной части, заключения, списка использованных источников и 2 приложений. Текст выпускной квалификационной работы равен 145 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы и приведено краткое содержание каждого из разделов.

В первом разделе приводится анализ предметной области управления проектами: рассматриваются ключевые понятия, методологии управления



проектами, типовая функциональность ИС УП и сопоставляются восемь существующих решений, доступных российским пользователям. По результатам сравнения сформулированы требования к разрабатываемой системе.

Во втором разделе приведено техническое задание на разработку: основание для разработки, цель и назначение, функциональные требования, требования к интерфейсу, описаны прецеденты использования системы, требования к надежности и информационной безопасности, к программной и технической совместимости, к оформлению документации.

В третьем разделе описано техническое проектирование системы: общая характеристика проектного решения, выбор средств и технологий реализации, архитектурное построение системы, проект базы данных с приведением ER-модели и описанием структуры таблиц, описание программного интерфейса между клиентской и серверной частями.

В четвертом разделе рассмотрена реализация системы: спецификация программных компонентов клиентской и серверной частей и результаты системного тестирования.

В заключении приведены основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

## **1 Анализ предметной области**

### **1.1 Управление проектами как область деятельности**

ГОСТ Р ИСО 21500–2023 определяет проект как временное предприятие, направленное на создание уникального продукта, услуги или результата [5]. ГОСТ Р 54869–2011 трактует проект иначе – как комплекс взаимосвязанных мероприятий, направленный на создание уникального результата в условиях ограничений по времени и ресурсам [6].

Из этих определений видны четыре признака, по которым проект отличается от операционной деятельности:

1. Конечные сроки – у проекта есть дата начала и дата, к которой результат должен быть готов.

2. Уникальность результата – речь не о том, что подобного никогда не делалось. Имеется в виду уникальное сочетание контекста, требований и условий: два внешне одинаковых по ТЗ сайта, сделанных для разных компаний, все равно остаются разными проектами [27].

3. Ресурсные ограничения – любой проект тратит время людей, бюджет и оборудование. Балансировка между этими ограничениями и объемом работ – большая часть работы руководителя.

4. Измеримая цель – без сформулированной цели не понять, в какой момент проект можно считать завершенным.

Управление проектом, по PMBOK, – это применение знаний, навыков, инструментов и методов к работам проекта для удовлетворения требований, которые к нему предъявлены [47]. Руководитель в любой момент должен знать состав работ, исполнителей и сроки. Без централизованного хранения эти сведения быстро расходятся по почте и переписке, теряют актуальность.

PMBOK делит процессы управления на пять групп: инициация, планирование, исполнение, мониторинг и контроль, завершение. Первые четыре образуют основной поток работ от запуска до сдачи проекта. Мониторинг и контроль идут параллельно планированию и исполнению – они отвечают за

то, чтобы план вовремя корректировался по фактическим результатам. Взаимосвязь процессов показана на рисунке 1.1.

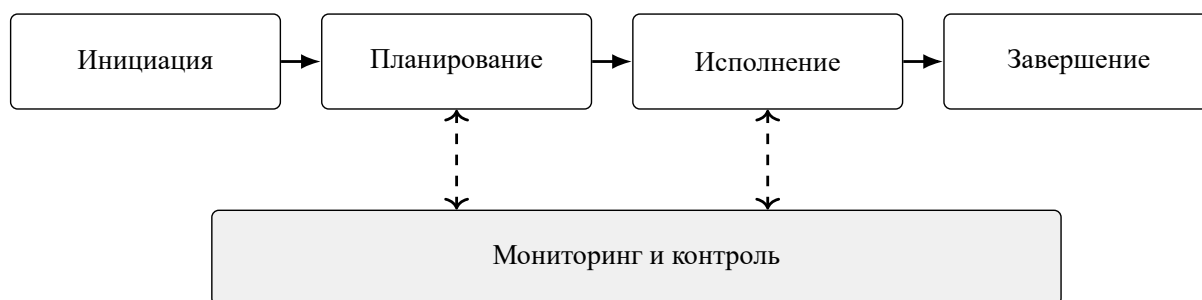


Рисунок 1.1 – Группы процессов управления проектом по PMBOK

ГОСТ Р 54869–2011 выделяет в проектной деятельности четыре ключевые роли: заказчик проекта, куратор, руководитель и команда проекта [6]. Основная часть рабочих процессов строится вокруг руководителя и команды – именно их повседневные потребности определяют, что должна уметь система автоматизации.

Как самостоятельная дисциплина управление проектами оформилось во второй половине XX века. Сначала появились формальные методы планирования: диаграмма Ганта в 1910 году, методы PERT и CPM в конце 1950-х. В 1969 году учрежден Project Management Institute (PMI), в 1996 году вышло первое издание PMBOK – Свода знаний по управлению проектами. К 1990-м годам управление проектами вышло за пределы инженерных и оборонных отраслей [47, 27]. Сегодня его принципы применяют в ИТ, маркетинге, образовании, государственных программах.

Чем больше проектов и распределенных команд, тем тяжелее держать планирование на бумаге и в почте. Отсюда возникла потребность в специализированных программных средствах.

## 1.2 Методологии управления проектами

Методология управления проектами – это набор принципов и практик, который описывает, как проект планируется, исполняется и контролируется

[37]. Конкретные инструменты она не предписывает. Внутри методологии команда сама выбирает программы и форматы документов под свои задачи.

Условно такие методологии делят на две большие группы: классические (последовательные) и гибкие. Их принципиальное отличие показано на рисунке 1.2.

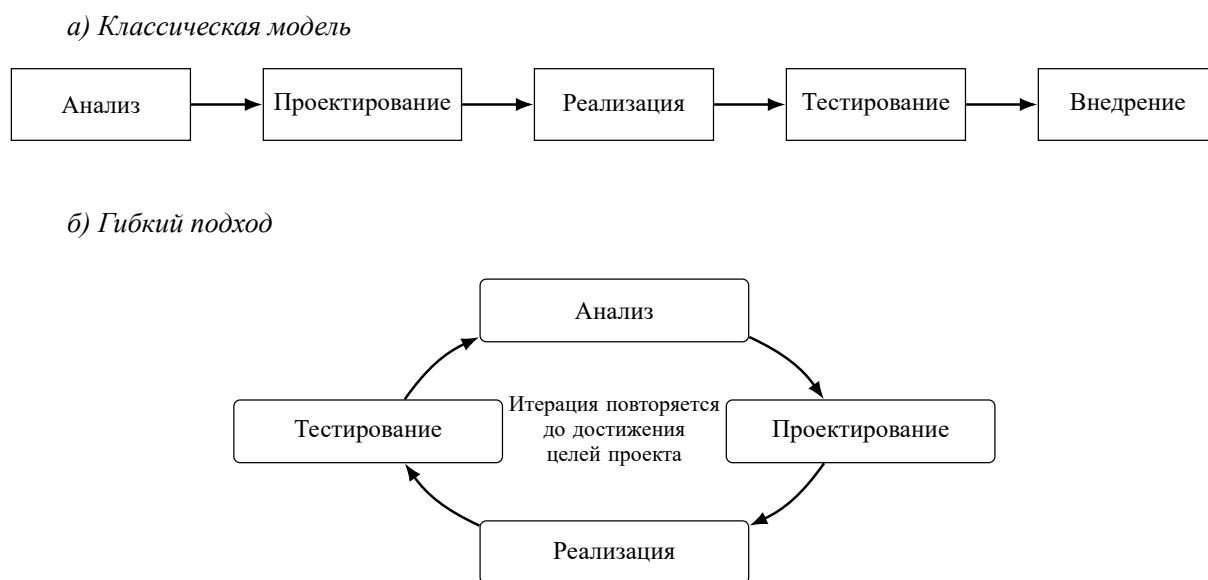


Рисунок 1.2 – Сравнение классической и гибкой моделей разработки

В классической модели все этапы выстраиваются в одну цепочку: каждый следующий начинается только после завершения предыдущего. Гибкий подход устроен иначе – работа разбивается на короткие повторяющиеся циклы, каждый из которых проходит все стадии и завершается работающим инкрементом продукта. На практике большинство команд комбинирует элементы обоих подходов – такие комбинации называют гибридными.

### 1.2.1 Классический подход

Классическая модель опирается на представление о проекте как о цепочке последовательных стадий: формирование требований, проектирование, реализация, тестирование, ввод в эксплуатацию. Каждая следующая стадия начинается после завершения предыдущей. Такой подход известен как

каскадная модель и впервые был сформулирован в 1970-х годах применительно к разработке программного обеспечения [27].

Каскадные методы применимы там, где требования зафиксированы на старте и не меняются по ходу работ: строительство, государственные контракты, разработка отраслевых регламентов. При соблюдении этого условия проект остается предсказуемым по срокам и стоимости.

Классический подход дал начало формальным инструментам сетевого планирования: диаграмме Ганта, методу критического пути и методу PERT [47]. Все три до сих пор применяются в системах управления проектами, когда нужно учесть порядок выполнения работ и влияние задержки отдельной задачи на общий срок проекта.

### **1.2.2 Диаграмма Ганта**

Диаграмма Ганта разработана американским инженером Генри Гантом в начале XX века и с тех пор остается стандартным инструментом планирования работ [10]. Каждая задача показана горизонтальным отрезком, длина которого передает ее продолжительность. Левый край отрезка соответствует дате начала задачи, правый – дате завершения. Когда между задачами добавляются связи, диаграмма позволяет определить критический путь – самую длинную взаимосвязанную цепочку работ, которая задает нижнюю границу общей длительности проекта. Главное достоинство такого представления – наглядность. На одном экране умещается план из десятков работ, и сразу видно, какие из них идут параллельно, а какие последовательно.

Диаграмма Ганта не является атрибутом исключительно классического подхода. В большинстве современных ИС УП она реализована как базовое представление данных, доступное в том числе командам, работающим по гибким методологиям: задачи группируются по спринтам, зависимости отражают связи между инкрементами. Ее присутствие в каждой из рассматриваемых далее систем подтверждает, что это рабочий инструмент независимо от выбранной методологии.

### 1.2.3 Agile и его частные методы

Гибкие методологии (Agile) появились в ИТ как ответ на жесткость каскадной модели. В 2001 году группа разработчиков сформулировала «Манифест гибкой разработки программного обеспечения»: работающий продукт ставится выше исчерпывающей документации, сотрудничество с заказчиком – выше контрактных формальностей, готовность к изменениям – выше следования первоначальному плану [29].

Среди конкретных методов Agile наибольшее применение в ИТ получили Scrum и Kanban.

Scrum строит работу команды короткими циклами фиксированной длительности – спринтами, обычно по две-четыре недели. На старте спринта команда отбирает задачи из общего журнала, в конце – показывает работающий результат и проводит ретроспективу [34]. Ролей в команде три: владелец продукта, скрам-мастер и разработчики. Подход хорошо ложится на продуктовую разработку с коротким горизонтом планирования.

Kanban пришел в ИТ из производственной системы Toyota. Дэвид Андерсон в середине 2000-х адаптировал его под работу со знаниями [8]. В основе – доска с колонками по этапам и ограничение на число задач в работе одновременно. Жестких итераций тут нет, поэтому Kanban удобен там, где задачи идут потоком: поддержка, сопровождение, разбор инцидентов.

Экстремальное программирование (XP) больше про инженерные практики, чем про управление: парное программирование, разработка через тестирование, непрерывная интеграция, простота проектных решений [11]. По части проектного управления XP менее формален, чем Scrum, но его инженерные приемы перешли в другие методологии.

### 1.2.4 Гибридные подходы

Большинство команд на практике не следует одной методологии строго – они заимствуют элементы из разных подходов: итеративную работу из

Scrum, доску с колонками из Kanban, формальное закрытие фаз из каскадной модели [45]. Такие комбинации принято называть гибридными.

Самый известный гибридный формат – Scrumban. От Scrum он берет ритм ретроспектив, а жесткие границы спринта меняет на непрерывный поток: задачи поступают на доску по мере необходимости, а лимит на число одновременных задач заимствуется из Kanban. Такая схема выручает команды со смешанной нагрузкой, где плановые задачи на новую функциональность чередуются с дефектами и запросами на поддержку.

Для крупных компаний с несколькими командами разработаны фреймворки масштабирования. SAgile (Scaled Agile Framework, масштабированный гибкий фреймворк) надстраивает над спринтами несколько уровней координации, где команды синхронизируются через квартальные программные инкременты, а на уровне портфеля остается стратегическое планирование, характерное для классической модели. За счет такой надстройки несколько команд могут работать в едином ритме, не теряя гибкости отдельного спринта.

Шестое издание PMBOK прямо признает: реальные проекты обычно идут по адаптированному процессу, а не по строгой реализации одной методологии [47]. Для ИС УП это означает простое требование. Система должна одновременно поддерживать спринтовое планирование, непрерывный поток задач и классические зависимости между работами, не заставляя команду выбирать что-то одно.

Ключевые характеристики рассмотренных методологий сведены в таблицу 1.1.

Таблица 1.1 – Сравнение основных методологий управления проектами

| Признак        | Каскадная        | Scrum       | Kanban      | XP          |
|----------------|------------------|-------------|-------------|-------------|
| Тип выполнения | Последовательный | Итеративный | Непрерывный | Итеративный |

Продолжение таблицы 1.1

| Признак              | Каскадная                    | Scrum                          | Kanban                      | XP                   |
|----------------------|------------------------------|--------------------------------|-----------------------------|----------------------|
| Цикл планирования    | Весь проект<br>единовременно | Спринт<br>1–4 нед.             | Нет фиксир.<br>цикла        | Итерация<br>1–2 нед. |
| Реакция на изменения | Затруднена                   | Возможна<br>между<br>спринтами | В любой<br>момент           | Приветствуется       |
| Документирование     | Подробное,<br>заранее        | Минимальное                    | Минимальное                 | Минимальное          |
| Область применения   | Строительство,<br>госзаказ   | Продуктовая<br>разработка      | Поддержка,<br>сопровождение | Разработка<br>ПО     |

### 1.3 Информационные системы для управления проектами

Информационная система управления проектами – это программный комплекс, который автоматизирует планирование, исполнение и контроль работ по одному или нескольким проектам [35]. Данные о проектах, задачах, ресурсах и сроках хранятся в одной базе, и команда работает с единой версией плана.

Первые специализированные программы для управления проектами появились в 1970-х. Работали они на мейнфреймах и обслуживали крупные промышленные и строительные корпорации. В 1984 году вышел Microsoft Project для MS-DOS – один из первых массовых настольных продуктов в этой области [9]. Когда персональные компьютеры стали повсеместными, рынок расширился за пределы крупных организаций. В 2000-е планирование перешло в браузер – чтобы начать работу, достаточно было зарегистрироваться в сервисе. Сегодня рынок делят между собой корпоративные платформы, специализированные таск-трекеры и отраслевые решения [9].



### **1.3.1 Ключевые сущности предметной области**

Продукты на рынке отличаются интерфейсом и набором надстроек, но базовый состав сущностей у большинства ИС УП один и тот же. Главная – проект. Он задает контекст работ, сроки, цель и состав участников. Без проекта система превращается в плоский список задач, где трудно понять, к какой инициативе относится та или иная работа.

Проект разбивается на задачи. У каждой задачи есть описание, даты начала и завершения, продолжительность и место в общей последовательности. Задача в проектной системе не существует сама по себе – она часть календарного плана.

Состав команды неоднороден. Один участник заводит проект и собирает остальных, у другого основная работа – правка задач, третий просто следит за ходом дел и не должен иметь возможности что-то изменить. Если система не разграничивает эти случаи, пользователи получают либо лишние права, либо избыточные ограничения. Поэтому ИС УП хранит не только сам факт участия пользователя в проекте, но и его роль.

Зависимости между задачами фиксируют порядок выполнения работ. Без них в плане легко получить противоречие: задача-преемник начинается раньше предшественника. В небольшом проекте из пяти-шести задач этим можно пренебречь, но для больших планов связи – единственный способ удержать управляемость.

### **1.3.2 Классификация систем управления проектами**

По масштабу ИС УП обычно делят на три группы [35]. На одном краю спектра – корпоративные системы, в которых сотни пользователей и есть интеграция с CRM, ERP, учетом рабочего времени. На другом краю – индивидуальные планировщики, которыми пользуется один человек. Промежуточный сегмент занимают системы для малых и средних команд, без тяжелых интеграций, но и без ограничения «один пользователь».

Вторая ось – модель распространения. Облачные системы (SaaS, программное обеспечение как услуга) размещаются на инфраструктуре поставщика, обновления и резервные копии тоже на его стороне. Пользователю достаточно браузера. Локальные решения устанавливаются на сервер заказчика, и эксплуатация переходит к нему. Большую долю рынка занимают облачные продукты: подключиться проще, входная стоимость ниже. Локальная установка нужна там, где требования к конфиденциальности не позволяют отдавать данные стороннему поставщику [9].

Третий признак – специализация. Универсальные системы подходят для любого проекта и дают гибкие настройки процессов. Отраслевые поставляются с готовыми шаблонами под конкретную сферу: разработку ПО, строительство, маркетинг, научные исследования.

### **1.3.3 Типовая функциональность**

Подходы и масштабы у систем разные, но базовый набор функций совпадает у большинства ИС УП:

- реестр проектов с описанием, сроками и ответственными;
- декомпозиция проекта на задачи – каждой можно назначить исполнителя, срок, статус и приоритет;
- диаграмма Ганта или иная визуализация календарного плана;
- зависимости между задачами для отражения порядка работ;
- разграничение прав доступа между участниками проекта;
- история изменений и операций;
- комментарии в контексте задачи;
- отчеты о состоянии проекта.

Поверх этого ядра конкретные продукты добавляют то, что считают нужным для своей аудитории. В одних системах это учет рабочего времени и автоматизация повторяющихся действий, в других – связь с системами контроля версий и мобильные клиенты. Аналитические панели с метриками встречаются ближе к корпоративному сегменту.

### **1.3.4 Проблемы применения ИС УП в небольших командах**

Крупные корпоративные платформы покрывают широкий круг задач, но требуют времени на настройку, обучение пользователей и администрирование. Для команды из пяти человек это явный перебор: вместо ведения проекта она обслуживает инструмент.

С простыми досками задач история обратная. На старте все удобно, но как только в проекте появляются сроки, взаимосвязанные работы и разные уровни доступа, досок перестает хватать. Перемещать карточки между колонками можно, а целостной картины сроков нет. Руководитель отдельно ведет календарный план в Excel, вручную проверяет зависимости и объясняет участникам, почему одну задачу нельзя начать раньше другой.

Поэтому небольшой команде нужна система, которая запускается без длительного внедрения и при этом покрывает базовый проектный контроль – единый перечень проектов, календарь задач, зависимости и разграничение прав. Без любого из этих пунктов инструмент либо перегружает команду настройкой, либо перестает справляться по мере роста проекта.

### **1.3.5 Технологические тенденции**

За последние годы рынок ИС УП заметно сдвинулся по нескольким направлениям.

Первое и самое заметное – переход к веб-архитектуре с тонким клиентом. Большинство систем стало браузерными, и пользователь работает с любого устройства, а обновления поставщик выпускает централизованно. Норма распространилась даже на исторически настольные продукты: у Microsoft Project появилась веб-версия Project for the web.

Открытый программный интерфейс из конкурентного преимущества превратился в обязательную часть. REST API сегодня есть почти у всех систем. Через него систему встраивают во внутреннюю инфраструктуру предприятия, выгружают данные в аналитические панели, связывают с системами контроля версий. Продукты, ориентированные на программные команды,

идут дальше и добавляют готовые связки с Git, GitHub, GitLab и сервисами непрерывной интеграции – так привязка коммитов к задачам выполняется автоматически, а ход работ отслеживается по реальным артефактам, а не только по статусам в отчетах.

Мобильные клиенты подтягиваются к веб-версии и по функциональности уже почти не уступают ей. Поддерживается работа без сети с последующей синхронизацией. Всплывающие уведомления особенно полезны в распределенных командах и проектах с выездными работами.

Отдельная тенденция – встроенная аналитика. В системы стали добавлять панели с проектными метриками (диаграммы выгорания, среднее время выполнения задачи, пропускная способность команды, прогноз сроков), часть которых опирается на машинное обучение: система ищет зависшие задачи и оценивает риски срыва сроков.

## **1.4 Сравнительный анализ существующих решений**

Для сопоставления выбраны восемь систем – три зарубежные с широкой базой пользователей в ИТ-среде и пять решений, доступных в Российской Федерации без ограничений и активно развивающихся в последние годы [56].

### **1.4.1 Краткая характеристика рассматриваемых систем**

**Jira** – одна из самых распространенных систем для управления ИТ-проектами. Поддерживает Scrum- и Kanban-доски, позволяет настраивать рабочие процессы, у нее большая экосистема плагинов. Выпускается компанией Atlassian и считается отраслевым стандартом отслеживания задач в командах разработки ПО. В марте 2022 года Atlassian приостановил продажи в РФ и Беларуси, к октябрю 2022 года компания полностью свернула здесь свою деятельность. В августе 2023 года Atlassian уведомил пользователей с российскими и белорусскими аккаунтами о принудительном отключении облачных сервисов в течение 30 дней. С 15 февраля 2024 года серверные версии Atlassian остались без официальной поддержки, обновлений безопасности и

исправлений ошибок. Для отечественных команд Jira после этого фактически непригодна.

**Trello** – простая Kanban-доска от того же разработчика. Низкий порог входа, большое сообщество. В 2017 году продукт перешел в собственность Atlassian. Платная функциональность скромная, поэтому Trello удобна для одного человека или небольшой группы. Для проектов, где нужны зависимости задач и диаграмма Ганта, инструмент не подходит.

**Asana** – универсальная система для команд среднего и крупного размера. Поддерживает разные представления одних и тех же данных: список, доска, диаграмма Ганта, календарь. Ориентирована на кросс-функциональные команды – маркетинг, операционные группы, проектный офис, разработка ПО проработана слабее. Русского интерфейса нет. С ноября 2025 года бесплатный тариф Asana Personal ограничи́ли двумя пользователями, ранее зарегистрированные команды сохраняют прежний лимит в десять человек.

**YouTrack** – разработка JetBrains, рассчитанная на команды разработки. Особенности – запросы на естественном языке для поиска задач и тесная связка с другими продуктами JetBrains. При локальном развертывании бесплатна для команд до десяти пользователей. Это удобно небольшим техническим коллективам, которые хотят держать данные у себя. Облачная версия тоже есть.

**Битрикс24** – комплексная российская платформа, объединяющая управление проектами с CRM, документооборотом и корпоративными коммуникациями [56]. Продукт компании 1С-Битрикс поставляется в облачном и локальном вариантах. Бесплатный тариф не ограничивает число пользователей, но дает только 5 ГБ облачного хранилища и ограничивает функциональность CRM и автоматизаций. Подходит крупным компаниям, но порог входа высокий – модулей много, и управление задачами теряется среди них.

**Kaiten** – российская система с упором на Kanban и Scrum. Сильна визуализацией потока работ и аналитикой, целевая аудитория – команды разработки и продуктовые отделы. Из особенностей – встроенная аналитика пото-

ка: среднее время выполнения задач, диаграммы накопленного потока, пропускная способность команды за произвольный период.

**YouGile** – российская система. Объединяет Kanban-доску со встроенным мессенджером прямо в карточке задачи. Переписку по конкретной работе ведут не в отдельных чатах, а в карточке – к ней потом всегда можно вернуться. Бесплатна для команд до десяти человек, порог входа низкий.

**Microsoft Project** – классический инструмент с глубокой поддержкой сетевого планирования и расчета критического пути. Первая версия вышла в 1984 году. Сегодня продукт идет в настольном варианте Project Professional и облачном Project for the web. По сетевому планированию, управлению ресурсами и межпроектным связям он богаче большинства конкурентов, но и порог освоения, и цена лицензии заметно выше. Применяется в проектах с жестким каскадным управлением, в основном вне ИТ.

#### **1.4.2 Сравнение по ключевым критериям**

Для сопоставления выбраны критерии, существенные для команды, которая ведет несколько параллельных программных проектов с распределенными участниками. Один из критериев требует пояснения. Под четырьмя типами зависимостей задач понимаются классические связи сетевого планирования: «финиш-старт», «старт-старт», «финиш-финиш» и «старт-финиш». Их графическое представление приведено на рисунке 1.3.

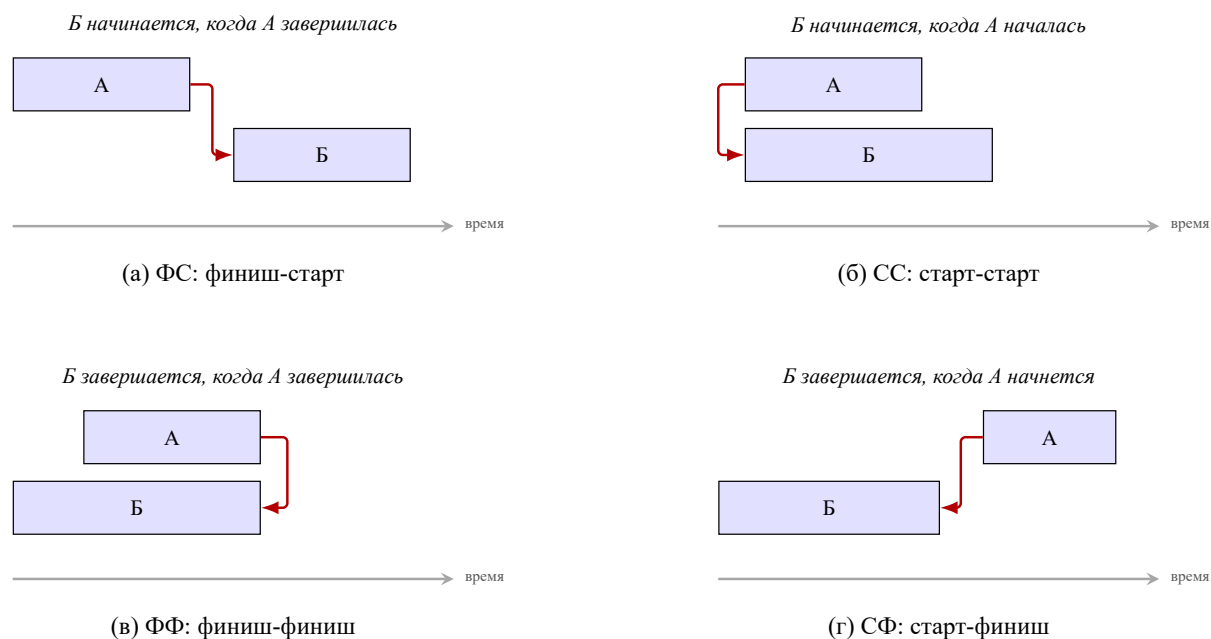


Рисунок 1.3 – Четыре типа зависимостей задач в проекте

В таблице 1.2 собраны характеристики доступности: модель распространения, наличие русского интерфейса, бесплатного тарифа и фактический доступ из Российской Федерации. В таблице 1.3 – ключевые функциональные возможности: диаграмма Ганта, связи задач, открытый программный интерфейс, мобильное приложение.

Таблица 1.2 – Сравнение систем по доступности и условиям использования

| Система    | Модель распространения | Русский язык | Бесплатный тариф | Доступ из РФ |
|------------|------------------------|--------------|------------------|--------------|
| Jira       | Обе                    | Частично     | 10 чел.          | Огранич.     |
| Trello     | Облачная               | Да           | 10 досок         | Да           |
| Asana      | Облачная               | Нет          | 2 чел.           | Огранич.     |
| YouTrack   | Обе                    | Да           | 10 чел.          | Да           |
| Битрикс24  | Обе                    | Да           | Без огр.         | Да           |
| Kaiten     | Обе                    | Да           | 5 чел.           | Да           |
| YouGile    | Обе                    | Да           | 10 чел.          | Да           |
| MS Project | Обе                    | Да           | Нет              | Да           |

Таблица 1.3 – Сравнение систем по функциональным возможностям

| Система    | Диаграмма Ганта     | Связи задач | Открытый API | Мобильное приложение |
|------------|---------------------|-------------|--------------|----------------------|
| Jira       | Частично (Timeline) | 4 типа      | Да           | Да                   |
| Trello     | Через плагин        | Нет         | Да           | Да                   |
| Asana      | Да                  | 1 тип       | Да           | Да                   |
| YouTrack   | Да                  | Да          | Да           | Да                   |
| Битрикс24  | Да                  | 4 типа      | Да           | Да                   |
| Kaiten     | Да                  | Да          | Да           | Да                   |
| YouGile    | Да                  | 1 тип       | Да           | Да                   |
| MS Project | Да                  | 4 типа      | Да           | Огранич.             |

### 1.4.3 Выводы по сравнению

Зарубежные Jira и Asana из РФ доступны с ограничениями – для постоянной работы российских команд они отпадают. Trello, MS Project и все рассмотренные российские системы доступны без оговорок.

Из доступных Asana русского интерфейса не имеет вовсе, у Jira локализация частичная. Команде, у которой английский не основной рабочий язык, это лишний барьер.

Trello – самая простая и популярная из зарубежных, но зависимости между задачами не поддерживает, нативной диаграммы Ганта тоже нет (только через сторонние плагины). Для проектов, где план строится на связях работ и сетевых графиках, не подходит.

Все четыре типа связей сетевого планирования среди свободно доступных в РФ систем поддерживают только Битрикс24 и MS Project. Kaiten и YouTrack дают зависимости без разделения на типы, YouGile – только «финиш-старт».

У MS Project нет бесплатного тарифа – для команд с минимальным бюджетом не вариант. Kaiten на бесплатном тарифе допускает пять пользовате-



лей, для среднего коллектива мало. Битрикс24 пользователей не ограничивает, но это универсальная корпоративная платформа с десятками модулей. Управление проектами там одна из многих подсистем, и порог входа высокий.

Получается, что ни одно из рассмотренных специализированных решений не объединяет в одной точке нативную диаграмму Ганта, четыре типа зависимостей, русский интерфейс, разумный бесплатный тариф и устойчивый доступ из РФ. Системы, закрывающие функциональную часть, либо уходят из России, либо превращаются в универсальную корпоративную платформу.

### **1.5 Выводы по анализу предметной области**

Рабочий план – это не плоский список задач. Чтобы он оставался актуальным, в системе должны храниться связи между задачами, сроки и состав участников. Когда эти сведения распределены по разным файлам и переписке, между ними появляется рассинхрон. План в Excel расходится со списком в трекере, ответственного за конкретную задачу никто не помнит, и руководитель тратит на сверку больше времени, чем на сами работы.

Веб-приложение, доступное всем участникам без установки клиента, удобно для распределенной работы. Руководитель и команда могут находиться в разных городах и при этом видеть один и тот же актуальный план.

Диаграмма Ганта удобна тем, что собирает в одно представление список задач, сроки и зависимости. Но она должна быть рабочим инструментом, а не картинкой в отчете. Любое изменение – сдвиг дат, перестановка задач, новая связь – обязано сразу попадать в данные проекта.

Прав доступа достаточно трех уровней. Полный набор прав – только у создателя проекта: он распоряжается и параметрами, и составом команды. Промежуточный уровень нужен тем, кто пишет план и расставляет связи между задачами, но не должен иметь возможности удалить проект целиком или поменять состав участников. Наконец, тем, кто работает в проекте только для справки, достаточно доступа на чтение. Эти три уровня покрывают

потребности небольшой команды и не требуют отдельного администрирования.

Разрабатываемая ИС УП нацелена на конкретную практическую задачу – календарное планирование с возможностью совместной работы. В рамках этой задачи решается ведение проектов и задач, отображение плана на диаграмме Ганта с зависимостями между работами и совместный доступ нескольких пользователей с разграничением их прав.

## **2 Техническое задание**

### **2.1 Основание для разработки**

Основанием для разработки является задание на выпускную квалификационную работу бакалавра. Тема работы – «Разработка программно-информационной системы для управления проектами».

Система разрабатывается в рамках преддипломной практики. Она автоматизирует ведение проектов, декомпозицию работ на задачи, управление составом команды и строит диаграмму Ганта как основной способ отображения плана. Требования сформированы по итогам анализа предметной области.

### **2.2 Цель и назначение разработки**

В рамках работы разрабатывается веб-ориентированная информационная система управления проектами. В ее составе – реестр проектов, декомпозиция работ на задачи, управление зависимостями и совместная работа участников с разграниченными правами доступа.

Система предоставляет руководителю и участникам единое рабочее пространство: планирование задач, контроль сроков, управление зависимостями между работами и визуализация плана на диаграмме Ганта. Клиентское программное обеспечение устанавливать не требуется – достаточно современного браузера.

Задачами разработки являются:

- реализация подсистемы регистрации, авторизации и управления учетными записями пользователей;
- разработка модуля создания проектов и управления их основными параметрами;
- реализация системы ролей участников проекта: владелец, редактор, наблюдатель;
- разработка модуля задач с назначением дат начала и завершения;

- реализация механизма зависимостей между задачами с поддержкой четырех типов: финиш-старт, старт-старт, финиш-финиш, старт-финиш;
- разработка диаграммы Ганта как основного представления календарного плана проекта;
- разработка адаптивного пользовательского интерфейса с минималистичным оформлением, корректно отображающегося на устройствах с различными размерами экрана;
- реализация персонализации цветовой схемы интерфейса.

## **2.3 Требования к функциональности**

Перечень функций системы:

1. Регистрация и авторизация. Система должна обеспечивать создание учетных записей с заданием имени пользователя, адреса электронной почты и пароля. Пароль должен соответствовать требованиям: длина не менее восьми символов, наличие хотя бы одной заглавной буквы, одной строчной буквы, одной цифры и одного специального символа, использование только допустимых символов. Форма регистрации также включает поле подтверждения пароля. После успешного входа сервер открывает сеанс работы и записывает идентификатор сеанса в защищенную cookie, которая передается браузером с каждым последующим запросом и используется для аутентификации.

2. Управление профилем. Пользователь должен иметь возможность просматривать и редактировать данные своей учетной записи: имя, фамилию, отчество и адрес электронной почты.

3. Настройка цветовой темы. Система должна предоставлять возможность выбора одного из четырех режимов темы оформления: системного (следует настройкам операционной системы), светлого, темного и пользовательского. В пользовательском режиме должна быть доступна настройка цветов отдельных элементов интерфейса: фона, заголовков, цвета задач на диаграмме, цвета стрелок зависимостей и прочих элементов.

4. Реестр проектов. Система должна отображать список проектов, в которых участвует пользователь, и обеспечивать переход к любому из них.

5. Создание, редактирование и удаление проекта. Пользователь должен иметь возможность создавать проект с указанием названия, описания и дат начала и завершения. При создании проекта пользователь автоматически получает роль владельца. Редактирование и удаление проекта доступны только владельцу.

6. Управление участниками. Владелец проекта должен иметь возможность добавлять участников по имени пользователя или ФИО, назначать им роли редактора или наблюдателя, изменять роль и удалять участников из проекта.

7. Выход из проекта. Любой участник проекта, кроме владельца, должен иметь возможность самостоятельно покинуть проект. После выхода доступ к данным проекта утрачивается.

8. Создание, редактирование и удаление задач. Пользователи с ролью владельца или редактора должны иметь возможность создавать задачи в рамках проекта с указанием названия, описания, даты начала и даты завершения, редактировать и удалять их. Каждой задаче автоматически присваивается порядковый номер в рамках проекта. При нажатии на полосу задачи на диаграмме Ганта открывается детальная карточка задачи, в которой отображаются и редактируются все поля, а также список зависимостей задачи. Дата начала и дата завершения в карточке ограничены снизу и сверху датами проекта и ограничениями, вытекающими из установленных зависимостей. Порядок задач на диаграмме должен быть изменяем перетаскиванием строк по вертикали.

9. Управление зависимостями задач. Владелец и редактор должны иметь возможность устанавливать зависимости между задачами. Поддерживаются четыре типа зависимостей: финиш-старт (задача-преемник начинается после завершения предшественника), старт-старт (одновременное начало), финиш-финиш (одновременное завершение) и старт-финиш (редко применяемый тип, при котором завершение преемника привязано к началу предшественника). Система должна исключать создание зависимости задачи от самой себя и дублирующих пар. На диаграмме Ганта должен быть преду-

смотрен режим добавления связей: в этом режиме на торцах каждой полосы задачи появляются точки соединения, и зависимость создается перетаскиванием от одной точки к другой с автоматическим определением типа по выбранным торцам. Зависимости также должны быть доступны для добавления и удаления из карточки задачи.

10. Диаграмма Ганта. Система должна отображать задачи проекта в виде диаграммы Ганта: каждая задача представлена горизонтальным отрезком, позиция и длина которого определяются датами начала и завершения. На полосе задачи отображаются ее название и длительность в днях. Зависимости между задачами должны отображаться стрелками соответствующего типа. Даты задачи должны изменяться перетаскиванием непосредственно на диаграмме: горизонтальное перемещение полосы смещает обе даты с сохранением длительности, а перетаскивание левого или правого края изменяет дату начала или дату завершения соответственно. При перемещении или изменении размеров полосы задачи система должна автоматически ограничивать допустимый диапазон дат в соответствии с установленными зависимостями: например, при зависимости финиш-старт задача-преемник не может быть сдвинута раньше даты завершения предшественника. Временная шкала охватывает исключительно период от даты начала до даты завершения проекта. Пользователь должен иметь возможность изменять масштаб отображения в диапазоне от 50 до 400 % с помощью кнопок панели инструментов, прокрутки колесом мыши с удержанием Ctrl или жестом сведения пальцев на сенсорном экране. Выбранный масштаб должен сохраняться отдельно для каждого проекта. Должна быть предусмотрена возможность блокировки диаграммы: в заблокированном состоянии перетаскивание полос и изменение их размеров недоступны, однако просмотр и открытие карточек задач сохраняется.

11. Разграничение прав доступа. Предусмотрены три уровня доступа к данным проекта. Владелец – полный доступ, включая управление составом участников и параметрами проекта. Редактор – работа с задачами и зависи-

мостями без права изменять состав участников. Наблюдатель – только чтение данных проекта.

12. Валидация данных. Система должна выполнять проверку корректности данных, вводимых при регистрации, редактировании профиля, создании и изменении проекта, а также при работе с задачами.

## **2.4 Требования к интерфейсу**

Интерфейс реализован как одностраничное веб-приложение (SPA). Страницы переключаются без перезагрузки. После авторизации на всех экранах остаются шапка и боковая панель навигации.

В шапке – выпадающий список проектов пользователя с постраничной подгрузкой, кнопка создания нового проекта и кнопка добавления задачи в текущий проект. Последняя отображается только пользователям с ролью редактора или владельца.

Боковая панель содержит четыре кнопки перехода между разделами: диаграмма Ганта, профиль, проект, настройки. Панель можно свернуть до иконок, чтобы освободить место для рабочей области.

Основные разделы интерфейса:

- страница входа и регистрации – видна неавторизованным пользователям, после входа становится недоступной;
- страница диаграммы Ганта – основной рабочий экран с интерактивным отображением задач и зависимостей;
- страница проекта – редактирование параметров текущего проекта (название, описание, даты) и управление составом участников;
- страница профиля пользователя – просмотр и редактирование персональных данных учетной записи;
- страница настроек – управление цветовой темой интерфейса.

Интерфейс адаптирован под разные размеры экрана – от смартфонов до ПК. Раскладка перестраивается по ширине. Горизонтальная прокрутка и потеря функциональности на узких экранах исключены.

Оформление минималистичное. Декоративных элементов нет, функциональные зоны разделены, поведение элементов предсказуемо. Назначение каждого элемента понятно без подсказок. Навигация не уходит глубже двух уровней.

Интерактивные элементы откликаются на действия пользователя: видны состояния наведения, нажатия и недоступности. Цвета вынесены в CSS-переменные, поэтому цветовую схему можно менять без правок исходного кода. Шрифты, отступы и элементы управления одинаковы на всех страницах.

Внешний вид страницы входа и регистрации показан на рисунке 2.1.



Рисунок 2.1 – Страница входа в систему

При вводе пароля форма регистрации показывает список требований к нему в реальном времени: каждый выполненный пункт подсвечивается сразу. На рисунке 2.2 показан вид подсказки при частично введенном пароле.



## Регистрация

✕

Логин

Email

Пароль

••|

Недопустимый пароль

Подтверждение пароля

Зарегистрироваться

Требования к паролю:

✕ Пароль должен содержать не менее 8 символов

✓ Пароль должен содержать хотя бы одну заглавную букву

✓ Пароль должен содержать хотя бы одну строчную букву

✕ Пароль должен содержать хотя бы одну цифру

✕ Пароль должен содержать хотя бы один из спецсимволов: (! @ # \$ % ^ & \* ( ) - \_ = + [ ] { } | ; : , . < > ? / ~ ` ' " )

✓ Пароль должен содержать только допустимые символы: A-Z, a-z, 0-9, спецсимволы (! @ # \$ % ^ & \* ( ) - \_ = + [ ] { } | ; : , . < > ? / ~ ` ' " )

Рисунок 2.2 – Интерактивная подсказка к паролю при регистрации

Рисунок 2.3 иллюстрирует страницу диаграммы Ганта с отображением задач, зависимостей, линии текущего времени и штриховки истекшей части полос.

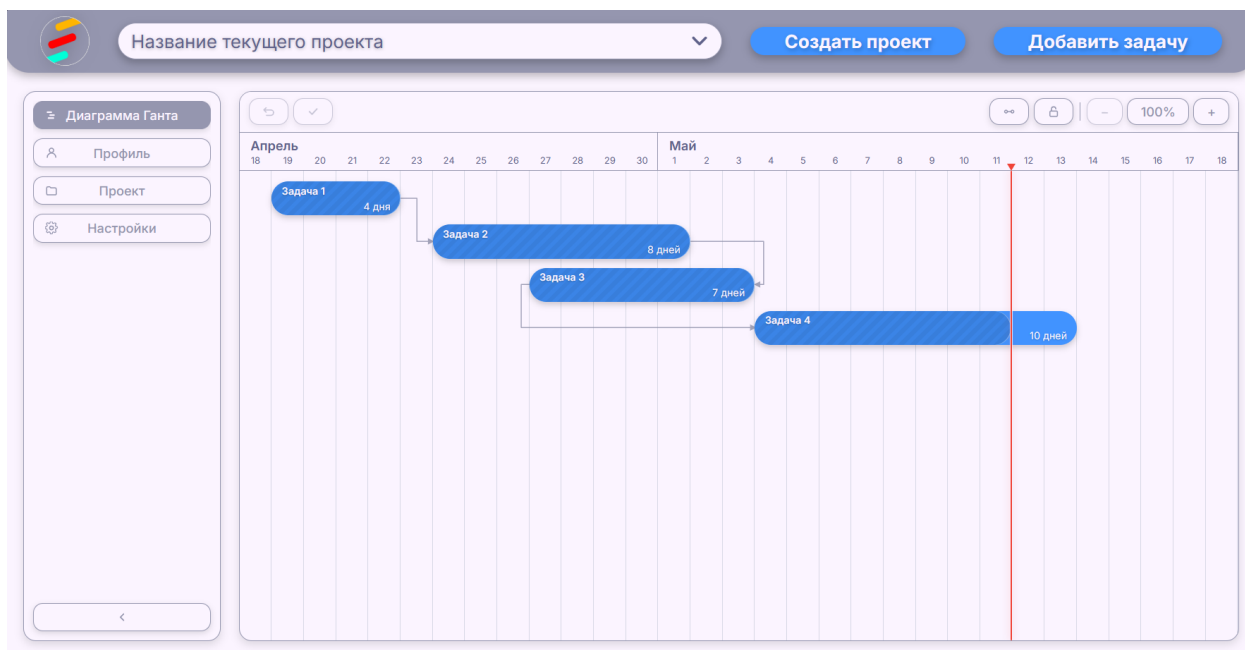


Рисунок 2.3 – Страница с диаграммой Ганта проекта

На диаграмме Ганта выводится вертикальная линия текущей даты – только если она попадает в период проекта. Истекшая часть каждой полосы задачи выделяется штриховкой.

При нажатии на полосу задачи открывается карточка с детальной информацией и элементами управления. На рисунке 2.4 показан вид карточки задачи.

↶

✓

🗑

✕

## Задача 3

27.04.2026 📅

—

03.05.2026 📅

7 дней

ОПИСАНИЕ

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia

⌵

ЗАВИСИТ ОТ

Завершается после завершения «Задача 2»

✕

+ Добавить связь

ОТ ЗАДАЧИ ЗАВИСЯТ

«Задача 4» начинается после начала текущей задачи

✕

+ Добавить связь

Рисунок 2.4 – Карточка задачи

Страница проекта с параметрами и составом участников приведена на рисунке 2.5.

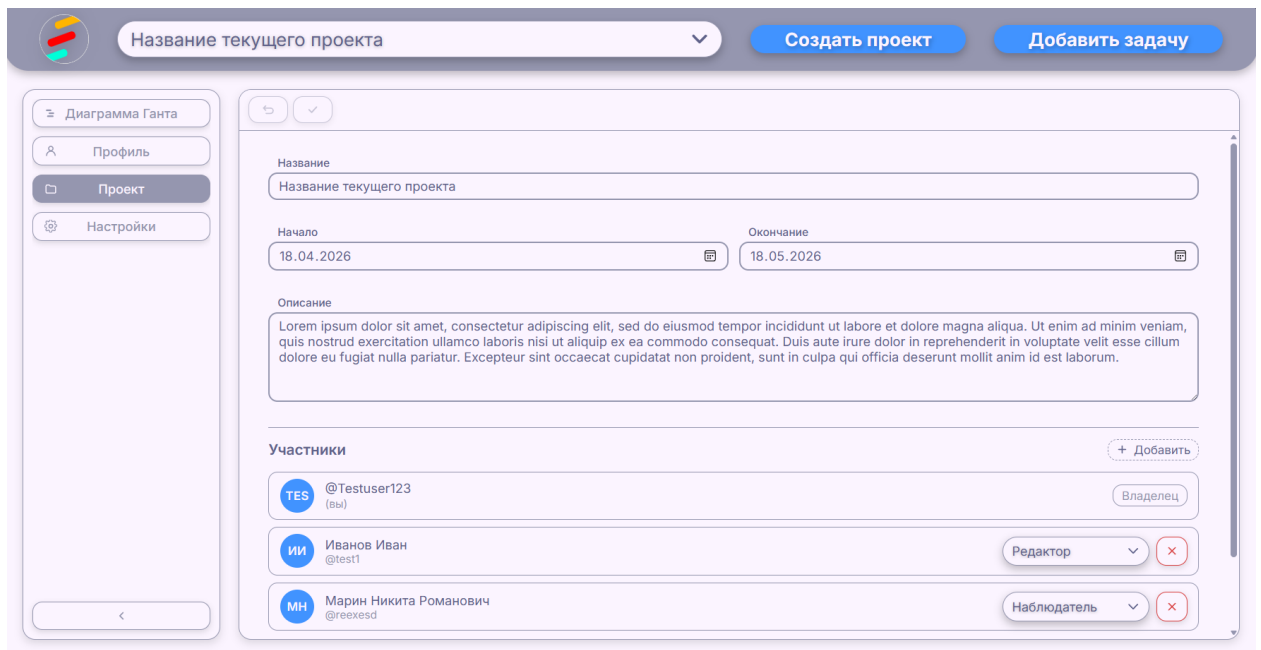


Рисунок 2.5 – Страница проекта

Страница настроек с элементами управления цветовой темой показана на рисунке 2.6.

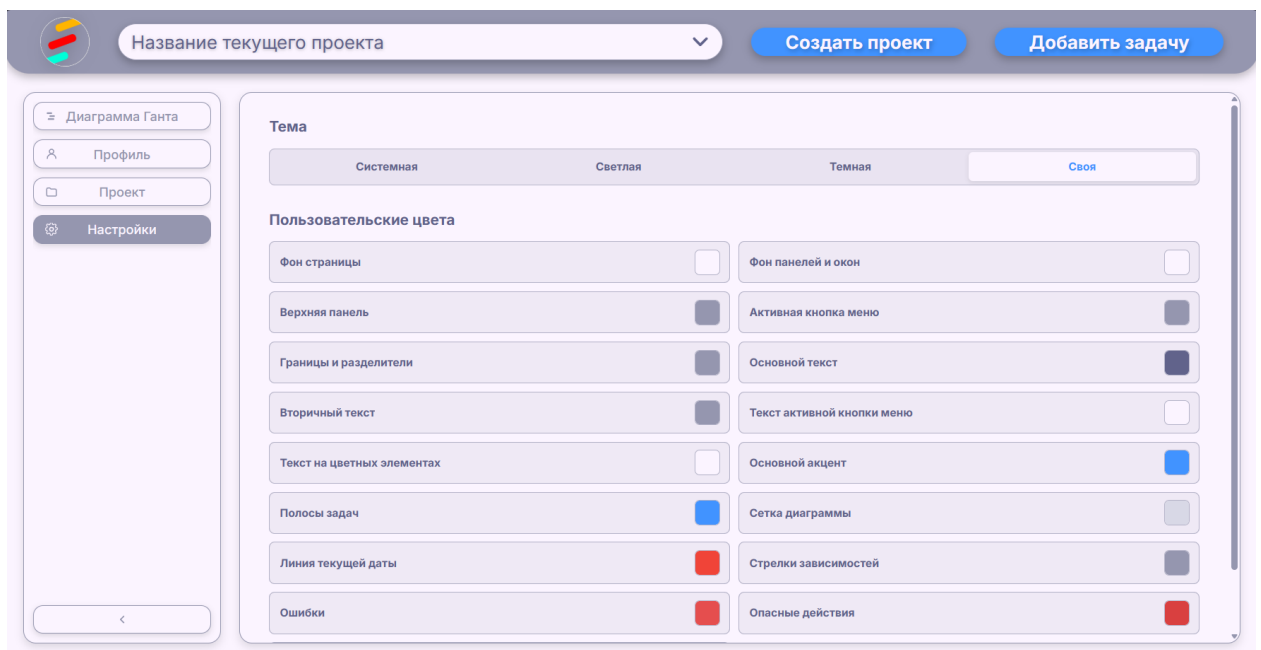


Рисунок 2.6 – Страница настроек

## 2.5 Моделирование вариантов использования

Модель прецедентов построена по ролям участников [46]. Диаграммы выполнены на языке UML [7].

В системе четыре роли. Гость – незарегистрированный посетитель, ему доступны только формы входа и регистрации. Наблюдатель сверх этого работает с реестром проектов, профилем, настройками темы и просматривает данные тех проектов, в которых он состоит. Редактор наследует права наблюдателя и управляет задачами – создает, редактирует, удаляет, меняет порядок и зависимости. Владелец обладает всеми правами редактора, а сверх того редактирует и удаляет проект, управляет составом команды и ролями участников.

На основании анализа предметной области в системе реализованы следующие прецеденты:

1. Регистрация пользователя.
2. Вход в систему.
3. Выход из системы.
4. Просмотр профиля пользователя.
5. Редактирование профиля пользователя.
6. Настройка цветовой темы интерфейса.
7. Просмотр списка проектов.
8. Создание проекта.
9. Редактирование параметров проекта.
10. Удаление проекта.
11. Добавление участника в проект.
12. Изменение роли участника.
13. Удаление участника из проекта.
14. Выход из проекта.
15. Просмотр диаграммы Ганта.
16. Создание задачи.
17. Редактирование задачи.
18. Удаление задачи.
19. Изменение порядка задач.
20. Создание зависимости между задачами.
21. Удаление зависимости между задачами.

Диаграмма прецедентов гостя приведена на рисунке 2.7.

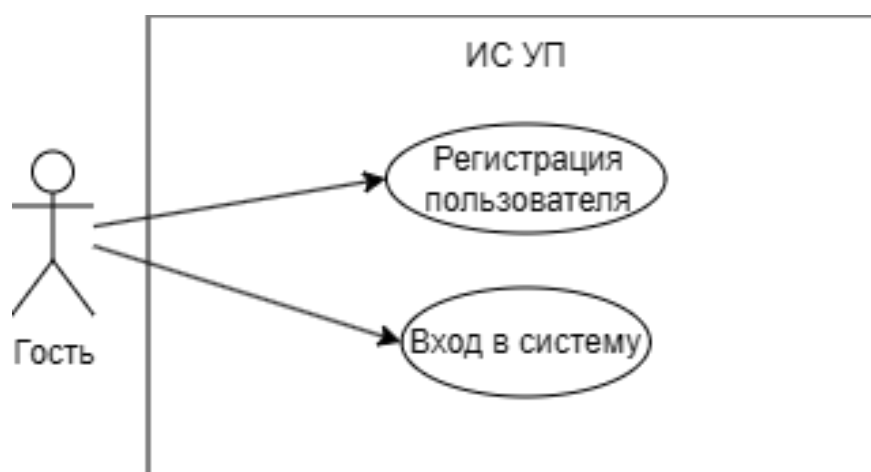


Рисунок 2.7 – Диаграмма прецедентов гостя

На рисунке 2.8 показана диаграмма прецедентов наблюдателя проекта.

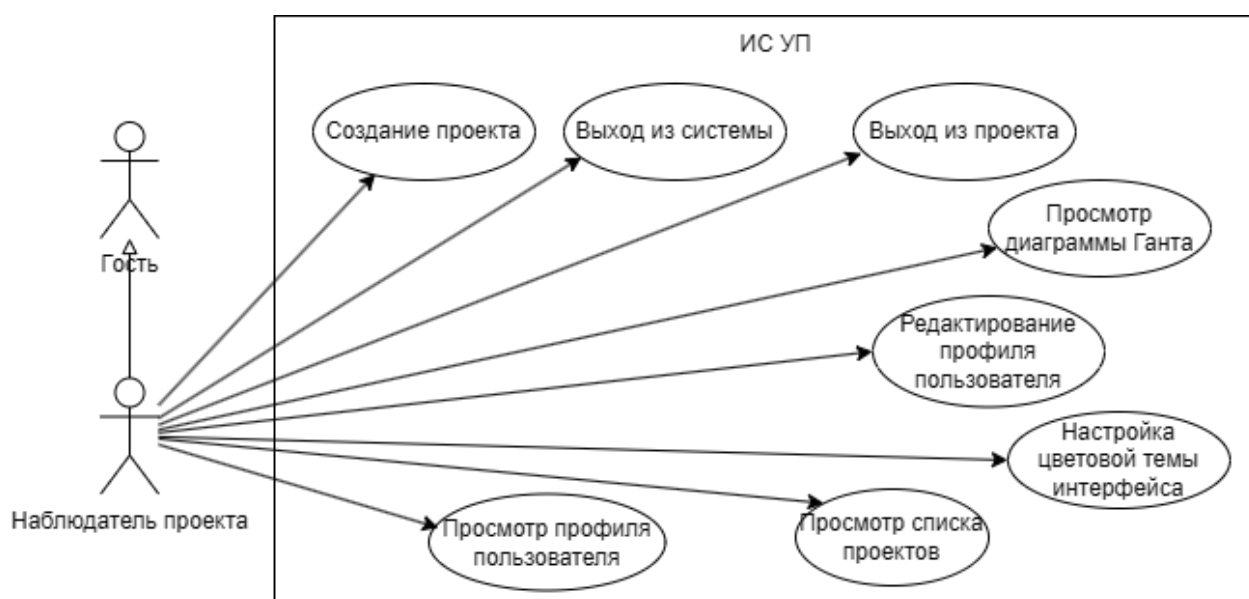


Рисунок 2.8 – Диаграмма прецедентов наблюдателя проекта

Рисунок 2.9 содержит диаграмму прецедентов редактора проекта.

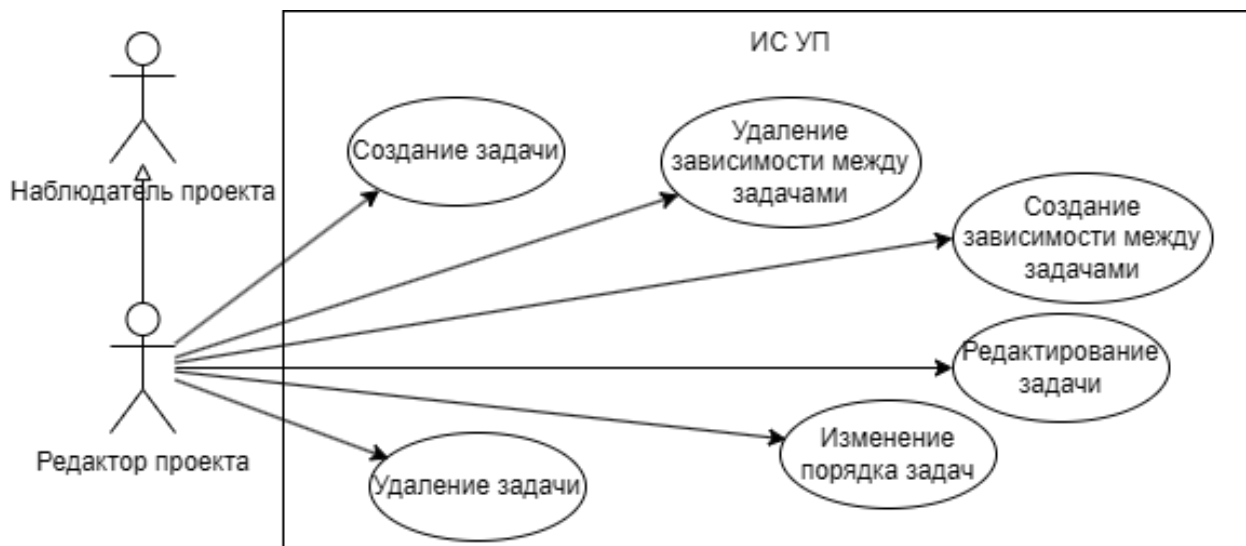


Рисунок 2.9 – Диаграмма прецедентов редактора проекта

Диаграмма прецедентов владельца проекта приведена на рисунке 2.10.

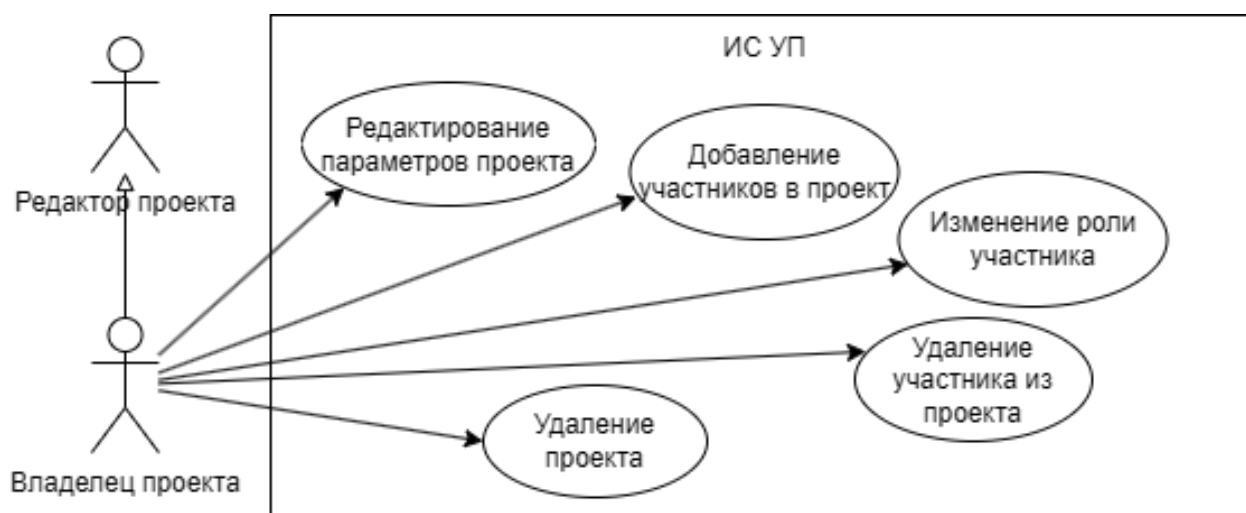


Рисунок 2.10 – Диаграмма прецедентов владельца проекта

### 2.5.1 Прецедент «Регистрация пользователя»

**Заинтересованные лица и их требования:** незарегистрированный пользователь хочет создать учетную запись для работы с системой.

**Предусловие:** пользователь не аутентифицирован в системе.

**Постусловие:** в системе создана новая учетная запись, пользователь выполнил вход.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «Зарегистрироваться» на стартовой странице.
2. Система открывает модальное окно регистрации с полями: логин, адрес электронной почты, пароль, подтверждение пароля.
3. Пользователь заполняет поля. При вводе пароля система в реальном времени отображает интерактивный список требований с индикацией выполнения каждого пункта.
4. Пользователь нажимает кнопку «Зарегистрироваться» в модальном окне.
5. Система проверяет уникальность логина и адреса электронной почты, соответствие пароля требованиям, совпадение паролей.
6. Система создает учетную запись с хешированным паролем.
7. Система выполняет вход и перенаправляет пользователя на страницу диаграммы Ганта.

#### **Альтернативный сценарий:**

1. На шаге 5 система обнаруживает, что логин или адрес электронной почты уже заняты либо пароль не удовлетворяет требованиям.
2. Система отображает сообщение об ошибке рядом с соответствующим полем.
3. Пользователь исправляет данные и повторяет отправку.

#### **2.5.2 Прецедент «Вход в систему»**

**Заинтересованные лица и их требования:** зарегистрированный пользователь намерен войти и продолжить работу с системой.

**Предусловие:** пользователь не аутентифицирован, открыта стартовая страница.

**Постусловие:** пользователь аутентифицирован, открыт сеанс работы, пользователь перенаправлен на страницу диаграммы Ганта.

#### **Основной успешный сценарий:**

1. Пользователь нажимает кнопку «Войти» на стартовой странице.
2. Система открывает модальное окно входа с полями: логин и пароль.



3. Пользователь вводит данные и нажимает кнопку «Войти».
4. Система проверяет существование учетной записи и корректность пароля.
5. Система открывает сеанс работы.
6. Пользователь перенаправляется на страницу диаграммы Ганта.

**Альтернативный сценарий:**

1. На шаге 4 система обнаруживает, что учетная запись не найдена или пароль неверен.
2. Система отображает сообщение об ошибке.
3. Пользователь остается на стартовой странице и может повторить попытку.

### **2.5.3 Прецедент «Выход из системы»**

**Заинтересованные лица и их требования:** пользователь намерен завершить текущий сеанс.

**Предусловие:** пользователь вошел в систему.

**Постусловие:** сеанс работы завершен, пользователь перенаправлен на стартовую страницу.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку выхода в интерфейсе приложения.
2. Система завершает сеанс работы пользователя.
3. Система перенаправляет пользователя на стартовую страницу.

### **2.5.4 Прецедент «Просмотр профиля пользователя»**

**Заинтересованные лица и их требования:** пользователь хочет посмотреть данные своей учетной записи.

**Предусловие:** пользователь авторизован.

**Постусловие:** на экране отображены данные профиля пользователя.

**Основной успешный сценарий:**

1. Пользователь выбирает раздел «Профиль» в боковой панели навигации.

2. Система отображает страницу профиля: имя пользователя, имя, фамилию, отчество и адрес электронной почты.

### **2.5.5 Прецедент «Редактирование профиля пользователя»**

**Заинтересованные лица и их требования:** пользователь собирается обновить данные учетной записи.

**Предусловие:** пользователь аутентифицирован, открыта страница собственного профиля.

**Постусловие:** обновленные данные сохранены в базе данных.

#### **Основной успешный сценарий:**

1. Пользователь переходит на страницу своего профиля.
2. Система отображает форму редактирования с полями: имя, фамилия, отчество, адрес электронной почты. Поле имени пользователя недоступно для изменения.
3. Пользователь изменяет нужные поля и подтверждает сохранение.
4. Система проверяет корректность данных.
5. Система сохраняет изменения и отображает обновленные данные.

#### **Альтернативный сценарий:**

1. На шаге 4 система обнаруживает, что адрес электронной почты уже используется другим пользователем.
2. Система отображает сообщение об ошибке.
3. Пользователь корректирует данные и повторяет сохранение.

### **2.5.6 Прецедент «Настройка цветовой темы интерфейса»**

**Заинтересованные лица и их требования:** пользователь хочет сменить цветовую тему интерфейса.

**Предусловие:** открыт авторизованный сеанс.

**Постусловие:** выбранная тема применена к интерфейсу.

#### **Основной успешный сценарий:**

1. Пользователь переходит в раздел «Настройки» через боковую панель навигации.

2. Система отображает страницу выбора режима темы: системный, светлый, темный, пользовательский.

3. Пользователь выбирает нужный режим. В пользовательском режиме дополнительно отображаются элементы выбора цвета для отдельных компонентов интерфейса.

4. Система применяет тему немедленно без перезагрузки страницы.

### **2.5.7 Прецедент «Просмотр списка проектов»**

**Заинтересованные лица и их требования:** пользователю нужно перейти к нужному проекту.

**Предусловие:** пользователь вошел в систему.

**Постусловие:** пользователю отображен список проектов, в которых он является участником.

**Основной успешный сценарий:**

1. Пользователь нажимает на выпадающий список проектов в шапке приложения.

2. Система загружает и отображает список проектов пользователя с страничной подгрузкой.

3. Пользователь выбирает проект для перехода к нему.

### **2.5.8 Прецедент «Создание проекта»**

**Заинтересованные лица и их требования:** пользователь хочет создать новый проект.

**Предусловие:** пользователь авторизован.

**Постусловие:** в системе создан новый проект, пользователь добавлен в него с ролью владельца.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку создания проекта в шапке приложения.

2. Система отображает форму с полями: название, описание, дата начала, дата завершения.

3. Пользователь заполняет поля и подтверждает создание.
4. Система проверяет корректность данных.
5. Система создает проект и назначает пользователю роль владельца.
6. Система переключает контекст на созданный проект и переходит к его диаграмме Ганта.

**Альтернативный сценарий:**

1. На шаге 4 система обнаруживает незаполненное обязательное поле или некорректные даты (дата завершения раньше даты начала).
2. Система отображает сообщение об ошибке.
3. Пользователь исправляет данные и повторяет создание.

### **2.5.9 Прецедент «Редактирование параметров проекта»**

**Заинтересованные лица и их требования:** владелец собирается изменить основные параметры проекта.

**Предусловие:** пользователь аутентифицирован, пользователь является владельцем проекта, открыта страница проекта.

**Постусловие:** обновленные параметры проекта сохранены в базе данных.

**Основной успешный сценарий:**

1. Владелец переходит на страницу проекта через боковую панель навигации.
2. Система отображает форму с текущими значениями: название, описание, дата начала, дата завершения.
3. Владелец изменяет нужные поля и сохраняет изменения.
4. Система проверяет корректность данных.
5. Система сохраняет изменения.

**Альтернативный сценарий:**

1. На шаге 4 система обнаруживает незаполненное обязательное поле или некорректный порядок дат.
2. Система отображает сообщение об ошибке.
3. Владелец исправляет данные и повторяет сохранение.

### 2.5.10 Прецедент «Удаление проекта»

**Заинтересованные лица и их требования:** владелец намерен удалить проект со всеми данными.

**Предусловие:** пользователь аутентифицирован, пользователь является владельцем проекта.

**Постусловие:** проект со всеми задачами, зависимостями и записями об участниках удален из системы.

**Основной успешный сценарий:**

1. Владелец на странице проекта нажимает кнопку удаления проекта.
2. Система отображает модальное окно подтверждения с пояснением о необратимости операции.
3. Владелец подтверждает удаление.
4. Система удаляет проект и все связанные с ним данные.
5. Если у пользователя остаются другие проекты, система переключает контекст на один из них; в противном случае создается новый пустой проект, и пользователь перенаправляется на его диаграмму Ганта.

**Альтернативный сценарий:**

1. На шаге 3 владелец закрывает модальное окно без подтверждения.
2. Система отменяет операцию, проект остается без изменений.

### 2.5.11 Прецедент «Добавление участника в проект»

**Заинтересованные лица и их требования:** владелец хочет добавить нового участника в проект.

**Предусловие:** пользователь аутентифицирован, пользователь является владельцем проекта, открыта страница проекта.

**Постусловие:** новый участник добавлен в проект с указанной ролью.

**Основной успешный сценарий:**

1. Владелец открывает раздел управления участниками на странице проекта.

2. Система отображает текущий состав участников и форму добавления.
3. Владелец вводит имя пользователя или ФИО добавляемого пользователя в поле поиска.
4. Система отображает список совпадений по имени пользователя и полному имени.
5. Владелец выбирает пользователя из списка и назначает ему роль: редактор или наблюдатель.
6. Система добавляет пользователя в проект с выбранной ролью.

**Альтернативный сценарий:**

1. На шаге 4 система не находит пользователей по введенному запросу.
2. Система отображает пустой список результатов.
3. Владелец уточняет запрос и повторяет поиск.

#### **2.5.12 Прецедент «Изменение роли участника»**

**Заинтересованные лица и их требования:** владелец намерен поменять роль одному из участников.

**Предусловие:** пользователь аутентифицирован, пользователь является владельцем проекта, открыта страница проекта.

**Постусловие:** роль участника обновлена в базе данных.

**Основной успешный сценарий:**

1. Владелец открывает раздел управления участниками на странице проекта.
2. Система отображает список участников с текущими ролями.
3. Владелец выбирает новую роль для нужного участника.
4. Система сохраняет изменение.

#### **2.5.13 Прецедент «Удаление участника из проекта»**

**Заинтересованные лица и их требования:** владелец собирается исключить участника из проекта.

**Предусловие:** пользователь аутентифицирован, пользователь является владельцем проекта, открыта страница проекта.

**Постусловие:** участник удален из проекта, доступ к его данным утрачен.

**Основной успешный сценарий:**

1. Владелец открывает раздел управления участниками на странице проекта.
2. Система отображает список участников.
3. Владелец нажимает кнопку удаления рядом с нужным участником.
4. Система отображает модальное окно подтверждения.
5. Владелец подтверждает удаление.
6. Система удаляет запись об участии пользователя в проекте.

#### **2.5.14 Прецедент «Выход из проекта»**

**Заинтересованные лица и их требования:** участник с ролью редактора или наблюдателя хочет самостоятельно покинуть проект.

**Предусловие:** пользователь аутентифицирован, пользователь является участником проекта с ролью редактора или наблюдателя.

**Постусловие:** запись об участии удалена, доступ к данным проекта утрачен.

**Основной успешный сценарий:**

1. Участник открывает раздел управления участниками на странице проекта.
2. Система отображает кнопку «Покинуть проект».
3. Участник нажимает кнопку.
4. Система удаляет запись об участии и перенаправляет пользователя.

#### **2.5.15 Прецедент «Просмотр диаграммы Ганта»**

**Заинтересованные лица и их требования:** участник намерен просмотреть план работ на диаграмме Ганта.

**Предусловие:** пользователь аутентифицирован и является участником хотя бы одного проекта.

**Постусловие:** на экране отображена интерактивная диаграмма Ганта с задачами, зависимостями и временной шкалой.

**Основной успешный сценарий:**

1. Пользователь выбирает проект в выпадающем списке шапки и переходит к разделу диаграммы через боковую панель.
2. Система загружает задачи и зависимости проекта.
3. Система строит временную шкалу в пределах дат проекта и отображает каждую задачу горизонтальной полосой. Позиция и длина полосы определяются датами начала и завершения задачи, на полосе выводятся название задачи и ее длительность в днях.
4. Система отображает зависимости стрелками, вертикальную линию текущего времени (если текущая дата входит в период проекта) и штриховку истекшей части каждой полосы.
5. Пользователь может изменять масштаб диаграммы от 50 до 400 % с помощью кнопок панели инструментов, прокрутки колесом мыши с удержанием Ctrl или жестом сведения пальцев. Масштаб сохраняется отдельно для каждого проекта.

#### **2.5.16 Прецедент «Создание задачи»**

**Заинтересованные лица и их требования:** редактор или владелец собирается добавить задачу в план проекта.

**Предусловие:** пользователь аутентифицирован, пользователь имеет роль редактора или владельца в текущем проекте.

**Постусловие:** задача создана, ей присвоен порядковый номер, она отображается на диаграмме Ганта.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку добавления задачи в шапке приложения.



2. Система отображает форму с полями: название, описание, дата начала, дата завершения.
3. Пользователь заполняет поля и подтверждает создание.
4. Система проверяет корректность данных.
5. Система создает задачу и присваивает ей следующий порядковый номер в рамках проекта.
6. Диаграмма Ганта обновляется: новая задача появляется в конце списка.

**Альтернативный сценарий:**

1. На шаге 4 система обнаруживает незаполненное обязательное поле или даты, выходящие за пределы дат проекта.
2. Система отображает сообщение об ошибке.
3. Пользователь исправляет данные и повторяет создание.

### **2.5.17 Прецедент «Редактирование задачи»**

**Заинтересованные лица и их требования:** редактор или владелец хочет отредактировать параметры задачи.

**Предусловие:** пользователь аутентифицирован, пользователь имеет роль редактора или владельца, диаграмма не заблокирована, задача существует в проекте.

**Постусловие:** измененные данные задачи сохранены, диаграмма Ганта обновлена.

**Основной успешный сценарий (через карточку задачи):**

1. Пользователь нажимает на полосу задачи на диаграмме Ганта.
2. Система открывает карточку задачи с полями: название, описание, дата начала, дата завершения. Поля дат ограничены датами проекта и ограничениями из установленных зависимостей.
3. Пользователь изменяет нужные поля и сохраняет.
4. Система проверяет корректность данных.
5. Система сохраняет изменения, диаграмма Ганта обновляется.

**Альтернативный сценарий (перетаскивание на диаграмме):**

1. Пользователь перетаскивает полосу задачи горизонтально: смещаются обе даты с сохранением длительности. Или перетаскивает левый край полосы для изменения даты начала, правый – для изменения даты завершения.

2. Система ограничивает допустимый диапазон перемещения согласно зависимостям.

3. После отпускания диаграмма отображает новые даты. В панели инструментов становятся доступны кнопки подтверждения и отката изменений.

4. Пользователь нажимает кнопку подтверждения.

5. Система отправляет измененные даты на сервер и обновляет диаграмму.

**Сценарий отката (перетаскивание на диаграмме):**

1. На шаге 4 пользователь нажимает кнопку отката.

2. Система возвращает задачи к исходным датам без обращения к серверу.

### **2.5.18 Прецедент «Удаление задачи»**

**Заинтересованные лица и их требования:** редактор или владелец намерен удалить задачу из проекта.

**Предусловие:** пользователь аутентифицирован, пользователь имеет роль редактора или владельца, задача существует в проекте.

**Постусловие:** задача удалена из проекта вместе со всеми ее зависимостями, диаграмма Ганта обновлена.

**Основной успешный сценарий:**

1. Пользователь открывает карточку задачи нажатием на полосу диаграммы.

2. Система отображает карточку с кнопкой удаления задачи.

3. Пользователь нажимает кнопку удаления.

4. Система отображает модальное окно подтверждения.

5. Пользователь подтверждает удаление.

6. Система удаляет задачу и все связанные зависимости.

7. Диаграмма Ганта обновляется.

### 2.5.19 Прецедент «Изменение порядка задач»

**Заинтересованные лица и их требования:** редактор или владелец хочет переставить задачи в другом порядке на диаграмме.

**Предусловие:** пользователь аутентифицирован, пользователь имеет роль редактора или владельца, в проекте существует не менее двух задач, диаграмма не заблокирована.

**Постусловие:** пользователь подтвердил изменения, порядковые номера задач обновлены на сервере, диаграмма отображает задачи в новом порядке.

#### **Основной успешный сценарий:**

1. Пользователь захватывает строку задачи за ручку перетаскивания в левой части строки.
2. Пользователь перемещает строку на новую позицию.
3. После отпущения диаграмма отображает обновленный порядок. В панели инструментов становятся доступны кнопки подтверждения и отката изменений.
4. Пользователь нажимает кнопку подтверждения.
5. Система отправляет измененный порядок задач на сервер и обновляет порядковые номера.

#### **Сценарий отката:**

1. На шаге 4 пользователь нажимает кнопку отката.
2. Система возвращает задачи к исходному порядку без обращения к серверу.

### 2.5.20 Прецедент «Создание зависимости между задачами»

**Заинтересованные лица и их требования:** редактор или владелец собирается создать зависимость между двумя задачами.

**Предусловие:** пользователь аутентифицирован, пользователь имеет роль редактора или владельца, в проекте существует не менее двух задач.

**Постусловие:** зависимость сохранена, на диаграмме Ганта появилась соответствующая стрелка.

**Основной успешный сценарий (через режим связей на диаграмме):**

1. Пользователь активирует режим добавления связей на панели инструментов диаграммы.
2. Система переходит в режим связей: на левом и правом торцах каждой полосы задачи появляются точки соединения.
3. Пользователь нажимает на точку соединения задачи-предшественника и тянет до точки задачи-преемника.
4. Система определяет тип зависимости по выбранным торцам: левый торец – старт, правый – финиш. Сочетание торцов однозначно задает тип связи (финиш-старт, старт-старт, финиш-финиш, старт-финиш).
5. Система сохраняет зависимость и отображает стрелку на диаграмме.

**Альтернативный сценарий (через карточку задачи):**

1. Пользователь открывает карточку задачи.
2. В разделе зависимостей пользователь выбирает связанную задачу из списка и задает тип зависимости.
3. Система сохраняет зависимость и обновляет диаграмму.

**Сценарий ошибки:**

1. Пользователь пытается создать зависимость задачи от самой себя или дублирующую пару с тем же типом.
2. Система отклоняет операцию и отображает сообщение об ошибке.

### **2.5.21 Прецедент «Удаление зависимости между задачами»**

**Заинтересованные лица и их требования:** редактор или владелец хочет удалить зависимость между задачами.

**Предусловие:** пользователь аутентифицирован, пользователь имеет роль редактора или владельца, зависимость между задачами существует.

**Постусловие:** зависимость удалена, стрелка на диаграмме Ганта исчезла.

**Основной успешный сценарий:**

1. Пользователь открывает карточку задачи.
2. В разделе зависимостей пользователь нажимает кнопку удаления рядом с нужной зависимостью.
3. Система удаляет зависимость и обновляет диаграмму.

## **2.6 Требования к надежности**

Целостность данных обеспечивается тем, что все данные хранятся в MySQL, а операции, затрагивающие несколько связанных таблиц (например, создание проекта с одновременным добавлением владельца в таблицу участников), выполняются в одной транзакции. Появление частично сохраненных или противоречивых записей исключено.

Сбой при выполнении операции не повреждает уже сохраненные данные – состояние базы остается прежним, а клиент получает сообщение об ошибке с причиной. Граничные случаи на стороне пользователя – повторная отправка формы, попытка добавить уже состоящего в проекте участника, даты в неверном порядке – обрабатываются без нарушения работоспособности.

Сервер проверяет все входящие данные. К проверкам относятся формат дат, длина строковых полей, уникальность пар (проект, пользователь) в таблице участников, отсутствие самозависимостей и дублирующих пар в таблице зависимостей.

Резервные копии базы создаются штатными средствами MySQL, восстановление выполняется из них без дополнительного инструментария.

## **2.7 Требования к информационной безопасности**

Все программные интерфейсы, кроме регистрации и входа, требуют действующего сеанса. Запросы без сеанса отклоняются с кодом 401. Для изменяющих операций требуется еще и действующий CSRF-токен – сервер отдает его клиенту отдельной cookie.

Пароли хранятся в виде хеша, алгоритм выбирается из встроенных механизмов Django. При входе сравниваются хеши, открытый пароль нигде не сохраняется.

Разграничение прав доступа выполнено на основе ролевой модели. Права проверяются на сервере до обработки каждого запроса, который требует повышенных привилегий. К данным проектов, в которых пользователь не состоит, доступа нет вообще – ни на чтение, ни на изменение. Каждый пользователь работает только со своей учетной записью, а возможность отредактировать чужой профиль, задачу или параметры проекта на сервере исключена.

Для защиты от внедрения SQL-кода [52, 38] все запросы к базе проходят через ORM Django с параметризованными запросами. Конкатенация пользовательского ввода в SQL-выражения не применяется.

При выходе сеанс закрывается на сервере, а cookie с идентификатором удаляется. Попытка использовать закрытый сеанс отклоняется.

## **2.8 Требования к программной и технической совместимости**

Операционная система. Серверная часть работает под Linux или Windows при наличии Python 3.10+ и MySQL 8.0+. Клиентская часть запускается в браузере – поддерживаются Windows, macOS, Linux, Android и iOS.

Аппаратные требования к серверу:

- процессор с тактовой частотой не ниже 2 ГГц;
- оперативная память объемом не менее 2 ГБ;
- наличие свободного дискового пространства для размещения программных файлов и базы данных;
- постоянный доступ к сети.

Программное обеспечение. Стек серверной части:

- язык программирования Python 3.10+;
- веб-фреймворк Django 6.0+;
- библиотека Django REST Framework 3.16+ для построения программного интерфейса;
- СУБД MySQL 8.0+.

На стороне клиента используются:

- библиотеки React 19+;

– языки HTML5, CSS3 и JavaScript.

Для работы с клиентской частью пользователю достаточно современного браузера с поддержкой HTML5 и JavaScript [51]: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari или иной совместимый.

Сетевые требования. Клиентская и серверная части обмениваются данными в формате JSON по протоколу HTTP. В производственной среде канал связи должен быть защищен HTTPS.

## **2.9 Требования к оформлению документации**

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102–77 [1] и ГОСТ 34.601–90 [2]. Качество разрабатываемого программного обеспечения оценивается по модели характеристик качества, установленной ГОСТ Р ИСО/МЭК 25010–2015 [4].

### **3 Технический проект**

#### **3.1 Общая характеристика проектного решения**

Система реализована как веб-приложение с клиент-серверной архитектурой [15]. Пользователь работает с ней из браузера, на компьютер ничего ставить не нужно. На сервере располагаются данные, бизнес-логика и проверки прав. Клиент отвечает за интерфейс и реакцию на действия пользователя.

Клиентская часть – одностраничное приложение (SPA) на React. Полная загрузка страницы происходит один раз, при первом входе. Далее браузер обращается к серверу только за конкретными данными, а сам интерфейс не собирается заново при каждом переходе между разделами. Трафика меньше, между разделами нет видимых пауз.

Серверная часть написана на Django с подключенной библиотекой DRF [50]. Программный интерфейс построен в стиле REST [22, 55]. У каждой основной сущности системы – проектов и задач, зависимостей между задачами, участников проекта и самих пользователей – свой набор маршрутов с типовыми операциями: получение, создание, изменение, удаление. Обмен данными идет в формате JSON по HTTP.

Постоянное хранилище – реляционная СУБД MySQL. Запросы к ней строит ORM-слой Django, никакой сборки SQL из пользовательского ввода в коде нет, и основная поверхность атак через SQL-инъекции при таком подходе закрыта [52].

В системе пять слоев ответственности [40]:

- уровень представления – компоненты React, страницы, маршрутизация, локальное состояние интерфейса;
- уровень обмена – модуль на стороне клиента, который формирует запросы к серверу и разбирает ответы;
- уровень программного интерфейса – представления и сериализаторы DRF, маршруты, фильтры, проверки прав;



- уровень бизнес-логики – модели Django, методы наборов запросов, проверки на стороне приложения, транзакции;
- уровень хранения – таблицы MySQL, индексы, ограничения целостности.

Вход в систему реализован на основе базового механизма сеансов в Django. После проверки логина и пароля сервер открывает сеанс и кладет его идентификатор в защищенную cookie. Браузер сам подставляет эту cookie ко всем последующим запросам. Изменяющие операции дополнительно подтверждаются CSRF-токеном, который сервер отдает отдельной cookie.

Общая схема организации системы приведена на рисунке 3.1.

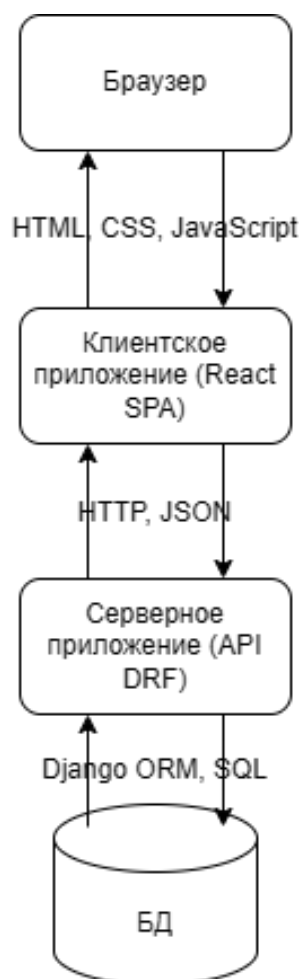


Рисунок 3.1 – Общая схема организации системы

В локальной разработке сервер поднимается через Uvicorn в ASGI-режиме, клиент – через сервер разработки React. На промышленном стенде

собранный клиентский код раздается как набор статических файлов, запросы к API идут по HTTPS.

## **3.2 Выбор средств и технологий реализации**

### **3.2.1 Состав применяемого технологического стека**

Для реализации системы выбран следующий набор инструментов:

- язык программирования Python версии 3.10 или выше – серверная часть;
- веб-фреймворк Django версии 6.0 – маршрутизация запросов, модели, сессии, проверки CSRF;
- библиотека Django REST Framework версии 3.16 – построение программного интерфейса в стиле REST;
- библиотека drf-nested-routers – вложенные маршруты вида `/projects/{id}/tasks`;
- библиотека django-filter – фильтрация коллекций по параметрам запроса;
- библиотека django-cors-headers – обработка кросс-доменных запросов в режиме разработки;
- библиотека React версии 19 – клиентская часть;
- библиотека react-router-dom версии 7 – маршрутизация на стороне клиента;
- СУБД MySQL версии 8.0 – постоянное хранилище;
- сервер приложений Uvicorn – запуск серверного процесса.

Стек подбирался с учетом нескольких соображений [30]. Главным был зрелостный критерий: Django, React и MySQL давно работают на боевых проектах в коммерческих компаниях и к экспериментальным технологиям не относятся, поэтому риск столкнуться с нерешенными проблемами фреймворка минимален. По каждой технологии есть подробная документация и крупное сообщество, что упрощает поиск ответов на вопросы реализации. Лицензионная сторона тоже устроена: все позиции бесплатны и распространяются

с открытым исходным кодом. Наконец, между собой эти технологии соединяются через стандартный канал – HTTP с телами в JSON – что позволяет заменить любую часть на аналог без переписывания остальных.

### **3.2.2 Серверный язык программирования и фреймворк Django**

Серверный язык – Python [26]. Под него есть пакеты практически на любую задачу – от работы с базами данных до сетевого взаимодействия. По скорости Python уступает компилируемым языкам, но в данном случае это не узкое место: основное время уходит на запросы к базе и сериализацию ответов.

Django существует с 2005 года и за это время вобрал в себя почти все, что нужно типичному веб-приложению [49, 14], в частности:

- ORM с миграциями – модели описываются как классы Python, миграции базы создаются автоматически;
- система аутентификации с сессиями и хешированием паролей;
- защита от CSRF, XSS и кликджекинга на уровне промежуточных обработчиков;
- встроенная проверка входных данных и система форм;
- панель администратора для просмотра и редактирования записей.

Инфраструктурный код в основном уже написан внутри Django – разработчику остается прикладная логика.

### **3.2.3 Django REST Framework для построения программного интерфейса**

DRF добавляет к Django средства для построения REST-интерфейсов. В разработке пригодились следующие его компоненты:

- классы-наборы `ModelViewSet` – для типового набора операций над сущностями;
- сериализаторы `ModelSerializer` – двусторонняя сборка JSON-представлений из моделей Django и обратно;

- классы разрешений – проверка прав на уровне запросов и отдельных действий;
- страничный вывод – для списков проектов через стандартный класс `PageNumberPagination`;
- фильтрация через `DjangoFilterBackend` – набор параметров запроса описывается как класс `FilterSet`.

За счет DRF шаблонного кода становится заметно меньше. Например, операции над проектами реализованы классом `ProjectViewSet`, унаследованным от `ModelViewSet`. Из дописанного руками – переопределенный `get_queryset`, который ограничивает выборку доступными пользователю проектами, `perform_create` с автоматическим добавлением создателя в участники с ролью владельца, а также `perform_update` и `perform_destroy` с проверкой роли владельца.

### 3.2.4 Клиентская библиотека React

React [54] – библиотека, на которой собрана клиентская часть. Среди инструментов для интерфейсов на JavaScript она самая распространенная. Компоненты в коде описаны как функции, состояние и побочные эффекты подключаются через хуки `useState`, `useEffect`, `useCallback`, `useMemo`. Глобальное состояние, общее для разных страниц – текущий пользователь, список доступных проектов, цветовая тема – вынесено в три контекста: `AuthContext`, `ProjectsContext`, `ThemeContext`. Маршрутизация подключена через `react-router-dom`. Разделы интерфейса привязаны к маршрутам вида `/projects/{id}/gant`, `/users/{username}`, `/settings`; переход между ними не вызывает перезагрузку документа.

Стили оформлены как CSS-модули: у каждого компонента свой файл со стилями, классы локальны и не конфликтуют между компонентами. Цвета вынесены в CSS-переменные, поэтому смена темы происходит без пересборки приложения.

### **3.2.5 СУБД MySQL**

MySQL – реляционная СУБД с открытым исходным кодом, одна из самых распространенных среди свободно доступных решений. Из возможностей MySQL 8.0 пригодились проверочные ограничения на уровне таблицы. Они нужны для того, чтобы запретить самозависимость задачи и дублирующие пары зависимостей прямо в схеме базы.

Схема управляется миграциями Django, ручное редактирование таблиц запрещено. Кодировка utf8mb4 поддерживает кириллические заголовки задач и проектов.

## **3.3 Архитектурное построение системы**

### **3.3.1 Уровневая модель архитектуры**

Код в системе разнесен по уровням, у каждого – своя зона ответственности. Уровень обращается только к следующему за ним, и за счет этого проще локализовать ошибки и заменять отдельные части. Уровневая схема приведена на рисунке 3.2.

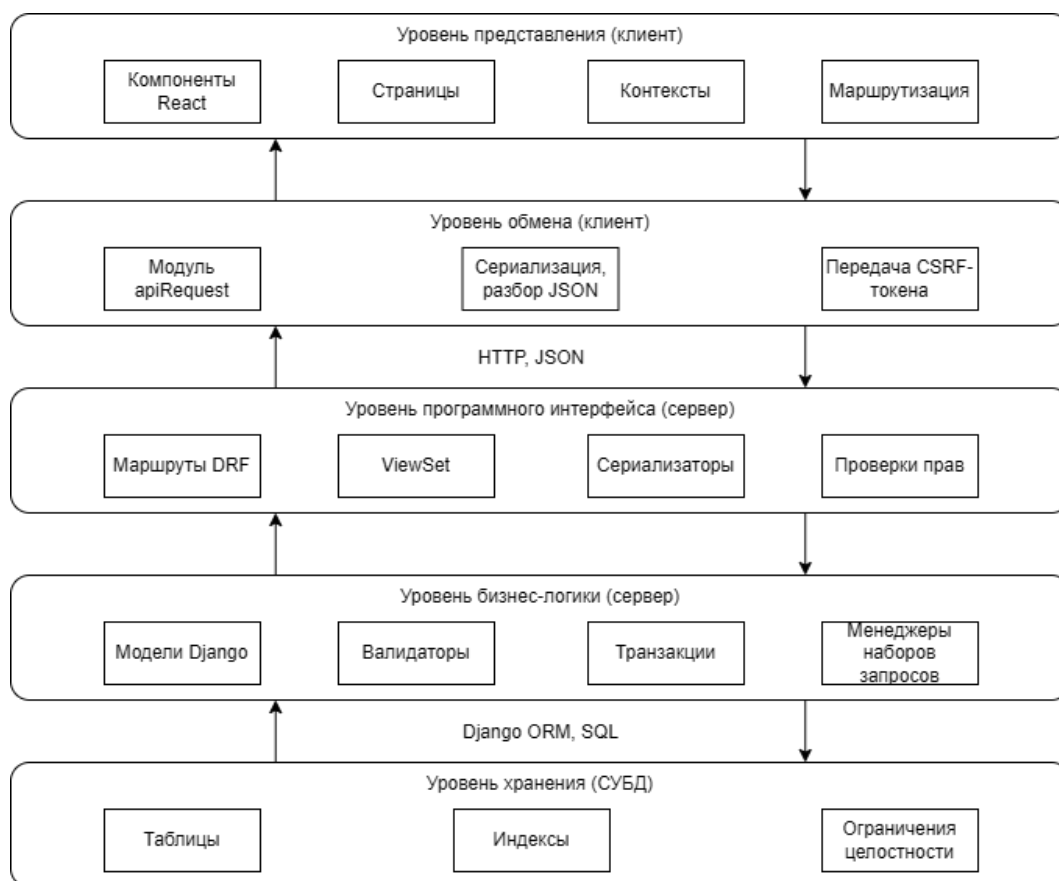


Рисунок 3.2 – Уровневая схема архитектуры системы

Из пяти уровней первые два работают в браузере, следующие два – на сервере, нижний представляет собой саму базу данных. Граница между клиентом и сервером проходит по HTTP-вызову. На клиенте уровень обмена упаковывает запрос в JSON и отправляет его на сервер, где уровень программного интерфейса принимает и разбирает этот запрос. Между бизнес-логикой и хранением встроен Django ORM, поэтому ни одна строка прикладного кода не собирает SQL вручную.

### 3.3.2 Компоненты клиентской части

На клиенте код собран в три группы файлов:

- компоненты – многоразовые элементы интерфейса (Button, Input, Modal, Select), а также крупные блоки (GanttChart, Header, Sidebar, TaskCard, ProjectMembers, SettingsPanel);
- страницы – сборочные единицы маршрутов (GanttPage, ProjectPage, ProfilePage, SettingsPage, WelcomePage);

– контексты – хранилища глобального состояния (AuthContext, ProjectsContext, ThemeContext).

AuthContext держит данные текущего пользователя и операции входа, выхода и регистрации. Через ProjectsContext страницам доступен список проектов пользователя, идентификатор последнего открытого проекта и операции создания, удаления, обновления. Отдельно стоит ThemeContext, отвечающий за оформление: он переключает режим темы (системный, светлый, темный, пользовательский), хранит набор цветов и применяет их к корневому элементу документа через CSS-переменные.

Большая часть клиентского кода приходится на компонент GanttChart. Он отвечает за всю отрисовку диаграммы – временной шкалы проекта, полос задач, стрелок зависимостей, линии текущей даты, – а также обрабатывает перетаскивание. Полосы выкладываются на сетку, шаг которой пересчитывается под выбранный масштаб в диапазоне 50–400 %. Стрелки зависимостей четырех типов соединяют точки на торцах полос.

### 3.3.3 Компоненты серверной части

Серверное приложение Django поделено на несколько пакетов:

- models – описания таблиц как классов Django: User, Project, Task, TaskDependency, ProjectMember, UserColorPalette;
- serializers – сериализаторы DRF, по одному файлу на сущность;
- views – классы-наборы операций над сущностями;
- filters – классы фильтров для списочных запросов;
- validators – проверки, не привязанные к одной модели (проверка пароля, проверка циклов в графе зависимостей задач);
- permissions – функции и классы проверки прав доступа;
- mixins – общие фрагменты поведения для наборов представлений.

Маршруты собираются через DefaultRouter из DRF и NestedDefaultRouter из drf-nested-routers. На верхнем уровне доступны /auth, /users, /projects, /tasks. Под маршрутом проекта вложены /projects/{id}/tasks, /projects/{id}/dependencies, /projects/{id}/members.

Права доступа проверяются на двух уровнях. Глобальный уровень требует аутентификации для всех маршрутов, кроме регистрации, входа и получения CSRF-токена. Уровень проекта смотрит, состоит ли пользователь в проекте и какая у него роль. Логика собрана в функциях `is_member`, `is_writer`, `is_owner` модуля `permissions`. Каждый набор представлений вызывает их в методе `initial` – до того, как выполнится само действие.

### **3.3.4 Компоненты уровня доступа к данным**

Все обращения к базе идут через Django ORM [32]. Таблицы описаны классами-наследниками `models.Model`, поля – атрибутами класса. Связи задаются через `ForeignKey`, `OneToOneField` и обратные отношения с параметром `related_name`. Ограничения целостности – уникальность пар, запрет самозависимости задачи – прописаны в `Meta.constraints` и переходят в схему базы как ограничения на уровне таблицы.

Если одна операция меняет несколько таблиц сразу, она оборачивается в `transaction.atomic`. Так, при создании проекта в одной транзакции выполняются запись самого проекта и добавление автора в таблицу участников с ролью владельца. Сбой на любом шаге откатывает обе записи.

Часто исполняемые запросы дополнены `select_related` (для связей по внешнему ключу) и `prefetch_related` (для обратных отношений). Эти подсказки ORM меняют стратегию выборки: вместо отдельного запроса за участниками на каждый проект из списка и за зависимостями на каждую задачу выполняется одно объединение, покрывающее всю коллекцию.

### **3.3.5 Диаграмма последовательности обработки запроса**

Последовательность обработки пользовательского действия показана на рисунке 3.3. Диаграмма последовательности построена в нотации UML [39]. На ней отдельно выделены браузер, React-приложение, серверный слой Django + DRF, ORM и СУБД MySQL – между этими участниками меняется форма представления данных. Событие интерфейса превращается в HTTP-



запрос, затем в операции над моделями и SQL-запросы, а после выполнения возвращается клиенту как JSON-ответ.

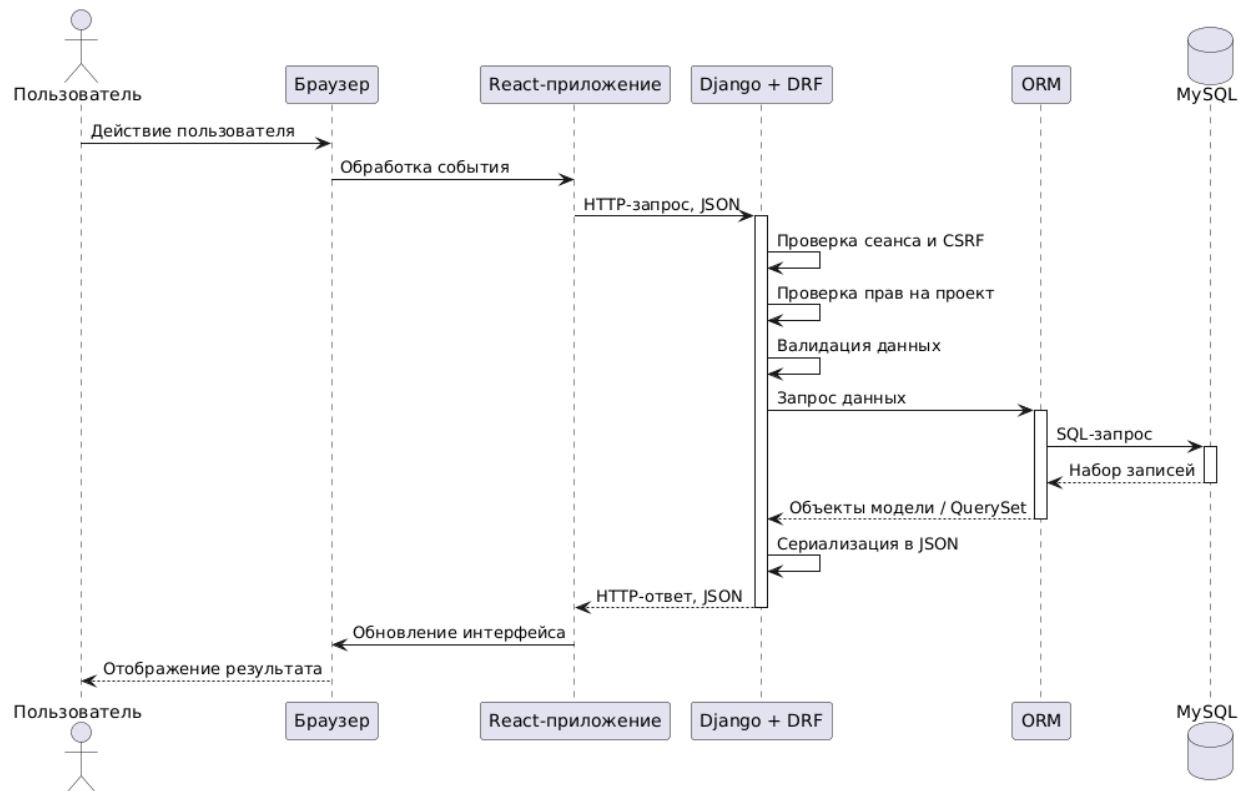


Рисунок 3.3 – Последовательность обработки запроса пользователя

Шаги диаграммы:

1. Пользователь выполняет действие в браузере: открывает список, сохраняет карточку задачи, меняет порядок работ или создает зависимость.
2. Браузер передает событие React-приложению. Клиентский код обрабатывает событие, собирает параметры операции и формирует HTTP-запрос с телом в формате JSON.
3. Запрос отправляется в серверную часть Django + DRF. Для изменяющих операций браузер прикладывает cookie сессии и CSRF-токен, полученные при входе в систему.
4. Сервер проверяет сессию пользователя и корректность CSRF-токена. Если сессия отсутствует или токен неверен, дальнейшая обработка прекращается.

5. После аутентификации выполняется проверка прав на проект: сервер определяет, входит ли пользователь в состав проекта и достаточно ли его роли для выбранного действия.

6. DRF-сериализатор и прикладные валидаторы проверяют входные данные. На этом этапе отбрасываются некорректные типы, отсутствующие обязательные поля, недопустимые роли и нарушения правил предметной области.

7. Если запрос требует чтения или изменения данных, сервер обращается к ORM. ORM строит SQL-запросы по моделям Django и передает их в MySQL.

8. MySQL выполняет SQL-запрос и возвращает набор записей. ORM преобразует результат в объекты моделей или QuerySet, с которыми работает серверный код.

9. DRF сериализует полученные объекты в JSON и возвращает HTTP-ответ React-приложению.

10. React-приложение обновляет локальное состояние интерфейса. Браузер перерисовывает изменившиеся элементы страницы и показывает пользователю результат операции без полной перезагрузки документа.

### **3.3.6 Форматы обмена данными**

Тела запросов и ответов – всегда JSON. Для календарных полей клиент передает дату строкой в формате «ГГГГ-ММ-ДД». Сервер хранит эти значения как DateTimeField и в ответах возвращает ISO-строки – клиент использует из них только календарную часть. Идентификаторы записей передаются целыми числами.

Списочные ресурсы приходят страницами по 25 записей. Формат страницы стандартный для DRF – объект с полями count, next, previous, results, где results – массив записей текущей страницы, а next и previous – ссылки на соседние.

При ошибках сервер возвращает JSON-объект с полем detail или словарь с пояснениями по полям. HTTP-коды соответствуют семантике REST:

200 – успешное чтение, 201 – создание, 204 – успешное действие без тела ответа, 400 – ошибка валидации, 401 – нет аутентификации, 403 – недостаточно прав, 404 – ресурс не найден.

### 3.3.7 Аутентификация и авторизация

Аутентификация работает через стандартный механизм сеансов Django. Сценарий входа выглядит так:

1. Перед вызовом изменяющих операций клиент запрашивает CSRF-токен по маршруту `/auth/csrf`. Сервер ставит cookie `csrftoken`.
2. Клиент отправляет логин и пароль на `/auth/login`. Сервер вызывает функцию `authenticate` Django, проверяет хеш пароля, открывает сеанс работы и устанавливает cookie `sessionid`.
3. При каждом последующем запросе браузер прикладывает обе cookie. Для изменяющих методов клиент дополнительно передает значение CSRF-токена в заголовке `X-CSRFToken`.

Пароли хранятся в виде хеша. По умолчанию Django вычисляет их через PBKDF2 с солью [21]. Открытый пароль нигде не сохраняется.

Проверка прав на ресурс делится на два этапа. Сначала DRF убеждается, что у запроса есть действующий сеанс, иначе возвращает 401. После этого серверный код проверяет, состоит ли пользователь в нужном проекте и какая у него роль. От роли зависит набор разрешенных действий: владельцу доступны все операции, включая редактирование самого проекта, удаление его и управление составом участников. У редактора есть права на задачи и зависимости, но менять состав команды он не может. Наблюдатель в проект только смотрит.

Сложность пароля контролирует собственный валидатор `PasswordStrengthValidator`. Минимальная длина – восемь символов. В пароле должны быть хотя бы одна строчная латинская буква, одна заглавная, одна цифра и один спецсимвол из заранее заданного набора. Кириллица и пробелы запрещены.

### 3.3.8 Описание программного интерфейса

Программный интерфейс размещен под префиксом `/api` и не использует завершающий слеш в маршрутах. Ниже приведены основные эндпоинты, тела запросов и ответы сервера. Для изменяющих методов POST, PATCH, PUT и DELETE браузер передает cookie сеанса и заголовок X-CSRFToken. В примерах показаны значимые поля JSON. Списочные запросы по ресурсам users, projects и вложенному списку tasks проекта возвращаются в стандартной постраничной обертке DRF с полями count, next, previous и results по 25 записей на странице. Списки зависимостей задач, участников проекта и задач пользователя (GET `/api/tasks`) приходят целиком в виде массива без пагинации — объем этих коллекций ограничен размером отдельного проекта.

#### Получение CSRF-токена.

GET `/api/auth/csrf`

Запрос не содержит тела. Сервер устанавливает cookie csrftoken и возвращает код 204 No Content.

#### Вход пользователя.

POST `/api/auth/login`

Тело запроса:

```
{
  "username": "ivanov.i",
  "password": "Password1!"
}
```

Ответ 200 OK:

```
{
  "id": 3,
  "full_name": "Иванов Иван Иванович",
  "first_name": "Иван",
  "middle_name": "Иванович",
  "last_name": "Иванов",
  "username": "ivanov.i",
  "email": "ivanov.i@example.com",
  "color_palette": {
    "theme_mode": "system",
    "colors": {
      "appBackground": "#fbf4ff",
      "surface": "#fbf4ff",
      "headerBackground": "#9596af",
    }
  }
}
```

```

    "activeMenuBackground": "#9596af",
    "border": "#9596af",
    "primaryText": "#61638B",
    "secondaryText": "#9596af",
    "activeMenuText": "#fbf4ff",
    "inverseText": "#fbf4ff",
    "accent": "#4193ff",
    "task": "#4193ff",
    "gantGridLine": "#d8d8e6",
    "gantCurrentTime": "#f04438",
    "gantArrow": "#9596af",
    "error": "#e54e4e",
    "danger": "#d94040",
    "success": "#4caf50"
  }
}
}

```

### **Выход пользователя.**

POST /api/auth/logout

Запрос не содержит тела. Сервер завершает сеанс и возвращает код 204 No Content.

### **Регистрация пользователя.**

POST /api/users

Создает новую учетную запись пользователя. Тело запроса:

```

{
  "first_name": "Иван",
  "last_name": "Иванов",
  "middle_name": "Иванович",
  "username": "ivanov.i",
  "email": "ivanov.i@example.com",
  "password": "Password1!"
}

```

Ответ 201 Created совпадает по структуре с ответом входа пользователя: сервер возвращает созданную запись пользователя с вложенной палитрой color\_palette. Поле password в ответ не включается.

### **Получение текущего пользователя.**

GET /api/users/me

Запрос не содержит тела. Ответ 200 OK совпадает по структуре с объектом пользователя из ответа /api/auth/login.

### **Получение и изменение пользователя по id.**

GET /api/users/{id}

Запрос не содержит тела. Ответ 200 OK содержит объект пользователя. Обычный пользователь может получить только свою запись, суперпользователь – любую.

PATCH /api/users/{id}

Обновляет поля профиля пользователя. Тело запроса содержит изменяемые поля профиля:

```
{
  "first_name": "Иван",
  "last_name": "Петров",
  "middle_name": "Иванович",
  "email": "petrov.i@example.com"
}
```

Ответ 200 OK содержит обновленный объект пользователя. Поле username после создания не изменяется.

### **Список и поиск пользователей.**

GET /api/users?search=ив&page=1

Возвращает постраничный список пользователей. Поддерживается параметр search – фильтрует по началу полного имени или логина. Запрос не содержит тела. Ответ 200 OK возвращается постранично:

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 3,
      "full_name": "Иванов Иван Иванович",
      "first_name": "Иван",
      "middle_name": "Иванович",
      "last_name": "Иванов",
      "username": "ivanov.i",
      "email": "ivanov.i@example.com",
      "color_palette": { "theme_mode": "system", "colors": {} }
    }
  ]
}
```

GET /api/users/search?q=иван

Этот маршрут используется при добавлении участников проекта. Запрос не содержит тела. Ответ 200 ОК – массив не более чем из десяти кратких записей:

```
[
  {
    "id": 3,
    "username": "ivanov.i",
    "full_name": "Иванов Иван Иванович"
  }
]
```

### Цветовая палитра текущего пользователя.

GET /api/users/color-palette

Возвращает цветовую палитру текущего пользователя. Запрос не содержит тела. Ответ 200 ОК:

```
{
  "theme_mode": "custom",
  "colors": {
    "appBackground": "#fbf4ff",
    "surface": "#ffffff",
    "headerBackground": "#9596af",
    "activeMenuBackground": "#9596af",
    "border": "#d8d8e6",
    "primaryText": "#61638B",
    "secondaryText": "#9596af",
    "activeMenuText": "#fbf4ff",
    "inverseText": "#fbf4ff",
    "accent": "#4193ff",
    "task": "#4193ff",
    "ganttGridLine": "#d8d8e6",
    "ganttCurrentTime": "#f04438",
    "ganttArrow": "#9596af",
    "error": "#e54e4e",
    "danger": "#d94040",
    "success": "#4caf50"
  }
}
```

PATCH /api/users/color-palette

Обновляет цветовую палитру текущего пользователя. Тело запроса может содержать theme\_mode и любой поднабор цветов:

```
{
  "theme_mode": "custom",
  "colors": {
    "appBackground": "#ffffff",
    "task": "#2f80ed",
  }
}
```

```
}
  "ganttArrow": "#61638B"
}
```

Ответ 200 OK содержит всю обновленную палитру.

### **Список проектов.**

GET /api/projects?page=1&search=разработка&overdue=false

Запрос не содержит тела. Поддерживаются параметры search, creator\_id, overdue и page. Ответ 200 OK:

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 43,
      "title": "Разработка ИС",
      "description": "Описание проекта",
      "start_date": "2026-02-01T00:00:00Z",
      "end_date": "2026-05-15T00:00:00Z",
      "creator_id": 3,
      "current_role": "owner"
    }
  ]
}
```

### **Создание проекта.**

POST /api/projects

Создает новый проект. Тело запроса:

```
{
  "title": "Разработка ИС",
  "description": "Описание проекта",
  "start_date": "2026-02-01",
  "end_date": "2026-05-15"
}
```

Ответ 201 Created содержит объект проекта. Сервер заполняет creator\_id и current\_role, а также создает запись участника с ролью owner для автора проекта.

### **Получение, изменение и удаление проекта.**

GET /api/projects/{id}

Запрос не содержит тела. Ответ 200 OK содержит объект проекта.



PATCH /api/projects/{id}

Обновляет поля проекта. Тело запроса содержит изменяемые поля проекта:

```
{
  "title": "Разработка ИС",
  "end_date": "2026-06-01"
}
```

Ответ 200 OK содержит обновленный объект проекта. Метод PUT использует такой же набор полей, но ожидает полную запись проекта. Удаление выполняется запросом DELETE /api/projects/{id} без тела и возвращает 204 No Content.

### Список задач пользователя.

GET /api/tasks?search=проектирование&overdue=false

Запрос не содержит тела. Маршрут возвращает задачи, созданные текущим пользователем; для суперпользователя доступен полный список. Ответ 200 OK – массив задач:

```
[
  {
    "id": 7,
    "title": "Проектирование API",
    "description": "Описать REST API",
    "start_date": "2026-02-10T00:00:00Z",
    "end_date": "2026-02-15T00:00:00Z",
    "number": 1,
    "creator_id": 3,
    "project_id": 43
  }
]
```

### Задачи проекта.

GET /api/projects/{project\_id}/tasks?page=1

Запрос не содержит тела. Поддерживаются параметры search, creator\_id, overdue и page. Ответ 200 OK возвращается постранично:

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 7,
```

```
{
  "title": "Проектирование API",
  "description": "Описать REST API",
  "start_date": "2026-02-10T00:00:00Z",
  "end_date": "2026-02-15T00:00:00Z",
  "number": 1,
  "creator_id": 3,
  "project_id": 43
}
```

POST /api/projects/{project\_id}/tasks

Создает задачу в проекте. Тело запроса:

```
{
  "title": "Проектирование API",
  "description": "Описать REST API",
  "start_date": "2026-02-10",
  "end_date": "2026-02-15"
}
```

Ответ 201 Created содержит созданную задачу. Поля id, number, creator\_id и project\_id назначаются сервером.

### Получение, изменение и удаление задачи.

Задачу можно адресовать по глобальному идентификатору /api/tasks/{id} или по номеру внутри проекта /api/projects/{project\_id}/tasks/{number}. Для GET тело не передается, ответ 200 OK содержит объект задачи.

Для изменения используется PATCH или PUT:

```
{
  "title": "Проектирование API",
  "end_date": "2026-02-20"
}
```

Ответ 200 OK содержит обновленную задачу. Удаление выполняется запросом DELETE без тела и возвращает 204 No Content.

### Изменение порядка задач.

POST /api/projects/{project\_id}/tasks/move

Меняет порядок задач в проекте: перемещает задачу после указанной.

Тело запроса:

```
{
  "task_id": 7,
```

```
"after_id": 12
}
```

Поле `after_id` может быть `null`, тогда задача переносится в начало списка. Ответ – 204 No Content.

### **Зависимости задач.**

GET `/api/projects/{project_id}/dependencies`

Запрос не содержит тела. Ответ 200 OK – массив зависимостей проекта:

```
[
  {
    "id": 5,
    "predecessor_id": 7,
    "successor_id": 8,
    "type": "FS"
  }
]
```

POST `/api/projects/{project_id}/dependencies`

Создает зависимость между двумя задачами проекта. Тело запроса:

```
{
  "predecessor_id": 7,
  "successor_id": 8,
  "type": "FS"
}
```

Ответ 201 Created содержит созданную зависимость. Сервер проверяет принадлежность обеих задач проекту, отсутствие самозависимости, дубли связи, цикла в графе и нарушения дат. Получение отдельной зависимости выполняется через GET `/api/projects/{project_id}/dependencies/{id}`, удаление – через DELETE по тому же маршруту без тела с ответом 204 No Content.

### **Участники проекта.**

GET `/api/projects/{project_id}/members`

Запрос не содержит тела. Ответ 200 OK – массив участников:

```
[
  {
    "id": 11,
    "user_id": 3,
    "username": "ivanov.i",
    "full_name": "Иванов Иван Иванович",
    "role": "owner"
  }
]
```

POST /api/projects/{project\_id}/members

Добавляет пользователя в состав участников проекта. Тело запроса:

```
{
  "user_id": 4,
  "role": "editor"
}
```

Ответ 201 Created содержит созданную запись участника. Через API нельзя назначить роль owner.

PATCH /api/projects/{project\_id}/members/{id}

Изменяет роль участника проекта. Тело запроса:

```
{
  "role": "viewer"
}
```

Ответ 200 OK содержит обновленную запись участника. Удаление участника выполняется через DELETE по тому же маршруту без тела и возвращает 204 No Content; владельца удалить или изменить таким способом нельзя.

DELETE /api/projects/{project\_id}/members/leave

Маршрут позволяет текущему пользователю выйти из проекта. Тело запроса не передается. Ответ – 204 No Content. Владелец проекта выйти не может, вместо этого он должен удалить проект.

### 3.4 Проект базы данных

#### 3.4.1 ER-модель данных

Концептуальная модель содержит шесть сущностей [17, 31].

Центральная сущность – проект. Из атрибутов важны название, описание, даты начала и завершения, плюс ссылка на создателя. Внутри проекта находятся задачи. У каждой свое название, описание, даты и порядковый номер для отображения на диаграмме. Между задачами могут быть зависимости одного из четырех типов: финиш-старт, старт-старт, финиш-финиш,

старт-финиш. У зависимости два внешних ключа на задачи – на предшественника и на преемника – и поле типа связи.

Пользователь системы описывается логином, адресом электронной почты, хешем пароля и ФИО. В одном проекте состоит несколько пользователей в разных ролях – владелец, редактор, наблюдатель. Эта связь представлена отдельной сущностью «Участие в проекте» с двумя внешними ключами (на пользователя и на проект) и полем роли.

Отдельно от учетных данных у пользователя есть цветовая палитра – набор цветов и выбранный режим темы. Связь с пользователем один к одному: у каждого аккаунта своя палитра, одна палитра принадлежит одному пользователю.

ER-диаграмма приведена на рисунке 3.4.

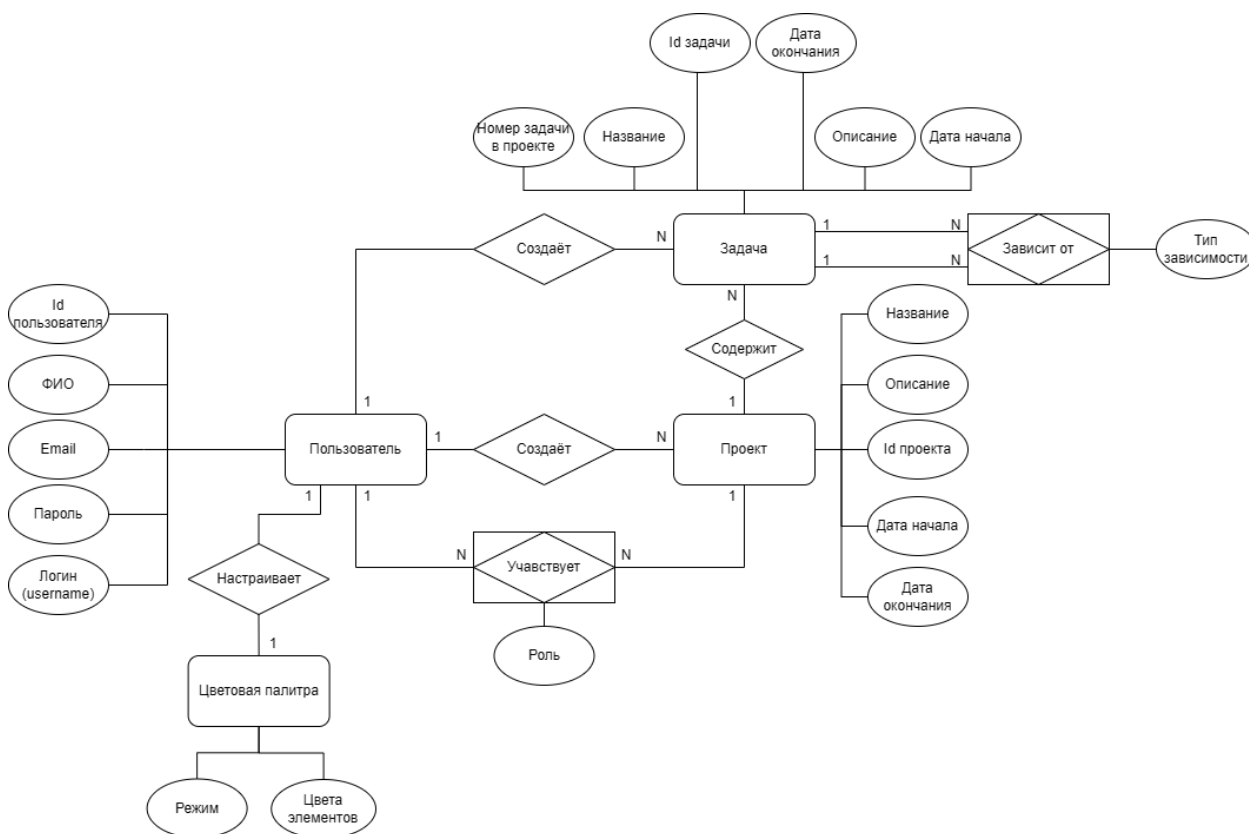


Рисунок 3.4 – ER-модель данных

### 3.4.2 Нормализация данных

Схема приведена к третьей нормальной форме [20].

Первые две формы выполняются автоматически. Атрибутов-списков и повторяющихся групп в схеме нет – цвета палитры разложены по отдельным полям, а не упакованы в строку или массив. Все таблицы построены на суррогатном первичном ключе, автоинкрементном поле `id`, поэтому частичной зависимости от части составного ключа здесь в принципе быть не может. Случаи, где напрашивался составной ключ (пара участник–проект, пара предшественник–преемник), реализованы как ограничения уникальности.

Третья нормальная форма потребовала отдельной проверки. В таблице задач хранятся идентификаторы проекта и автора, а не сами эти объекты. В таблице участников нет ни имени пользователя, ни названия проекта – только ссылки. Единственная особенность – производное поле `full_name` у пользователя. Оно сделано генерируемым столбцом, который база вычисляет сама по фамилии, имени и отчеству. На третью нормальную форму это не влияет: значение не задается отдельно от других полей строки, оно выводится из них полностью.

### 3.4.3 Описание структуры базы данных

Физическая схема базы состоит из прикладных таблиц системы и служебных таблиц Django. К прикладным относятся `users`, `projects`, `tasks`, `project_members`, `task_dependencies` и `users_color_palette`. В них хранятся пользователи, проекты, задачи, участие пользователей в проектах, зависимости между задачами и настройки цветовой темы. Остальные таблицы создают встроенные приложения Django: `auth_group`, `auth_permission`, `auth_group_permissions`, `users_groups`, `users_user_permissions`, `django_content_type`, `django_admin_log`, `django_migrations` и `django_session`.

Прикладные модели используют тип `BigAutoField`, поэтому первичные ключи `id` и внешние ключи на них в MySQL имеют тип `BIGINT`. Текстовые описания проектов и задач хранятся как `LONGTEXT`, даты – как `DATETIME(6)`, булевы признаки пользователя – как `TINYINT(1)`. Связи между прикладными таблицами реализованы внешними ключами. От-

ношение многие ко многим между пользователями и проектами развернуто в таблицу `project_members` с полем роли. Зависимости задач вынесены в `task_dependencies` с двумя внешними ключами на `tasks` – на предшественника и на преемника. Цветовая палитра связана с пользователем отношением один к одному через поле `user_id`.

Схема таблиц с типами полей, первичными и внешними ключами показана на рисунке 3.5.

Рисунок 3.5 – Схема базы данных



Внешние ключи прикладной части настроены либо на каскадное удаление, либо на запрет удаления. При удалении проекта вместе с ним удаляются его задачи, зависимости и записи об участии. При удалении пользователя удаляются его палитра, записи об участии в проектах, связи с группами и индивидуальные разрешения. Пользователя, который числится автором хотя бы одной задачи или создателем хотя бы одного проекта, база удалить не позволит – у полей `creator_id` в таблицах `projects` и `tasks` указано `on_delete=RESTRICT`.

Структура таблицы `users` приведена в таблице 3.1.

Таблица 3.1 – Структура таблицы `users`

| Поле                      | Тип                       | Обязательность | Назначение                                |
|---------------------------|---------------------------|----------------|-------------------------------------------|
| <code>id</code>           | <code>BIGINT</code>       | Да             | Первичный ключ                            |
| <code>password</code>     | <code>VARCHAR(128)</code> | Да             | Хеш пароля                                |
| <code>last_login</code>   | <code>DATETIME(6)</code>  | Нет            | Дата последнего входа                     |
| <code>is_superuser</code> | <code>TINYINT(1)</code>   | Да             | Признак суперпользователя                 |
| <code>username</code>     | <code>VARCHAR(150)</code> | Да             | Имя пользователя, уникально               |
| <code>first_name</code>   | <code>VARCHAR(150)</code> | Нет            | Имя                                       |
| <code>last_name</code>    | <code>VARCHAR(150)</code> | Нет            | Фамилия                                   |
| <code>email</code>        | <code>VARCHAR(254)</code> | Да             | Адрес электронной почты, уникален         |
| <code>is_staff</code>     | <code>TINYINT(1)</code>   | Да             | Признак доступа к административной панели |
| <code>is_active</code>    | <code>TINYINT(1)</code>   | Да             | Признак активного пользователя            |
| <code>date_joined</code>  | <code>DATETIME(6)</code>  | Да             | Дата регистрации                          |

Продолжение таблицы 3.1

| Поле        | Тип          | Обязательность | Назначение                    |
|-------------|--------------|----------------|-------------------------------|
| middle_name | VARCHAR(150) | Нет            | Отчество                      |
| full_name   | VARCHAR(452) | Да             | Полное имя, вычисляется базой |

Структура таблицы projects приведена в таблице 3.2.

Таблица 3.2 – Структура таблицы projects

| Поле        | Тип          | Обязательность | Назначение              |
|-------------|--------------|----------------|-------------------------|
| id          | BIGINT       | Да             | Первичный ключ          |
| title       | VARCHAR(150) | Да             | Название проекта        |
| description | LONGTEXT     | Нет            | Описание                |
| start_date  | DATETIME(6)  | Нет            | Дата начала проекта     |
| end_date    | DATETIME(6)  | Нет            | Дата завершения проекта |
| creator_id  | BIGINT       | Да             | Внешний ключ на users   |

Структура таблицы tasks приведена в таблице 3.3.

Таблица 3.3 – Структура таблицы tasks

| Поле        | Тип          | Обязательность | Назначение      |
|-------------|--------------|----------------|-----------------|
| id          | BIGINT       | Да             | Первичный ключ  |
| title       | VARCHAR(150) | Да             | Название задачи |
| description | LONGTEXT     | Нет            | Описание        |
| start_date  | DATETIME(6)  | Нет            | Дата начала     |
| end_date    | DATETIME(6)  | Нет            | Дата завершения |

Продолжение таблицы 3.3

| Поле       | Тип    | Обязательность | Назначение                 |
|------------|--------|----------------|----------------------------|
| number     | INT    | Да             | Порядковый номер в проекте |
| creator_id | BIGINT | Да             | Внешний ключ на users      |
| project_id | BIGINT | Да             | Внешний ключ на projects   |

Пара (project\_id, number) в таблице tasks ограничена уникальностью – две задачи в одном проекте не могут иметь одинаковый порядковый номер.

Структура таблицы project\_members приведена в таблице 3.4.

Таблица 3.4 – Структура таблицы project\_members

| Поле       | Тип         | Обязательность | Назначение                  |
|------------|-------------|----------------|-----------------------------|
| id         | BIGINT      | Да             | Первичный ключ              |
| role       | VARCHAR(10) | Да             | Роль: owner, editor, viewer |
| project_id | BIGINT      | Да             | Внешний ключ на projects    |
| user_id    | BIGINT      | Да             | Внешний ключ на users       |

Пара (project\_id, user\_id) ограничена уникальностью – один пользователь не может состоять в одном проекте дважды.

Структура таблицы task\_dependencies приведена в таблице 3.5.

Таблица 3.5 – Структура таблицы task\_dependencies

| Поле           | Тип        | Обязательность | Назначение                             |
|----------------|------------|----------------|----------------------------------------|
| id             | BIGINT     | Да             | Первичный ключ                         |
| type           | VARCHAR(2) | Да             | Тип связи: FS, SS, FF, SF              |
| predecessor_id | BIGINT     | Да             | Внешний ключ на tasks (предшественник) |
| successor_id   | BIGINT     | Да             | Внешний ключ на tasks (преемник)       |

Пара (predecessor\_id, successor\_id) ограничена уникальностью. На уровне таблицы запрещено равенство predecessor\_id и successor\_id – задача не может зависеть от самой себя. Дополнительная проверка на отсутствие циклов выполняется на уровне приложения при создании зависимости.

Структура таблицы users\_color\_palette приведена в таблице 3.6.

Таблица 3.6 – Структура таблицы users\_color\_palette

| Поле            | Тип         | Обязательность | Назначение                              |
|-----------------|-------------|----------------|-----------------------------------------|
| id              | BIGINT      | Да             | Первичный ключ                          |
| theme_mode      | VARCHAR(16) | Да             | Режим темы: system, light, dark, custom |
| primary_color   | VARCHAR(7)  | Да             | Базовый цвет палитры                    |
| secondary_color | VARCHAR(7)  | Да             | Дополнительный цвет палитры             |

Продолжение таблицы 3.6

| Поле                            | Тип        | Обязательность | Назначение                                     |
|---------------------------------|------------|----------------|------------------------------------------------|
| darker_<br>secondary_<br>color  | VARCHAR(7) | Да             | Темный оттенок до-<br>полнительного цве-<br>та |
| accent_color                    | VARCHAR(7) | Да             | Акцентный цвет<br>кнопок и ссылок              |
| task_color                      | VARCHAR(7) | Да             | Цвет полос задач на<br>диаграмме Ганта         |
| gantt_grid_<br>line_color       | VARCHAR(7) | Да             | Цвет линий сетки<br>диаграммы                  |
| gantt_current_<br>time_color    | VARCHAR(7) | Да             | Цвет линии текуще-<br>го времени               |
| error_color                     | VARCHAR(7) | Да             | Цвет сообщений об<br>ошибках                   |
| danger_color                    | VARCHAR(7) | Да             | Цвет опасных дей-<br>ствий                     |
| success_color                   | VARCHAR(7) | Да             | Цвет сообщений об<br>успехе                    |
| user_id                         | BIGINT     | Да             | Внешний ключ на<br>users, уникален             |
| app_<br>background_<br>color    | VARCHAR(7) | Да             | Цвет фона приложе-<br>ния                      |
| surface_color                   | VARCHAR(7) | Да             | Цвет поверхностей<br>карточек и панелей        |
| header_<br>background_<br>color | VARCHAR(7) | Да             | Цвет фона шапки                                |

Продолжение таблицы 3.6

| Поле                         | Тип        | Обязательность | Назначение                            |
|------------------------------|------------|----------------|---------------------------------------|
| border_color                 | VARCHAR(7) | Да             | Цвет границ                           |
| primary_text_color           | VARCHAR(7) | Да             | Основной цвет текста                  |
| secondary_text_color         | VARCHAR(7) | Да             | Дополнительный цвет текста            |
| inverse_text_color           | VARCHAR(7) | Да             | Цвет текста на акцентных поверхностях |
| active_menu_background_color | VARCHAR(7) | Да             | Цвет фона активного пункта меню       |
| active_menu_text_color       | VARCHAR(7) | Да             | Цвет текста активного пункта меню     |
| gantt_arrow_color            | VARCHAR(7) | Да             | Цвет стрелок зависимостей             |

Назначение служебных таблиц Django приведено в таблице 3.7.

Таблица 3.7 – Назначение служебных таблиц Django

| Таблица                | Назначение                                                            |
|------------------------|-----------------------------------------------------------------------|
| auth_group             | Группы пользователей встроенной системы авторизации Django.           |
| auth_permission        | Разрешения доступа, связанные с типами моделей через content_type_id. |
| auth_group_permissions | Связка многие ко многим между группами и разрешениями.                |

Продолжение таблицы 3.7

| Таблица                | Назначение                                                                                 |
|------------------------|--------------------------------------------------------------------------------------------|
| users_groups           | Связка многие ко многим между пользователями и группами.                                   |
| users_user_permissions | Связка многие ко многим между пользователями и индивидуальными разрешениями.               |
| django_content_type    | Реестр моделей Django, который используется системой разрешений и журналом администратора. |
| django_admin_log       | Журнал действий, выполненных через административную панель.                                |
| django_migrations      | Список примененных миграций базы данных.                                                   |
| django_session         | Данные пользовательских сеансов; ключ сеанса хранится в cookie браузера.                   |

Для ускорения поиска по часто используемым полям заданы индексы: по title в таблицах проектов и задач, по username и full\_name в таблице пользователей. Первичные ключи и внешние ключи индексируются автоматически.

## 4 Рабочий проект

### 4.1 Спецификация классов и компонентов программной системы

В данном разделе приведено описание основных классов и модулей, реализованных в ходе разработки серверной и клиентской частей программной системы. Для каждого класса указаны назначение, ключевые атрибуты или поля, а также перечень методов с модификаторами доступа и кратким описанием. Для модулей, состоящих из набора функций, приводится описание каждой функции.

Серверный код разделен на пакеты моделей, сериализаторов, представлений, фильтров, валидаторов, общих фрагментов поведения, а также модули проверки прав доступа и маршрутизации. Клиентский код разделен на модули обмена с сервером, многоразовые элементы интерфейса, страницы маршрутов, контексты глобального состояния и пользовательские хуки.

#### 4.1.1 Класс User

Класс User описывает модель учетной записи пользователя системы. Он наследуется от стандартного класса AbstractUser платформы Django и расширяет его полем отчества и вычисляемым полем полного имени, объединяющим фамилию, имя и отчество. Класс используется при аутентификации, разграничении прав доступа и поиске пользователей по полному имени или логину. От базового класса наследуются методы хеширования и проверки пароля.

Описание полей класса User приведено в таблице 4.1.

Таблица 4.1 – Дополнительные поля класса User

| Поле        | Назначение                                                        |
|-------------|-------------------------------------------------------------------|
| middle_name | Отчество пользователя                                             |
| full_name   | Полное имя, вычисляемое базой данных по фамилии, имени и отчеству |



### 4.1.2 Класс Project

Класс Project описывает проект как центральную сущность системы. Хранит название, описание, даты начала и завершения, ссылку на создателя. На уровне базы данных задан индекс по названию проекта для ускорения поиска. Через обратное отношение members класс предоставляет доступ к составу участников проекта.

### 4.1.3 Класс Task

Класс Task описывает задачу проекта. Содержит название, описание, даты начала и завершения, порядковый номер задачи в проекте, ссылки на создателя и на проект. Уникальность пары (project, number) запрещает существование двух задач с одинаковым порядковым номером в одном проекте. На поле названия задачи задан индекс. Через обратные отношения outgoing\_dependencies и incoming\_dependencies класс возвращает связи, в которых задача выступает соответственно предшественником и преемником.

### 4.1.4 Класс TaskDependency

Класс TaskDependency хранит зависимость между двумя задачами. Содержит ссылки на задачу-предшественника и задачу-преемника, а также тип связи. Перечень типов связей вынесен в константы класса.

Константы типов связей приведены в таблице 4.2.

Таблица 4.2 – Константы типов связей класса TaskDependency

| Константа        | Значение | Описание                                                       |
|------------------|----------|----------------------------------------------------------------|
| FINISH_TO_START  | FS       | Преемник начинается после завершения предшественника           |
| START_TO_START   | SS       | Преемник начинается одновременно с предшественником или позже  |
| FINISH_TO_FINISH | FF       | Преемник завершается одновременно с предшественником или позже |

#### Продолжение таблицы 4.2

| Константа       | Значение | Описание                                              |
|-----------------|----------|-------------------------------------------------------|
| START_TO_FINISH | SF       | Преемник завершается не раньше начала предшественника |

На уровне таблицы базы данных заданы ограничение уникальности пары (predecessor, successor) и проверочное ограничение, запрещающее равенство этих полей.

#### 4.1.5 Класс ProjectMember

Класс ProjectMember реализует отношение многие ко многим между пользователями и проектами с указанием роли участника. Содержит ссылки на проект и пользователя, а также поле роли. Перечень ролей вынесен в константы класса.

Константы ролей приведены в таблице 4.3.

Таблица 4.3 – Константы ролей класса ProjectMember

| Константа | Значение | Описание                                     |
|-----------|----------|----------------------------------------------|
| OWNER     | owner    | Владелец проекта – полный доступ             |
| EDITOR    | editor   | Редактор – работа с задачами и зависимостями |
| VIEWER    | viewer   | Наблюдатель – доступ только на чтение        |

Пара (project, user) ограничена уникальностью – один пользователь не может состоять в одном проекте дважды.

#### 4.1.6 Класс UserColorPalette

Класс UserColorPalette хранит индивидуальные настройки оформления интерфейса пользователя. Связан с пользователем отношением один к одному. Содержит поле режима темы theme\_mode (system, light, dark, custom) и на-

бор полей с цветами отдельных элементов: фона приложения, поверхности, шапки, активного пункта меню, акцента, полос задач на диаграмме Ганта, стрелок зависимостей, линий сетки, линии текущего времени, сообщений об ошибках, опасных действий и успешных операций.

#### 4.1.7 Класс AuthViewSet

Класс AuthViewSet наследуется от ViewSet библиотеки DRF и предоставляет действия аутентификации пользователя. Описание методов класса приведено в таблице 4.4.

Таблица 4.4 – Методы класса AuthViewSet

| Метод  | Доступ | Описание                                                                                        |
|--------|--------|-------------------------------------------------------------------------------------------------|
| login  | public | Проверяет логин и пароль, открывает сеанс работы пользователя, возвращает данные учетной записи |
| logout | public | Завершает сеанс работы пользователя, возвращает код 204 без тела ответа                         |
| csrf   | public | Устанавливает cookie csrftoken для последующих изменяющих запросов                              |

#### 4.1.8 Класс UserViewSet

Класс UserViewSet наследуется от ModelViewSet библиотеки DRF и реализует операции над учетными записями пользователей. Метод get\_queryset ограничивает выборку текущим пользователем (суперпользователь видит всех). Метод get\_serializer\_class переключает класс сериализатора в зависимости от действия. Метод get\_permissions разрешает создание учетной записи без аутентификации.

Описание методов класса UserViewSet приведено в таблице 4.5.

Таблица 4.5 – Методы класса UserViewSet

| Метод                | Доступ | Описание                                                                        |
|----------------------|--------|---------------------------------------------------------------------------------|
| get_queryset         | public | Возвращает множество доступных пользователю записей                             |
| get_serializer_class | public | Выбирает класс сериализатора по действию                                        |
| get_permissions      | public | Определяет требования к авторизации по действию                                 |
| create               | public | Создает учетную запись пользователя                                             |
| me                   | public | Возвращает данные текущего пользователя                                         |
| search               | public | Поиск пользователей по логину и полному имени для добавления участников проекта |
| color_palette        | public | Чтение и частичное обновление цветовой палитры текущего пользователя            |

#### 4.1.9 Класс ProjectViewSet

Класс ProjectViewSet реализует операции над проектами. Метод get\_queryset возвращает только те проекты, в которых текущий пользователь состоит участником. При создании проекта автор автоматически добавляется в участники с ролью владельца в рамках одной транзакции. Изменение и удаление проекта доступны только владельцу.

Описание методов класса ProjectViewSet приведено в таблице 4.6.

Таблица 4.6 – Методы класса ProjectViewSet

| Метод        | Доступ | Описание                                                      |
|--------------|--------|---------------------------------------------------------------|
| get_queryset | public | Возвращает проекты, в которых пользователь состоит участником |

Продолжение таблицы 4.6

| Метод           | Доступ | Описание                                                                           |
|-----------------|--------|------------------------------------------------------------------------------------|
| perform_create  | public | Создает проект и добавляет автора в участники с ролью владельца в одной транзакции |
| perform_update  | public | Изменяет проект, требуется роль владельца                                          |
| perform_destroy | public | Удаляет проект, требуется роль владельца                                           |

#### 4.1.10 Класс ProjectTaskViewSet

Класс ProjectTaskViewSet реализует операции над задачами в рамках проекта. Метод initial выполняется до основного действия и проверяет принадлежность пользователя к проекту и его роль. При создании задачи метод perform\_create вычисляет следующий порядковый номер задачи в проекте и записывает его в одной транзакции с самой задачей. Дополнительное действие move изменяет порядок задач в проекте.

Описание методов класса ProjectTaskViewSet приведено в таблице 4.7.

Таблица 4.7 – Методы класса ProjectTaskViewSet

| Метод          | Доступ | Описание                                                                          |
|----------------|--------|-----------------------------------------------------------------------------------|
| get_queryset   | public | Возвращает задачи проекта, упорядоченные по порядковому номеру                    |
| initial        | public | Проверяет принадлежность пользователя к проекту и его роль до выполнения действия |
| perform_create | public | Создает задачу, присваивает порядковый номер в проекте                            |
| move           | public | Изменяет порядок задач в проекте, перенумеровывает все задачи в транзакции        |

#### 4.1.11 Класс TaskViewSet

Класс TaskViewSet реализует чтение, изменение и удаление задач по глобальному идентификатору. Запросы метода GET без параметра проекта возвращают задачи, созданные текущим пользователем. Запросы PATCH, PUT и DELETE требуют прав редактора или владельца проекта, которому принадлежит задача.

#### 4.1.12 Класс ProjectDependencyViewSet

Класс ProjectDependencyViewSet реализует операции над зависимостями между задачами в рамках проекта. Поддерживаются методы GET, POST и DELETE. Метод initial проверяет роль пользователя: для чтения требуется любое участие в проекте, для изменения – роль редактора или владельца. При создании зависимости в контексте сериализатора передается объект проекта, что используется валидатором для проверки принадлежности обеих задач этому проекту.

#### 4.1.13 Класс ProjectMemberViewSet

Класс ProjectMemberViewSet реализует операции над составом участников проекта. Доступ к операциям над участниками имеет только владелец проекта. Изменение и удаление записи об участнике с ролью владельца запрещено. Дополнительное действие leave позволяет участнику самостоятельно покинуть проект.

Описание методов класса ProjectMemberViewSet приведено в таблице 4.8.

Таблица 4.8 – Методы класса ProjectMemberViewSet

| Метод   | Доступ | Описание                                                            |
|---------|--------|---------------------------------------------------------------------|
| initial | public | Проверяет аутентификацию и роль пользователя до выполнения действия |

Продолжение таблицы 4.8

| Метод           | Доступ | Описание                                                              |
|-----------------|--------|-----------------------------------------------------------------------|
| get_queryset    | public | Возвращает состав участников проекта, упорядоченный по идентификатору |
| create          | public | Добавляет нового участника в проект с ролью редактора или наблюдателя |
| perform_update  | public | Изменяет роль участника, запрещает изменение роли владельца           |
| perform_destroy | public | Удаляет участника, запрещает удаление владельца                       |
| leave           | public | Позволяет текущему пользователю выйти из проекта                      |

#### 4.1.14 Класс PasswordStrengthValidator

Класс PasswordStrengthValidator реализует проверку сложности пароля по требованиям технического задания. Подключается через настройку AUTH\_PASSWORD\_VALIDATORS Django.

Описание методов класса приведено в таблице 4.9.

Таблица 4.9 – Методы класса PasswordStrengthValidator

| Метод         | Доступ | Описание                                                                                                                         |
|---------------|--------|----------------------------------------------------------------------------------------------------------------------------------|
| validate      | public | Проверяет, что пароль состоит из допустимых символов и содержит строчную и заглавную латинские буквы, цифру и специальный символ |
| get_help_text | public | Возвращает описание требований к паролю для пользовательского интерфейса                                                         |

#### 4.1.15 Модуль проверки зависимостей задач

Модуль проверки зависимостей задач содержит функции, которые применяются сериализаторами при создании и изменении задач и зависимостей. Функции модуля приведены в таблице 4.10.

Таблица 4.10 – Функции модуля проверки зависимостей задач

| Функция                               | Доступ | Описание                                                                                    |
|---------------------------------------|--------|---------------------------------------------------------------------------------------------|
| is_pair_satisfied                     | public | Проверяет, удовлетворяют ли даты пары задач выбранному типу связи                           |
| check_dependency_dates                | public | Поднимает исключение валидации, если даты задач не соответствуют типу связи                 |
| would_create_cycle                    | public | Поиском в ширину проверяет, образует ли новая зависимость цикл в графе зависимостей проекта |
| check_task_dates_against_dependencies | public | Проверяет, что новые даты задачи не нарушают входящих и исходящих зависимостей              |

#### 4.1.16 Модуль проверки прав доступа

Модуль проверки прав доступа реализует функции определения роли пользователя в проекте. Используется классами-представлениями для разграничения доступа. Функции модуля приведены в таблице 4.11.



Таблица 4.11 – Функции модуля проверки прав доступа

| Функция       | Доступ | Описание                                                                                  |
|---------------|--------|-------------------------------------------------------------------------------------------|
| get_user_role | public | Возвращает роль пользователя в проекте или None, если пользователь не является участником |
| is_member     | public | Истинно, если пользователь состоит в проекте                                              |
| is_writer     | public | Истинно, если пользователь имеет роль владельца или редактора                             |
| is_owner      | public | Истинно, если пользователь имеет роль владельца                                           |

#### 4.1.17 Компонент GanttChart

Компонент GanttChart реализует основное представление календарного плана проекта – диаграмму Ганта. Отвечает за построение временной шкалы в пределах дат проекта, отрисовку полос задач, стрелок зависимостей, линии текущего времени и штриховки истекшей части полос. Обрабатывает перетаскивание полос задач, перетаскивание левого и правого краев полосы, перетаскивание строки за ручку для изменения порядка задач, режим добавления связей с точками соединения на торцах полос, масштабирование диаграммы в диапазоне 50–400 % колесом мыши с удержанием Ctrl, кнопками панели инструментов и жестом сведения пальцев. Допустимый диапазон перемещения каждой полосы ограничивается датами проекта и установленными зависимостями.

#### 4.1.18 Компонент TaskCard

Компонент TaskCard отображает детальную карточку задачи. Содержит редактируемые поля названия, описания, дат начала и завершения, а также список зависимостей задачи с возможностью добавления и удаления свя-

зей. Поля дат ограничены датами проекта и ограничениями из установленных зависимостей.

#### 4.1.19 Компонент ProjectMembers

Компонент ProjectMembers реализует управление составом участников проекта. Содержит список текущих участников с их ролями, форму поиска пользователей по логину или ФИО с выводом не более десяти совпадений, элементы изменения роли участника и кнопки удаления. Для текущего пользователя с ролью редактора или наблюдателя предусмотрена кнопка «Покинуть проект».

#### 4.1.20 Компонент SettingsPanel

Компонент SettingsPanel реализует страницу настройки цветовой темы. Содержит переключатель режима темы (системный, светлый, темный, пользовательский) и элементы выбора цвета для отдельных компонентов интерфейса в пользовательском режиме. Применение темы выполняется немедленно через изменение CSS-переменных корневого элемента документа без перезагрузки страницы.

#### 4.1.21 Контексты клиентского приложения

Глобальное состояние клиентского приложения вынесено в три контекста React. Описание контекстов приведено в таблице 4.12.

Таблица 4.12 – Контексты клиентского приложения

| Контекст    | Назначение                                                                                                                 |
|-------------|----------------------------------------------------------------------------------------------------------------------------|
| AuthContext | Хранит данные текущего пользователя, реализует операции входа, выхода и регистрации, отслеживает состояние загрузки сеанса |

Продолжение таблицы 4.12

| Контекст        | Назначение                                                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| ProjectsContext | Хранит список проектов пользователя и идентификатор последнего открытого проекта, реализует операции создания, обновления и удаления проектов |
| ThemeContext    | Хранит режим темы и набор цветов палитры, применяет цвета к корневому элементу документа через CSS-переменные                                 |

#### 4.1.22 Модули обмена с сервером

Модули обмена с сервером инкапсулируют обращения клиентской части к серверу. Базовый модуль формирует HTTP-запросы с правильными заголовками и cookie, остальные модули реализуют операции над сущностями системы. Описание модулей приведено в таблице 4.13.

Таблица 4.13 – Модули обмена с сервером

| Модуль          | Назначение                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------|
| client.js       | Базовая функция HTTP-запроса с подстановкой cookie сеанса и заголовка X-CSRFToken                                          |
| users.js        | Операции над учетными записями: регистрация, чтение, обновление, поиск пользователей, чтение и обновление цветовой палитры |
| projects.js     | Операции над проектами: чтение списка с фильтрацией, создание, обновление, удаление                                        |
| tasks.js        | Операции над задачами проекта: чтение списка, создание, обновление, удаление, изменение порядка                            |
| dependencies.js | Операции над зависимостями задач: чтение, создание, удаление                                                               |

Продолжение таблицы 4.13

| Модуль               | Назначение                                                                                               |
|----------------------|----------------------------------------------------------------------------------------------------------|
| members.js           | Операции над составом участников проекта: чтение, добавление, изменение роли, удаление, выход из проекта |
| dates.js             | Вспомогательные функции преобразования дат между ISO-строками и календарным представлением               |
| dependencyHelpers.js | Вспомогательные функции определения типа зависимости по выбранным торцам полос задач на диаграмме        |
| roles.js             | Константы ролей участников проекта и функции проверки прав на стороне клиента                            |

## 4.2 Системное тестирование

Системное тестирование программной системы проводилось по сценариям, сформированным на основе прецедентов использования из технического задания. Проверялись основные функции системы: регистрация и аутентификация пользователя, управление проектами и составом участников, работа с задачами и зависимостями, интерактивное редактирование на диаграмме Ганта, настройка цветовой темы.

Тестирование выполнялось в операционной системе Windows 11 с использованием браузера Google Chrome. Серверная часть запускалась локально через uvicorn в режиме ASGI, клиентская – через сервер разработки React. Все сценарии завершены успешно.

### 4.2.1 Сводная таблица тест-кейсов

Сводная информация о выполненных тест-кейсах приведена в таблице 4.14.

Таблица 4.14 – Сводная таблица тест-кейсов

| ID    | Название тест-кейса                            | Результат |
|-------|------------------------------------------------|-----------|
| ТС-01 | Регистрация и вход пользователя в систему      | Пройден   |
| ТС-02 | Создание проекта                               | Пройден   |
| ТС-03 | Добавление задач и просмотр диаграммы Ганта    | Пройден   |
| ТС-04 | Создание зависимости между задачами            | Пройден   |
| ТС-05 | Редактирование задачи через карточку задачи    | Пройден   |
| ТС-06 | Управление составом участников проекта         | Пройден   |
| ТС-07 | Настройка цветовой темы интерфейса             | Пройден   |
| ТС-08 | Адаптивное отображение на мобильном устройстве | Пройден   |

#### 4.2.2 ТС-01. Регистрация и вход пользователя в систему

**Предусловие.** Учетная запись с указанным логином не существует, пользователь не аутентифицирован.

**Шаги:**

1. Открыть стартовую страницу системы.
2. Нажать кнопку «Зарегистрироваться».
3. Заполнить форму регистрации: логин, адрес электронной почты, пароль, подтверждение пароля.
4. Нажать кнопку «Зарегистрироваться» в модальном окне.

**Ожидаемый результат.** Система проверяет данные, создает учетную запись, открывает сеанс и перенаправляет пользователя на страницу диаграммы Ганта.

**Фактический результат.** Пройден.

Внешний вид страницы входа и регистрации представлен на рисунке 4.1.



1. Нажать кнопку создания проекта в шапке приложения.
2. Заполнить форму: название, описание, дата начала, дата завершения.
3. Подтвердить создание.

**Ожидаемый результат.** Система создает проект, назначает текущему пользователю роль владельца, переключает контекст на созданный проект и переходит к его диаграмме Ганта.

**Фактический результат.** Пройден.

Внешний вид окна создания проекта представлен на рисунке 4.3.

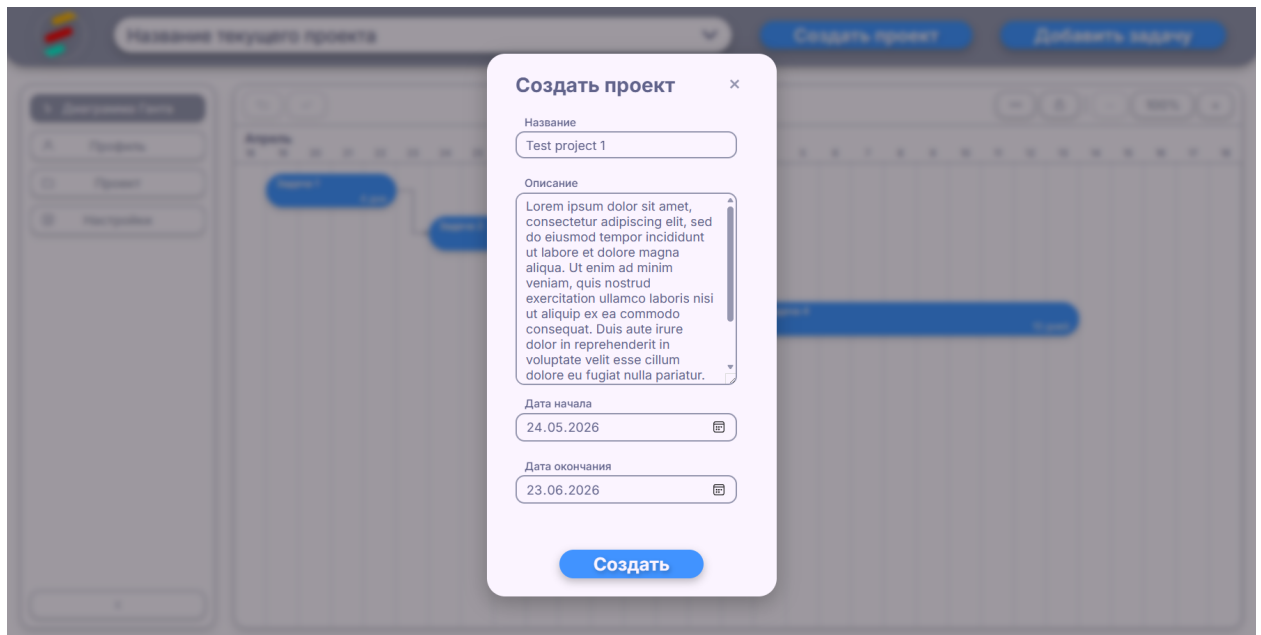


Рисунок 4.3 – Модальное окно создания проекта

#### 4.2.4 ТС-03. Добавление задач и просмотр диаграммы Ганта

**Предусловие.** Пользователь аутентифицирован, открыт проект, пользователь имеет роль владельца или редактора.

**Шаги:**

1. Нажать кнопку добавления задачи в шапке приложения.
2. Заполнить поля формы и подтвердить создание.
3. Повторить шаги 1–2 для нескольких задач.
4. Открыть страницу диаграммы Ганта.

**Ожидаемый результат.** Каждая созданная задача получает следующий порядковый номер в проекте и появляется на диаграмме Ганта в виде горизонтальной полосы с названием и длительностью.

**Фактический результат.** Пройден.

Диаграмма Ганта с задачами проекта представлена на рисунке 4.4.

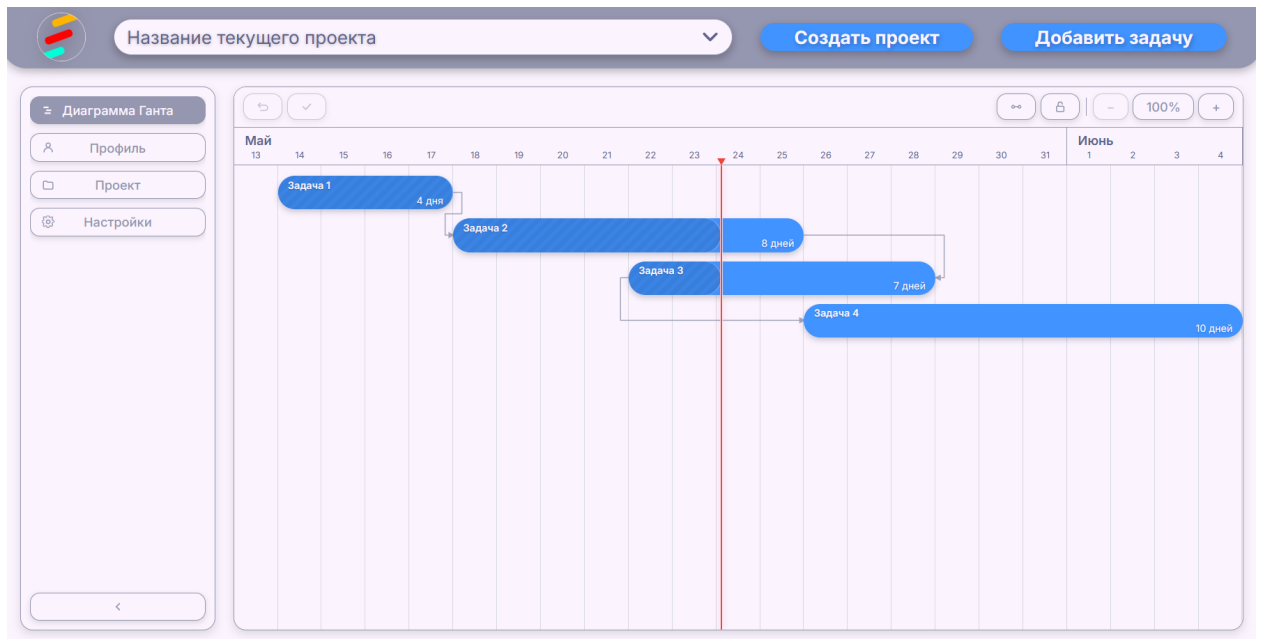


Рисунок 4.4 – Страница диаграммы Ганта проекта

#### 4.2.5 ТС-04. Создание зависимости между задачами

**Предусловие.** В проекте существует не менее двух задач, пользователь имеет роль владельца или редактора.

##### Шаги:

1. Активировать режим добавления связей кнопкой панели инструментов диаграммы.
2. Нажать на точку соединения на торце задачи-предшественника.
3. Перетащить до точки соединения на торце задачи-преемника.

**Ожидаемый результат.** Система определяет тип зависимости по выбранным торцам и сохраняет связь. На диаграмме отображается стрелка соответствующего типа.

**Фактический результат.** Пройден.



Диаграмма с активированным режимом добавления связей и созданной зависимостью представлена на рисунке 4.5.

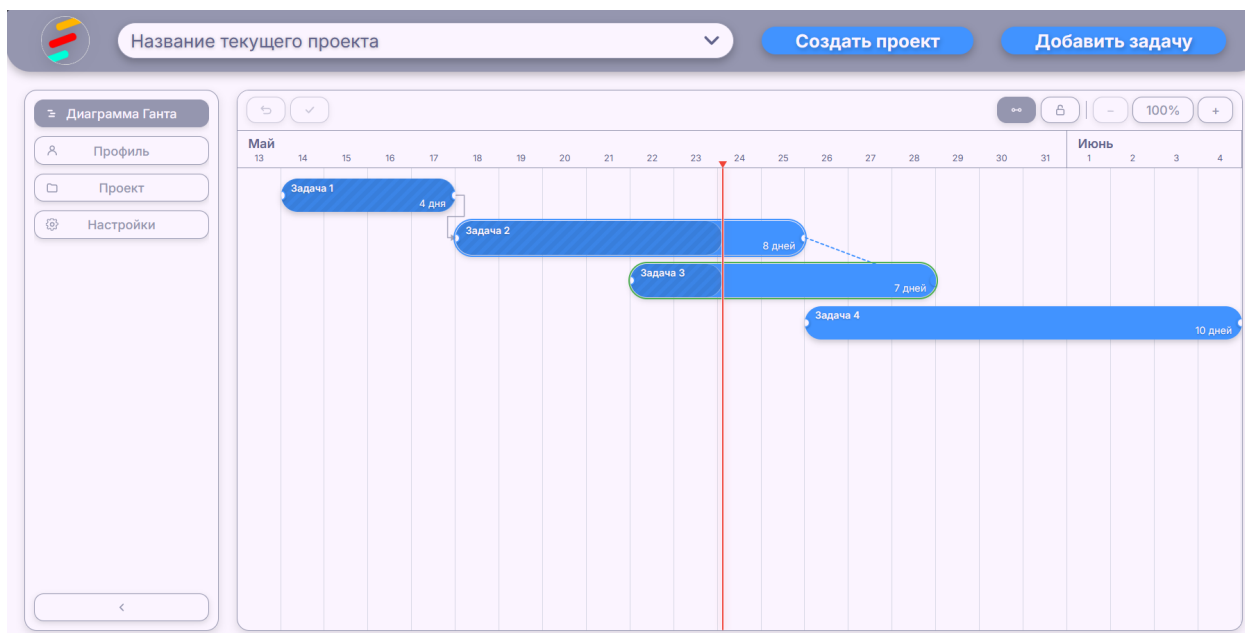


Рисунок 4.5 – Создание зависимости задач перетаскиванием на диаграмме

#### 4.2.6 ТС-05. Редактирование задачи через карточку задачи

**Предусловие.** В проекте существует задача с установленной входящей зависимостью, пользователь имеет роль владельца или редактора.

##### Шаги:

1. Нажать на полосу задачи на диаграмме Ганта.
2. Изменить название, описание и даты в открывшейся карточке.
3. Подтвердить сохранение.

**Ожидаемый результат.** Система сохраняет изменения, диаграмма Ганта обновляется. Поля дат в карточке ограничены датами проекта и ограничениями из установленных зависимостей – задать дату, нарушающую связь, нельзя.

**Фактический результат.** Пройден.

Карточка задачи представлена на рисунке 4.6.

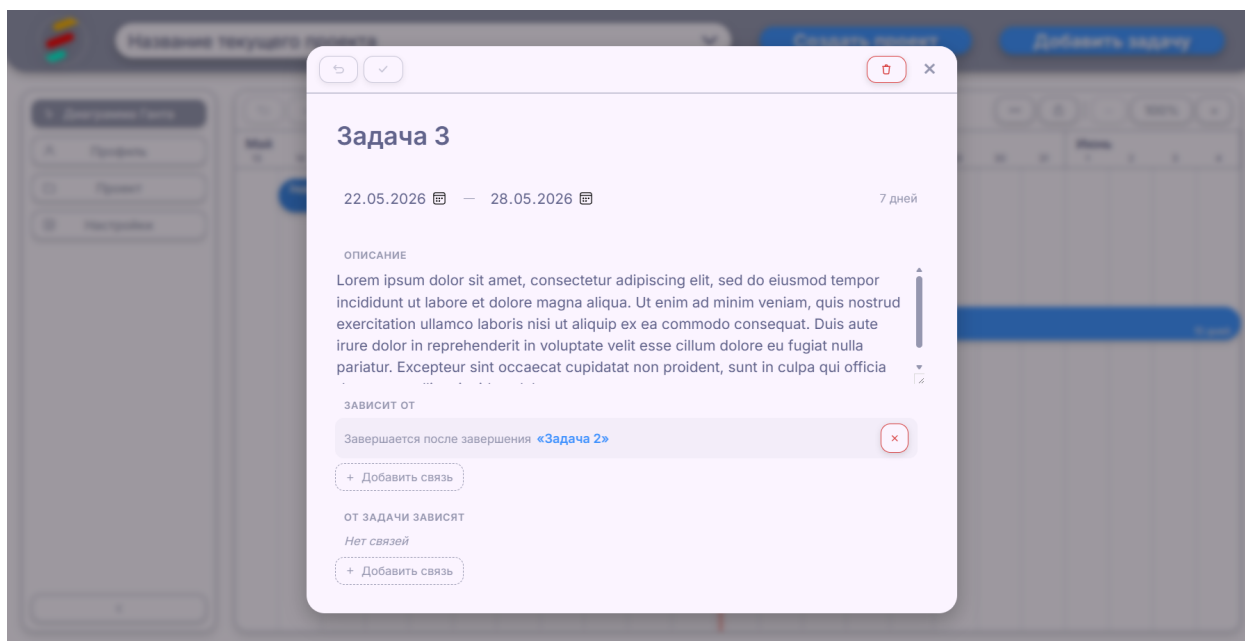


Рисунок 4.6 – Карточка задачи

#### 4.2.7 ТС-06. Управление составом участников проекта

**Предусловие.** Пользователь является владельцем проекта.

**Шаги:**

1. Перейти на страницу проекта.
2. В разделе участников ввести логин или ФИО добавляемого пользователя.
3. Выбрать пользователя из списка совпадений и назначить ему роль редактора или наблюдателя.
4. Изменить роль одного из существующих участников.

**Ожидаемый результат.** Новый участник добавлен в проект с указанной ролью, роль существующего участника обновлена.

**Фактический результат.** Пройден.

Страница проекта с разделом участников представлена на рисунке 4.7.

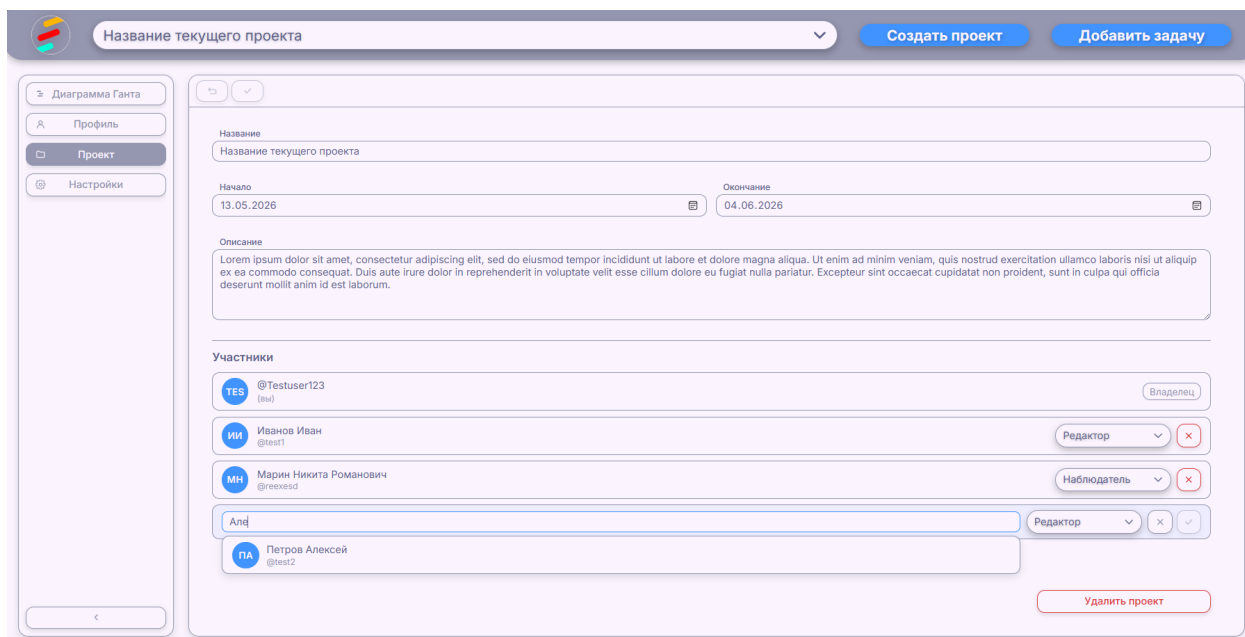


Рисунок 4.7 – Страница проекта с управлением составом участников

#### 4.2.8 ТС-07. Настройка цветовой темы интерфейса

**Предусловие.** Пользователь аутентифицирован.

**Шаги:**

1. Перейти в раздел «Настройки» через боковую панель навигации.
2. Выбрать пользовательский режим темы.
3. Изменить цвета отдельных элементов интерфейса.

**Ожидаемый результат.** Выбранные цвета применяются к интерфейсу немедленно без перезагрузки страницы и сохраняются в учетной записи пользователя.

**Фактический результат.** Пройден.

Страница настроек цветовой темы представлена на рисунке 4.8.

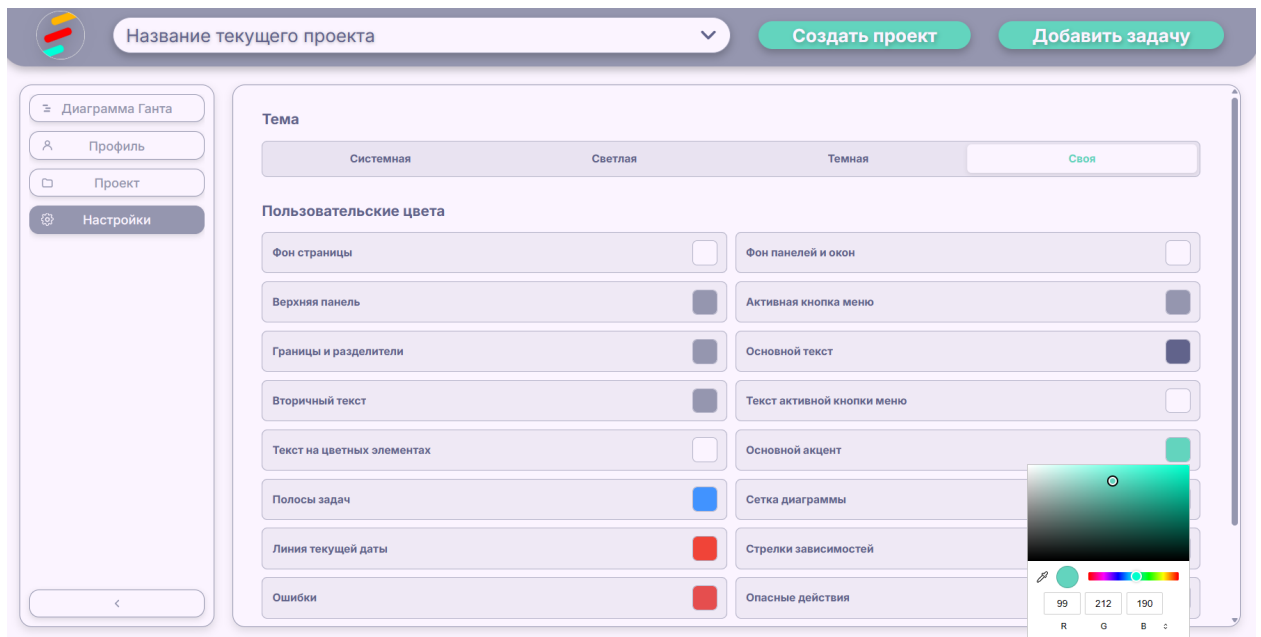


Рисунок 4.8 – Страница настроек цветовой темы

#### 4.2.9 ТС-08. Адаптивное отображение на мобильном устройстве

**Предусловие.** Пользователь аутентифицирован, ширина окна браузера соответствует размерам экрана смартфона.

##### **Шаги:**

1. Открыть страницу диаграммы Ганта на узком экране.
2. Перейти на страницу проекта и в раздел настроек.

**Ожидаемый результат.** Раскладка перестраивается под ширину экрана. Горизонтальная прокрутка вне диаграммы Ганта отсутствует, функциональность сохраняется.

**Фактический результат.** Пройден.

Вид интерфейса системы на мобильном устройстве представлен на рисунке 4.9.

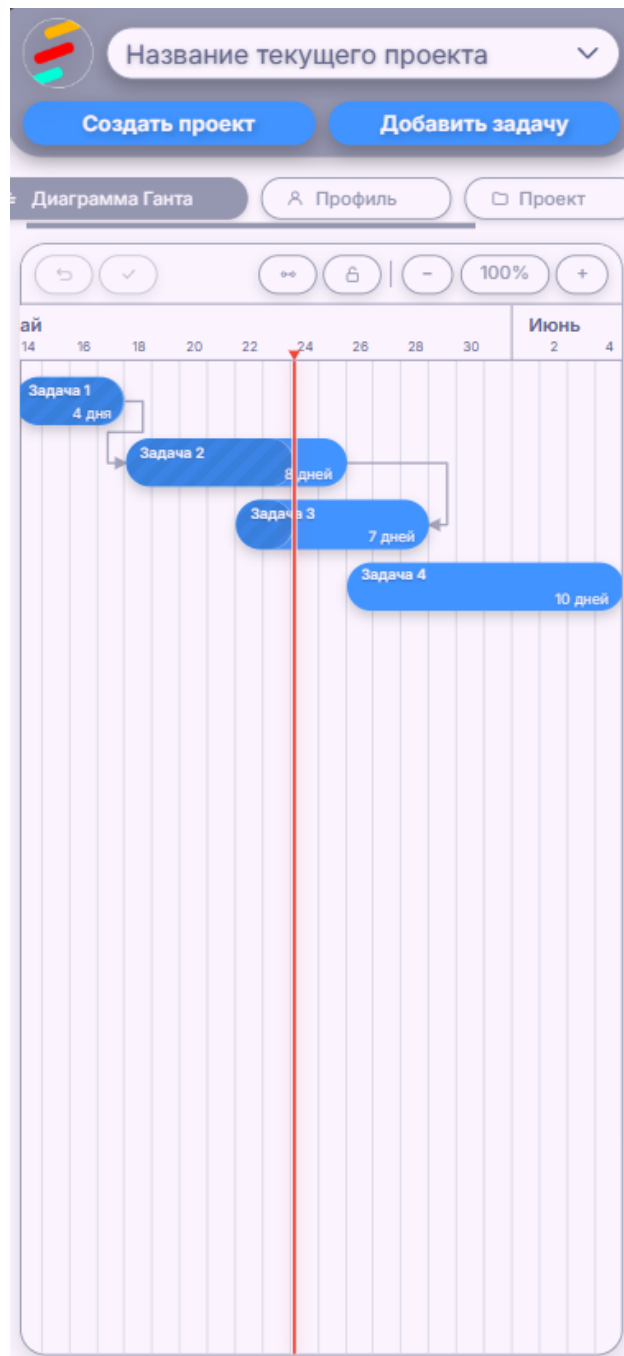


Рисунок 4.9 – Адаптивное отображение интерфейса на мобильном устройстве

## ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы разработана веб-ориентированная программно-информационная система для управления проектами. Система рассчитана на небольшие команды, ведущие несколько параллельных программных проектов, и доступна пользователям через современный браузер без установки клиентского программного обеспечения.

В ходе работы проведен анализ предметной области управления проектами и сопоставлены функциональные возможности восьми существующих ИС УП, доступных российским пользователям. Анализ показал, что ни одна из рассмотренных систем не объединяет в одной точке нативную диаграмму Ганта, поддержку четырех типов зависимостей сетевого планирования, русскоязычный интерфейс, бесплатный тариф для небольшой команды и устойчивый доступ из РФ. По результатам сравнения сформулированы требования к разрабатываемой системе.

Разработанная система реализует следующие основные возможности:

- регистрацию и аутентификацию пользователей с сеансовым механизмом доступа;
- ведение реестра проектов и декомпозицию работ на задачи;
- построение диаграммы Ганта как основного представления календарного плана;
- интерактивное изменение дат задач и их порядка непосредственно на диаграмме;
- установление зависимостей между задачами с поддержкой четырех типов сетевого планирования: финиш-старт, старт-старт, финиш-финиш, старт-финиш;
- автоматическую проверку циклов в графе зависимостей и согласованности дат с типом связи;
- разграничение прав участников проекта по трем ролям: владелец, редактор, наблюдатель;

- управление составом участников проекта с поиском пользователей по логину и полному имени;
- настройку цветовой темы интерфейса в четырех режимах: системный, светлый, темный, пользовательский;
- адаптивный интерфейс, корректно отображающийся на устройствах с различными размерами экрана.

Для реализации системы выбрана трехуровневая клиент-серверная архитектура. Серверная часть реализована на платформе Django с библиотекой Django REST Framework, клиентская – на библиотеке React. В качестве постоянного хранилища использована реляционная СУБД MySQL. Доступ к базе организован через ORM Django с параметризованными запросами, что исключает основную поверхность атак через внедрение SQL-кода. Аутентификация выполнена на стандартном механизме сеансов Django, изменяющие операции защищены CSRF-токеном. Пароли хранятся в виде хеша, открытый пароль нигде не сохраняется.

Системное тестирование подтвердило корректность работы реализованной функциональности по всем сценариям технического задания.

Основные результаты выпускной квалификационной работы:

1. Проведен анализ предметной области управления проектами и сравнительный анализ восьми существующих ИС УП, доступных российским пользователям. Обоснована потребность в собственной разработке.
2. Сформировано техническое задание на разработку, описаны прецеденты использования системы по четырем ролям пользователей.
3. Спроектирована архитектура программной системы, построена ER-модель данных, описана структура базы данных, разработан программный интерфейс между клиентской и серверной частями.
4. Реализована серверная часть программной системы на Django и Django REST Framework с использованием СУБД MySQL.
5. Реализована клиентская часть программной системы на React с интерактивной диаграммой Ганта, поддерживающей четыре типа зависимостей задач, перетаскивание полос и изменение порядка строк.

6. Реализована ролевая модель разграничения прав участников проекта и управление составом команды.

7. Реализована настройка цветовой темы интерфейса с сохранением палитры в учетной записи пользователя.

8. Проведено системное тестирование, подтвердившее корректность работы реализованной функциональности.

Все задачи, поставленные в начале выпускной квалификационной работы, успешно решены. Цель работы – разработка программно-информационной системы для управления проектами – достигнута.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.102–77. Единая система программной документации. Стадии разработки : межгосударственный стандарт : издание официальное : утвержден Постановлением Государственного комитета стандартов Совета Министров СССР от 20 декабря 1977 г. № 2851 : дата введения 1980-01-01. – Москва : Стандартинформ, 2010. – Текст : непосредственный.

2. ГОСТ 34.601–90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания : межгосударственный стандарт : издание официальное : утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 29 декабря 1990 г. № 3469 : дата введения 1992-01-01. – Москва : Стандартинформ, 2009. – Текст : непосредственный.

3. ГОСТ Р 7.0.5–2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления : национальный стандарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 28 апреля 2008 г. № 95-ст : дата введения 2009-01-01. – Москва : Стандартинформ, 2008. – Текст : непосредственный.

4. ГОСТ Р ИСО/МЭК 25010–2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов : национальный стандарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 29 мая 2015 г. № 442-ст : дата введения 2016-06-01. – Москва : Стандартинформ, 2015. – Текст : непосредственный.

5. ГОСТ Р ИСО 21500–2023. Управление проектами, программами и портфелями проектов. Контекст и основные понятия : национальный стан-

дарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 30 октября 2023 г. № 1296-ст : дата введения 2024-06-01. – Москва : Российский институт стандартизации, 2023. – Текст : непосредственный.

6. ГОСТ Р 54869–2011. Проектный менеджмент. Требования к управлению проектом : национальный стандарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 22 декабря 2011 г. № 1582-ст : дата введения 2012-09-01. – Москва : Стандартинформ, 2012. – Текст : непосредственный.

7. OMG Unified Modeling Language (OMG UML). Version 2.5.1 : specification / Object Management Group. – Needham : OMG, 2017. – 796 p. – URL: <https://www.omg.org/spec/UML/2.5.1> (дата обращения: 10.05.2026).

8. Андерсон, Д. Канбан. Альтернативный путь в Agile / Д. Андерсон ; перевод с английского А. Коробейникова. – Москва : Манн, Иванов и Фербер, 2017. – 336 с. – ISBN 978-5-00100-530-8. – Текст : непосредственный.

9. Ахмадуллин, Д. Ф. Системы управления проектами: анализ существующих программных решений / Д. Ф. Ахмадуллин. – DOI 10.24411/2658-4964-2020-1120. // StudNet. – 2020. – № 9. – URL: <https://cyberleninka.ru/article/n/sistemy-upravleniya-proektami-analiz-suschestvuyuschih-programmnyh-resheniy> (дата обращения: 10.05.2026).

10. Богнюков, А. А. Применение инструментов проектного планирования: диаграмма Ганта и сетевой график / А. А. Богнюков, Д. Ю. Зорькин, И. А. Тарасова. // Известия Южного федерального университета. Технические науки. – 2025. – № 4. – С. 102–110. – URL: <https://cyberleninka.ru/article/n/primenenie-instrumentov-proektnogo-planirovaniya-diagramma-ganta-i-setevoy-grafik> (дата обращения: 10.05.2026).

11. Бек, К. Экстремальное программирование. Разработка через тестирование / К. Бек ; перевод с английского. – Санкт-Петербург : Питер, 2017.

– 224 с. – (Библиотека программиста). – ISBN 978-5-496-02570-6. – Текст : непосредственный.

12. Буч, Г. Язык UML. Руководство пользователя / Г. Буч, Дж. Рамбо, И. Якобсон ; перевод с английского. – 2-е изд. – Москва : ДМК Пресс, 2015. – 496 с. – ISBN 978-5-94074-644-7. – Текст : непосредственный.

13. Буч, Г. Объектно-ориентированный анализ и проектирование с примерами приложений / Г. Буч, Р. А. Максимчук, М. У. Энгл [и др.] ; перевод с английского. – 3-е изд. – Москва : ООО «И.Д. Вильямс», 2010. – 720 с. – ISBN 978-5-8459-1401-9. – Текст : непосредственный.

14. Васильев, П. А. Web-программирование на языке Python. Фреймворки Django, Flask / П. А. Васильев. // Наука, техника и образование. – 2016. – № 12 (30). – URL: <https://cyberleninka.ru/article/n/web-programmirovanie-na-yazyke-python-freymvorki-django-flask> (дата обращения: 10.05.2026).

15. Гайнанова, Р. Ш. Создание клиент-серверных приложений / Р. Ш. Гайнанова, О. А. Широкова. // Вестник Казанского технологического университета. – 2017. – Т. 20, № 17. – URL: <https://cyberleninka.ru/article/n/sozдание-klient-servernyh-prilozheniy> (дата обращения: 10.05.2026).

16. Гамма, Э. Паттерны объектно-ориентированного проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес ; перевод с английского. – Санкт-Петербург : Питер, 2020. – 448 с. – (Библиотека программиста). – ISBN 978-5-4461-1595-2. – Текст : непосредственный.

17. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; перевод с английского. – 8-е изд. – Москва : ООО «И.Д. Вильямс», 2005. – 1328 с. – ISBN 5-8459-0788-8. – Текст : непосредственный.

18. Дружинина, В. Д. Основные принципы методологии SCRUM для управления проектами / В. Д. Дружинина. // Экономика. Социология. Право. – 2022. – № 1 (25). – URL: <https://cyberleninka.ru/article/n/osnovnye-printsipy-metodologii-scrum-dlya-upravleniya-proektami> (дата обращения: 10.05.2026).

19. Зайцева, И. А. Практика применения методологий Agile, Scrum в ИТ-проектах / И. А. Зайцева, В. Ш. Ебата, Н. А. Ковбаса. // Индустриальная экономика. – 2021. – № 1. – URL: <https://cyberleninka.ru/article/n/praktika>

primeneniya-metodologiy-agile-scrum-v-it-proektah (дата обращения: 10.05.2026).

20. Игнатюк, В. А. Логическое проектирование реляционной базы данных на основе принципов нормализации / В. А. Игнатюк, Е. А. Сторожок. // Территория новых возможностей. Вестник Владивостокского государственного университета экономики и сервиса. – 2011. – № 4 (13). – URL: <https://cyberleninka.ru/article/n/logicheskoe-proektirovanie-relyatsionnoy-bazy-dannyh-na-osnove-printsipov-normalizatsii> (дата обращения: 10.05.2026).

21. Айдинян, А. Р. Эффективность применения некоторых алгоритмов хеширования для веб-приложения с интерфейсами регистрации и аутентификации пользователей / А. Р. Айдинян, О. Л. Легонько, Н. С. Мамонов. // Инженерный вестник Дона. – 2025. – № 8. – URL: <https://cyberleninka.ru/article/n/effektivnost-primeneniya-nekotoryh-algoritmov-heshirovaniya-dlya-veb-prilozheniya-s-interfeysami-registratsii-i-autentifikatsii> (дата обращения: 10.05.2026).

22. Ковалев, В. В. Актуальность использования архитектуры REST для обмена данными между клиент-серверными приложениями / В. В. Ковалев, В. В. Храмков, Е. И. Семенова. // Актуальные проблемы авиации и космонавтики. – 2019. – Т. 2. – URL: <https://cyberleninka.ru/article/n/aktualnost-ispolzovaniya-arhitektury-rest-dlya-obmena-dannymi-mezhdu-klient-servernymi-prilozheniyami> (дата обращения: 10.05.2026).

23. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг ; перевод с английского. – 3-е изд. – Москва : ООО «И.Д. Вильямс», 2017. – 1440 с. – ISBN 978-5-8459-2020-1. – Текст : непосредственный.

24. Куренков, А. Л. Управление проектами цифровой трансформации: современные тенденции и особенности / А. Л. Куренков. // Вестник Российского экономического университета им. Г. В. Плеханова. – 2024. – № 5. – URL: <https://cyberleninka.ru/article/n/upravlenie-proektami-tsifrovoy-transformatsii-sovremennye-tendentsii-i-osobennosti> (дата обращения: 10.05.2026).

25. Лоре, А. Проектирование веб-API / А. Лоре ; перевод с английского. – Москва : ДМК Пресс, 2020. – 440 с. – ISBN 978-5-97060-861-6. – Текст : непосредственный.

26. Лутц, М. Изучаем Python / М. Лутц ; перевод с английского. – 5-е изд. – Санкт-Петербург : ООО «Диалектика», 2019–2020. – Т. 1: 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.

27. Управление проектами : учебное пособие / И. И. Мазур, В. Д. Шапиро, Н. Г. Ольдерогге, А. В. Полковников ; под общей редакцией И. И. Мазура и В. Д. Шапиро. – 10-е изд., стер. – Москва : Омега-Л, 2014. – 960 с. – ISBN 978-5-370-02800-7. – Текст : непосредственный.

28. Макеева, О. В. Моделирование информационных процессов с помощью UML / О. В. Макеева, М. В. Сартаков, Е. А. Чернов. // Инновации и инвестиции. – 2021. – № 5. – URL: <https://cyberleninka.ru/article/n/modelirovanie-informatsionnyh-protsessov-s-pomoschyu-uml> (дата обращения: 10.05.2026).

29. Мокшин, В. В. Современные подходы к проектам. Методологии. AGILE: SCRUM / В. В. Мокшин, А. М. Гайнутдинова, С. О. Самсонов. // StudNet. – 2021. – № 6. – URL: <https://cyberleninka.ru/article/n/sovremennye-podhody-k-proektam-metodologii-agile-scrum> (дата обращения: 10.05.2026).

30. Орлов, С. А. Программная инженерия. Технологии разработки программного обеспечения : учебник для вузов / С. А. Орлов. – 5-е изд., обнов. и доп. – Санкт-Петербург : Питер, 2020. – 640 с. – (Учебник для вузов. Стандарт третьего поколения). – ISBN 978-5-4461-1348-4. – Текст : непосредственный.

31. Понин, Ф. Н. Методология проектирования и создания баз данных для современного программного обеспечения / Ф. Н. Понин. // Universum: технические науки. – 2024. – № 1 (118). – URL: <https://cyberleninka.ru/article/n/metodologiya-proektirovaniya-i-sozdaniya-baz-dannyh-dlya-sovremennogo-programmnogo-obespecheniya> (дата обращения: 10.05.2026).

32. Романов, С. С. Достоинства, недостатки и альтернативы объектно-реляционного отображения (ORM) / С. С. Романов. // Таврический научный обозреватель. – 2016. – № 11-2 (16). – URL:

<https://cyberleninka.ru/article/n/dostoinstva-nedostatki-i-alternativy-obektno-relyatsionnogo-otobrazheniya-orm> (дата обращения: 10.05.2026).

33. Робсон, Э. Изучаем HTML и CSS / Э. Робсон, Э. Фримен ; перевод с английского. – 2-е изд. – Санкт-Петербург : Питер, 2014. – 720 с. – (Head First O'Reilly). – ISBN 978-5-496-00653-8. – Текст : непосредственный.

34. Сазерленд, Дж. Scrum. Революционный метод управления проектами / Дж. Сазерленд ; перевод с английского. – Москва : Манн, Иванов и Фербер, 2016. – 272 с. – ISBN 978-5-00057-722-6. – Текст : непосредственный.

35. Сафонова, А. А. Информационные системы управления проектами / А. А. Сафонова, О. Н. Куксачева. // Формула менеджмента. – 2020. – № 1. – URL: <https://cyberleninka.ru/article/n/informatsionnye-sistemy-upravleniya-proektami> (дата обращения: 10.05.2026).

36. Басов, А. С. Сравнение современных СУБД / А. С. Басов. // Вестник науки. – 2020. – № 9 (30). – URL: <https://cyberleninka.ru/article/n/sravnenie-sovremennyh-subd> (дата обращения: 10.05.2026).

37. Терехова, А. Е. Обзор методологий управления проектами / А. Е. Терехова, Н. Ю. Вербя. // Вестник университета. – 2014. – № 2. – URL: <https://cyberleninka.ru/article/n/obzor-metodologiy-upravleniya-proektami> (дата обращения: 10.05.2026).

38. Усков, И. Д. Безопасность веб-приложений: XSS, CSRF и современные методы защиты / И. Д. Усков, В. Д. Сандаков. // Вестник науки. – 2026. – URL: <https://cyberleninka.ru/article/n/bezopasnost-veb-prilozheniy-xss-csrf-i-sovremennye-metody-zaschity> (дата обращения: 10.05.2026).

39. Фаулер, М. UML. Основы : краткое руководство по стандартному языку объектного моделирования / М. Фаулер ; перевод с английского. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2013. – 192 с. – ISBN 5-93286-060-X. – Текст : непосредственный.

40. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер, Д. Райс, М. Фоммел [и др.] ; перевод с английского. – Москва : ООО «И.Д. Вильямс», 2018. – 544 с. – ISBN 978-5-8459-1611-2. – Текст : непосредственный.

41. Флэнаган, Д. JavaScript. Полное руководство / Д. Флэнаган ; перевод с английского. – 7-е изд. – Москва : ООО «Диалектика», 2021. – 720 с. – ISBN 978-5-907203-79-2. – Текст : непосредственный.
42. Форсье, Дж. Django. Разработка веб-приложений на Python / Дж. Форсье, П. Биссекс, У. Чан ; перевод с английского. – Санкт-Петербург : Символ-Плюс, 2009. – 456 с. – ISBN 978-5-93286-167-7. – Текст : непосредственный.
43. Чиннатамби, К. Изучаем React / К. Чиннатамби ; перевод с английского. – 2-е изд. – Москва : Эксмо, 2019. – 368 с. – (Мировой компьютерный бестселлер). – ISBN 978-5-04-098028-4. – Текст : непосредственный.
44. Шор, А. М. Сравнительный анализ подходов в разработке API веб-приложений / А. М. Шор. // StudNet. – 2020. – № 9. – URL: <https://cyberleninka.ru/article/n/sravnitelnyy-analiz-podhodov-v-razrabotke-api-veb-prilozheniy> (дата обращения: 10.05.2026).
45. Яковчук, А. И. Гибридный подход к управлению проектами планирования мероприятий / А. И. Яковчук. // Экономический вектор. – 2022. – Т. 28, № 1. – С. 56–60. – URL: <https://cyberleninka.ru/article/n/gibridnyy-podhod-k-upravleniyu-proektami-planirovaniya-meropriyatiy> (дата обращения: 10.05.2026).
46. Якобсон, А. Унифицированный процесс разработки программного обеспечения / А. Якобсон, Г. Буч, Дж. Рамбо ; перевод с английского. – Санкт-Петербург : Питер, 2002. – 496 с. – (Классика computer science). – ISBN 5-318-00358-3. – Текст : непосредственный.
47. Руководство к своду знаний по управлению проектами (Руководство РМВОК). Шестое издание + Agile : практическое руководство / Project Management Institute ; перевод с английского. – Москва : Олимп-Бизнес, 2022. – 974 с. – ISBN 978-5-9693-0402-4. – Текст : непосредственный.
48. Banks, A. Learning React. Modern Patterns for Developing React Apps / A. Banks, E. Porcello. – 2nd ed. – Sebastopol : O'Reilly Media, 2020. – 310 p. – ISBN 978-1-492-05172-5. – Текст : непосредственный.

49. Django documentation : официальный сайт. – San Francisco : Django Software Foundation, 2005–. – URL: <https://docs.djangoproject.com> (дата обращения: 10.05.2026).
50. Django REST framework : сайт. – 2011–. – URL: <https://www.django-rest-framework.org> (дата обращения: 10.05.2026).
51. MDN Web Docs : официальный сайт. – Mountain View : Mozilla Foundation, 2005–. – URL: <https://developer.mozilla.org> (дата обращения: 10.05.2026).
52. OWASP Top 10:2021 // The OWASP Foundation : [сайт]. – 2021. – URL: <https://owasp.org/Top10/2021/> (дата обращения: 10.05.2026).
53. PMI Standards & Publications // Project Management Institute : [сайт]. – Newtown Square, 1969–. – URL: <https://www.pmi.org/standards> (дата обращения: 10.05.2026).
54. React : официальный сайт. – Menlo Park : Meta Platforms, 2013–. – URL: <https://react.dev> (дата обращения: 10.05.2026).
55. Hypertext Transfer Protocol (HTTP) Semantics : RFC 9110 / R. Fielding, M. Nottingham, J. Reschke. – Internet Engineering Task Force, June 2022. – URL: <https://datatracker.ietf.org/doc/html/rfc9110> (дата обращения: 10.05.2026).
56. Лучшие таск-трекеры для управления проектами и задачами в 2026 году: обзор 19 российских систем. // Хабр. – 2025. – URL: <https://habr.com/ru/companies/yougile/articles/991204/> (дата обращения: 10.05.2026).



## ПРИЛОЖЕНИЕ А

### Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.11.

# Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

## ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

Программно-информационная система для управления проектами

Руководитель ВКРБ  
к.т.н, доцент  
Чаплыгин Александр Александрович

Автор ВКРБ  
студент группы ПО-226  
Марин Никита Романович

|                                             |               |         |           |
|---------------------------------------------|---------------|---------|-----------|
| ВКРБ 2206067.09.03.04.26.017                |               |         |           |
| Авторы                                      | Фамилия И.О.  | Подпись | Дата      |
| Чаплыгин А.А.                               | Марин Н.Р.    |         |           |
| Чаплыгин А.А.                               | Чаплыгин А.А. |         |           |
| Чаплыгин А.А.                               | Чаплыгин А.А. |         |           |
| Сведения о ВКРБ                             |               | Лист 1  | Листов 11 |
| Выпускная квалификационная работа бакалавра |               | ПО-226  |           |

Рисунок А.1 – Сведения о ВКРБ

## Цель и задачи разработки

**Цель настоящей работы** – разработка веб-ориентированной программно-информационной системы для управления проектами, рассчитанной на небольшие команды и объединяющей в одном инструменте реестр проектов, диаграмму Ганта с поддержкой четырех типов зависимостей задач и разграничение прав участников по трем ролям.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и сопоставить функциональные возможности существующих ИС УП, доступных российским пользователям;
- сформировать техническое задание на разработку, описать прецеденты использования системы;
- спроектировать архитектуру системы, схему базы данных и программный интерфейс взаимодействия клиентской и серверной частей;
- реализовать клиентскую и серверную части, диаграмму Ганта с интерактивным изменением дат и порядка задач, режим добавления связей и разграничение прав участников;
- провести системное тестирование программной системы.

|                                   |              |                          |         |
|-----------------------------------|--------------|--------------------------|---------|
| ВКРБ 2206067.09.03.04.26.017      |              |                          |         |
| Автор работы                      | Иванов И.И.  | Получено                 | Дата    |
| Рецензент                         | Петров П.П.  | Цель и задачи разработки | Лист 1  |
| Проверенный                       | Сидоров С.С. | Лист 2                   | Лист 11 |
| Выпускная квалификационная работа |              | 03/04/2026               |         |

Рисунок А.2 – Цель и задачи разработки

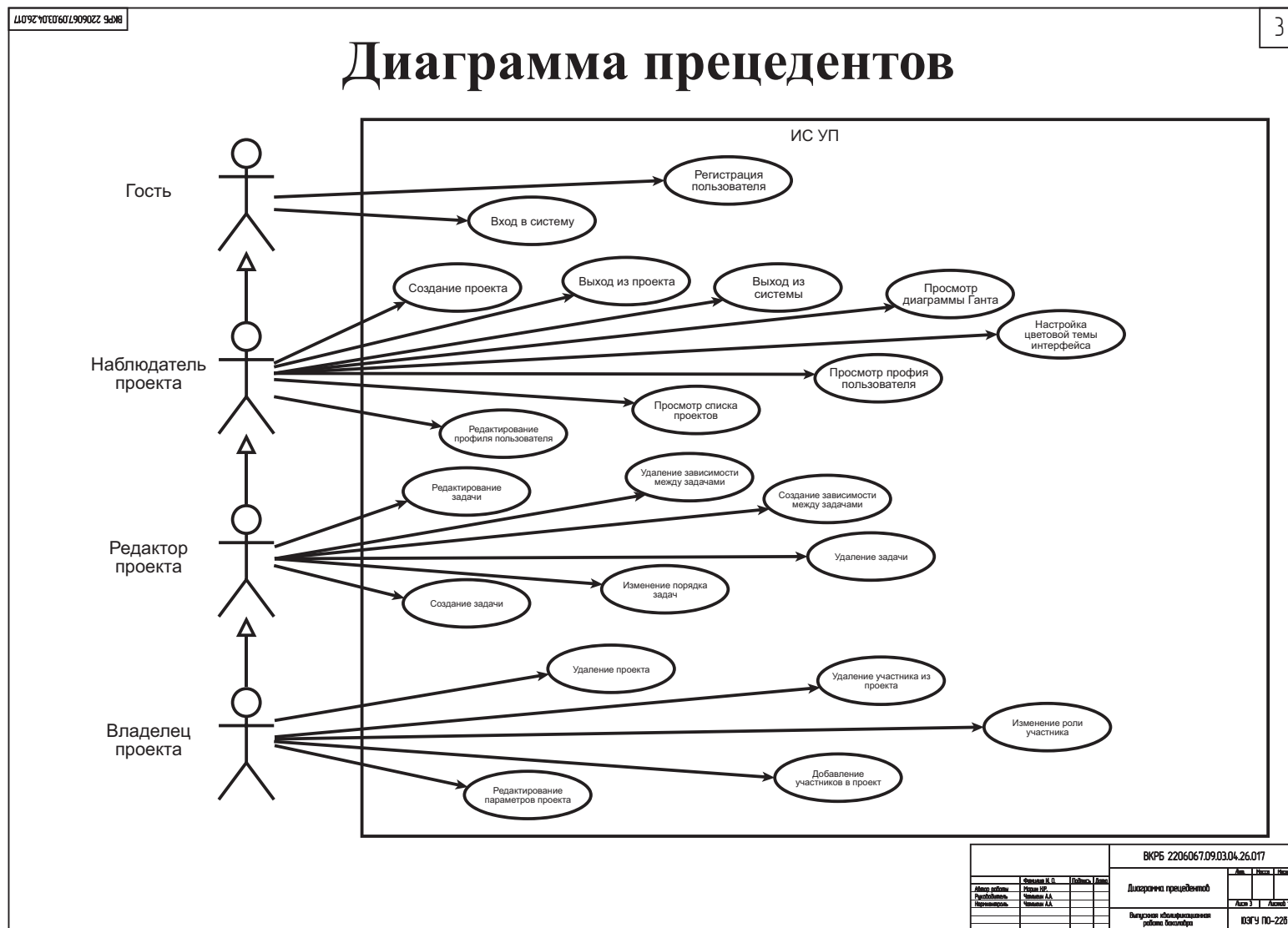


Рисунок А.3 – Диаграмма прецедентов

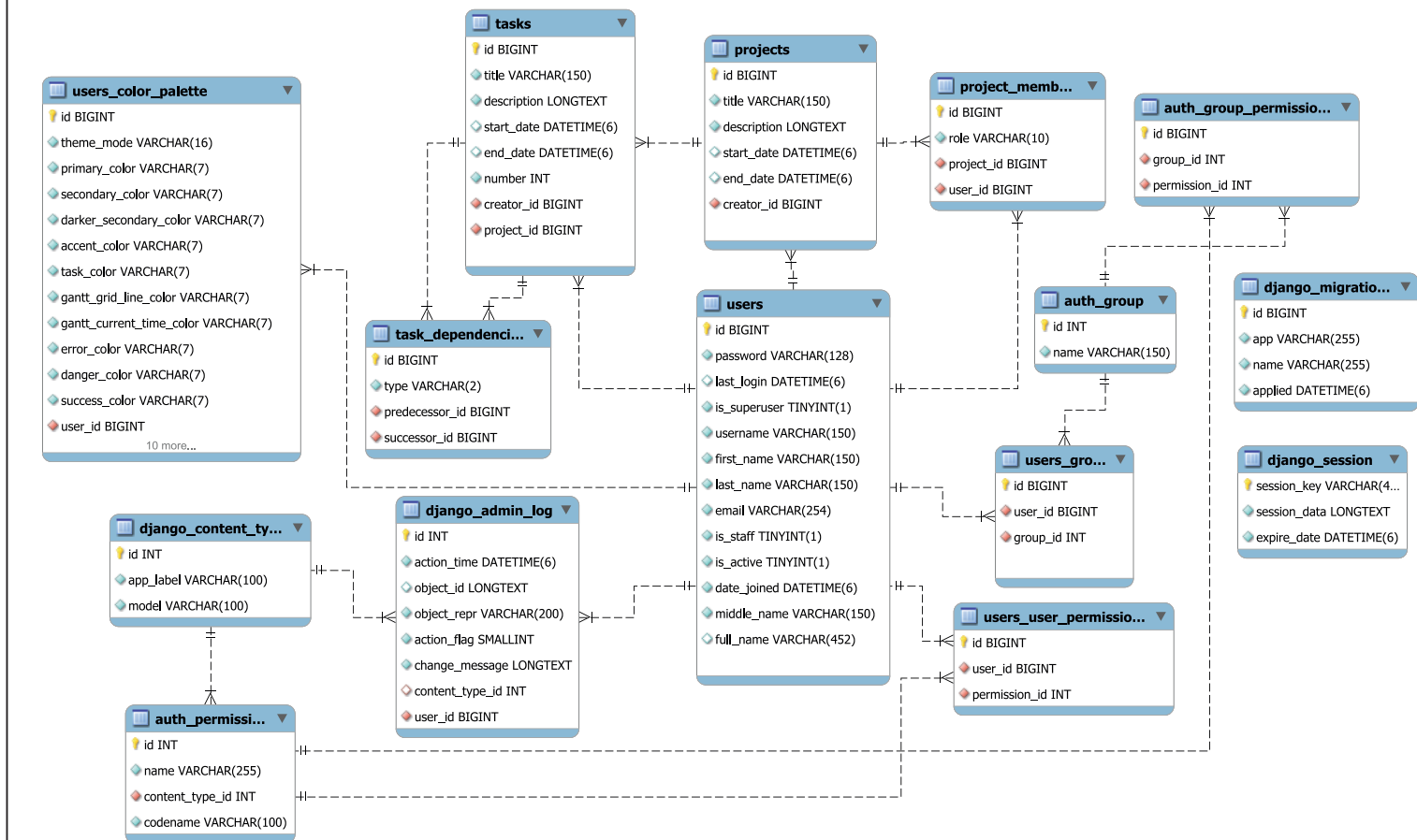


Рисунок А.4 – Концептуальная модель данных



Рисунок А.5 – Архитектура системы

# Схема базы данных



|                                              |             |              |         |
|----------------------------------------------|-------------|--------------|---------|
| БКР6 2206067.09.03.04.26.017                 |             |              |         |
| Автом. админ.                                | Создан И.И. | Получен И.И. | Исключ. |
| Публиковать                                  | Создан И.И. | Получен И.И. | Исключ. |
| Исключать                                    | Создан И.И. | Получен И.И. | Исключ. |
| Схема базы данных                            |             |              |         |
| Выполнен идентификационный релиз базы данных |             |              |         |
| 03/04/2020                                   |             |              |         |

Рисунок А.6 – Схема базы данных

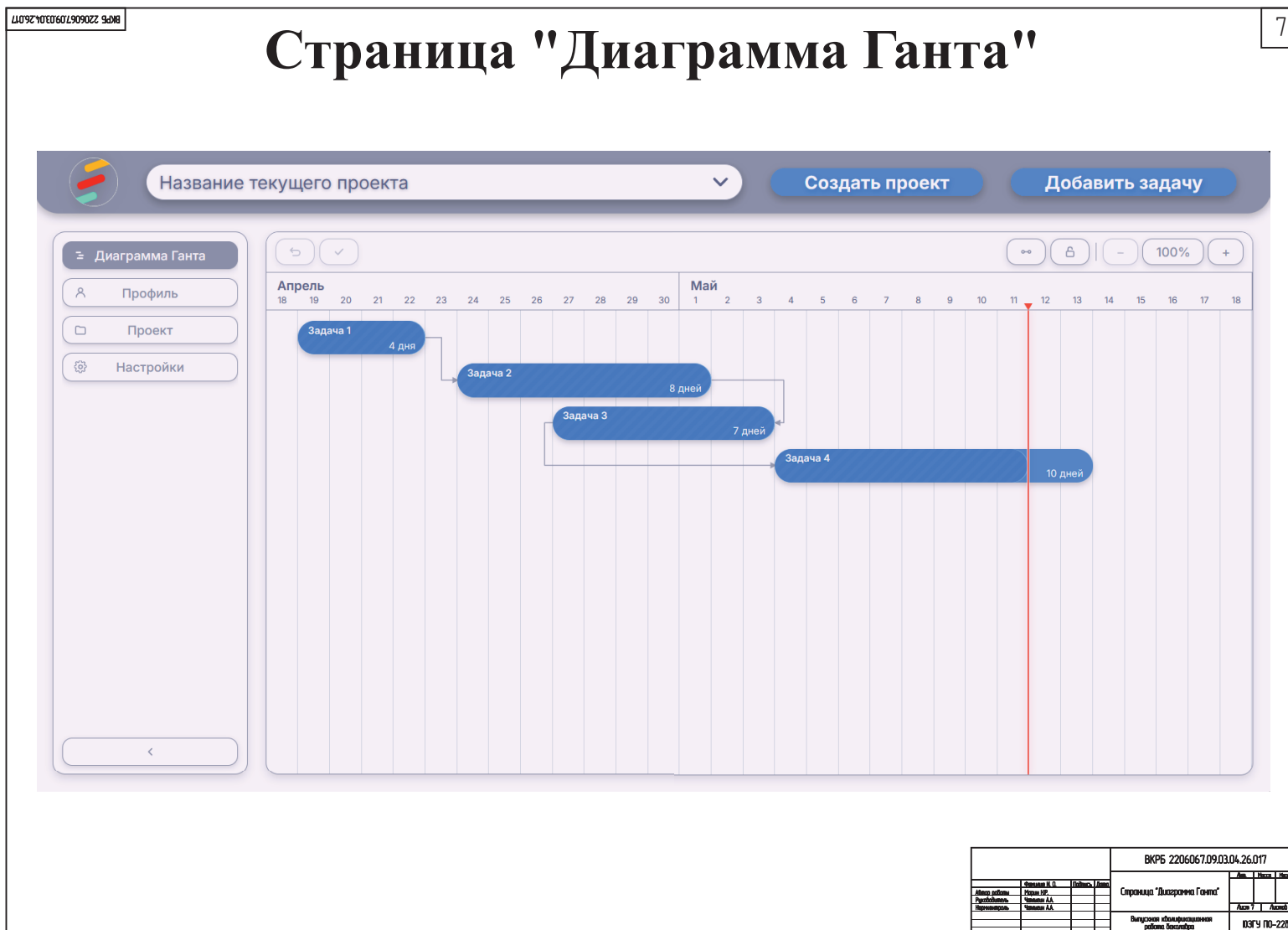


Рисунок А.7 – Страница "Диаграмма Ганта"



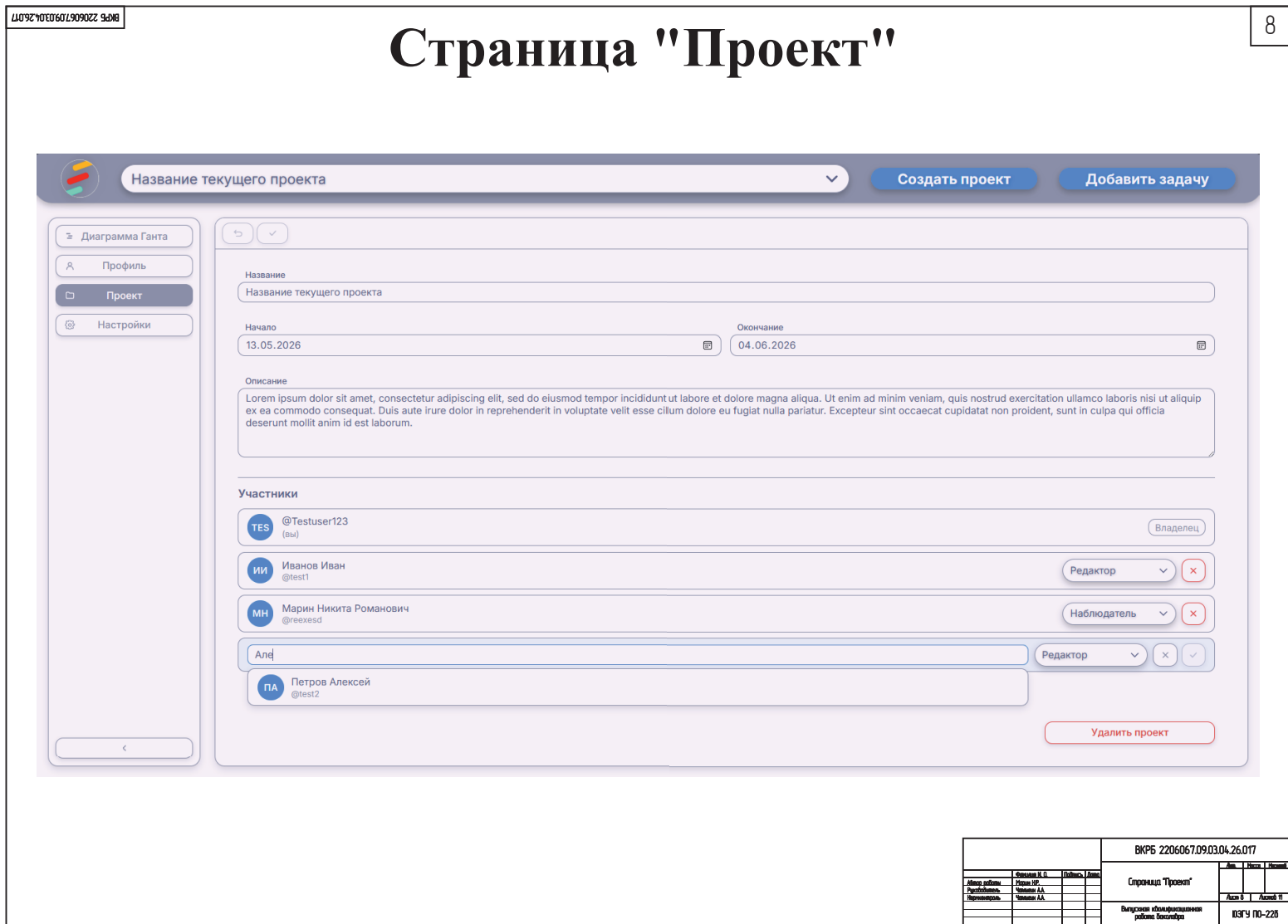


Рисунок А.8 – Страница "Проект"



Рисунок А.9 – Карточка задачи

|  |  |  |  |                              |  |  |
|--|--|--|--|------------------------------|--|--|
|  |  |  |  | ВКР6 2206067.09.03.04.26.017 |  |  |
|  |  |  |  | Справка настройк             |  |  |
|  |  |  |  |                              |  |  |
|  |  |  |  | Выполнен комплексный         |  |  |
|  |  |  |  | работ по установке           |  |  |
|  |  |  |  | 03/14 10-22                  |  |  |

Рисунок А.10 – Страница настроек

## Заключение

В результате выполнения выпускной квалификационной работы разработана веб-ориентированная программно-информационная система для управления проектами.

В процессе разработки решены следующие задачи:

- проведен анализ предметной области управления проектами и сравнительный анализ восьми существующих ИС УП, доступных российским пользователям; обоснована потребность в собственной разработке;
- сформировано техническое задание на разработку, описаны прецеденты использования системы по четырем ролям пользователей;
- спроектирована архитектура программной системы, построена ER-модель данных, описана структура базы данных, разработан программный интерфейс между клиентской и серверной частями;
- реализованы клиентская и серверная части системы, интерактивная диаграмма Ганта с поддержкой четырех типов зависимостей задач, ролевая модель разграничения прав и управление составом команды;
- проведено системное тестирование, подтвердившее корректность работы реализованной функциональности.

Все поставленные задачи решены. Цель работы достигнута.

|                                            |              |                |                |                                   |            |          |
|--------------------------------------------|--------------|----------------|----------------|-----------------------------------|------------|----------|
|                                            |              |                |                | ВКРБ 2206067.09.03.04.26.017      |            |          |
| Автор работы<br>Перепечатка<br>Перепечатка | Форман Н.А.  | Полосин А.А.   | Лисица А.А.    | Заключение                        | Лист 1     | Листов 1 |
|                                            | Ульянов И.В. | Чемоданов А.А. | Чемоданов А.А. |                                   | Лист 1     | Листов 1 |
|                                            |              |                |                |                                   |            |          |
|                                            |              |                |                |                                   |            |          |
|                                            |              |                |                | Выпускная квалификационная работа | 03/04/2026 |          |

Рисунок А.11 – Заключение

## ПРИЛОЖЕНИЕ Б

### Фрагменты исходного кода программы

Проверка сложности пароля при регистрации.

password.py

```
14 def validate(self, password, user=None):
15     errors = []
16
17     if not ALLOWED_CHARS_RE.match(password):
18         errors.append(ValidationError('Password contains invalid
19             characters (spaces, Cyrillic or other disallowed characters).')
20         ))
21
22     if not LOWERCASE_RE.search(password):
23         errors.append(ValidationError('Password must contain at least one
24             lowercase Latin letter.'))
25
26     if not UPPERCASE_RE.search(password):
27         errors.append(ValidationError('Password must contain at least one
28             uppercase Latin letter.'))
29
30     if not DIGIT_RE.search(password):
31         errors.append(ValidationError('Password must contain at least one
32             digit.'))
33
34     if not SPECIAL_RE.search(password):
35         errors.append(ValidationError('Password must contain at least one
36             special character (!@#$%^&*()_+-=[]{}|;:,.<>?/~\'\"').'))
37
38     if errors:
39         raise ValidationError(errors)
```

Поиск в ширину для обнаружения циклов в графе зависимостей задач перед сохранением новой связи.

dependency.py

```
61 def would_create_cycle(project_id, predecessor_id, successor_id):
62     if predecessor_id == successor_id:
63         return True
64
65     edges = defaultdict(list)
66
67     raw = TaskDependency.objects.filter(
68         predecessor__project_id=project_id
69     ).values_list('predecessor_id', 'successor_id')
70
71     for src, dst in raw:
72         edges[src].append(dst)
73
74     edges[predecessor_id].append(successor_id)
```

```

75
76 visited = set()
77 queue = deque([successor_id])
78
79 while queue:
80     current = queue.popleft()
81
82     if current == predecessor_id:
83         return True
84
85     if current in visited:
86         continue
87
88     visited.add(current)
89
90     for neighbor in edges.get(current, ()):
91         queue.append(neighbor)
92
93 return False

```

Проверка согласованности дат пары задач с выбранным типом связи.

dependency.py

```

15 def is_pair_satisfied(dep_type, pred_start, pred_end, succ_start, succ_end):
16     pred_start_day = to_day(pred_start)
17     pred_end_day = to_day(pred_end)
18     succ_start_day = to_day(succ_start)
19     succ_end_day = to_day(succ_end)
20
21     if dep_type == TaskDependency.FINISH_TO_START:
22         if pred_end_day is None or succ_start_day is None:
23             return True
24
25         return succ_start_day > pred_end_day
26
27     if dep_type == TaskDependency.START_TO_START:
28         if pred_start_day is None or succ_start_day is None:
29             return True
30
31         return succ_start_day >= pred_start_day
32
33     if dep_type == TaskDependency.FINISH_TO_FINISH:
34         if pred_end_day is None or succ_end_day is None:
35             return True
36
37         return succ_end_day >= pred_end_day
38
39     if dep_type == TaskDependency.START_TO_FINISH:
40         if pred_start_day is None or succ_end_day is None:
41             return True
42
43         return succ_end_day >= pred_start_day
44

```

```
45     return True
```

Определение роли пользователя в проекте и производные проверки прав доступа.

permissions.py

```
9 def get_user_role(user, project_id):
10     if user is None or not user.is_authenticated:
11         return None
12
13     membership = (
14         ProjectMember.objects
15         .filter(project_id=project_id, user=user)
16         .only('role')
17         .first()
18     )
19
20     return membership.role if membership else None
21
22
23 def is_member(user, project_id):
24     if user and user.is_superuser:
25         return True
26
27     return get_user_role(user, project_id) is not None
28
29
30 def is_writer(user, project_id):
31     if user and user.is_superuser:
32         return True
33
34     return get_user_role(user, project_id) in WRITER_ROLES
35
36
37 def is_owner(user, project_id):
38     if user and user.is_superuser:
39         return True
40
41     return get_user_role(user, project_id) == ProjectMember.OWNER
```

Создание проекта с автоматическим добавлением автора в участники с ролью владельца в одной транзакции.

project.py

```
26 def perform_create(self, serializer):
27     with transaction.atomic():
28         project = serializer.save(creator=self.request.user)
29         ProjectMember.objects.create(
30             project=project,
31             user=self.request.user,
32             role=ProjectMember.OWNER
```

33

)

Определение типа связи по выбранным торцам полос задач на диаграмме Ганта.

dependencyHelpers.js

```

285 export function inferDependencyType(fromSide, toSide) {
286   if (fromSide === 'end' && toSide === 'start') {
287     return DEPENDENCY_TYPE_FS;
288   }
289
290   if (fromSide === 'start' && toSide === 'start') {
291     return DEPENDENCY_TYPE_SS;
292   }
293
294   if (fromSide === 'end' && toSide === 'end') {
295     return DEPENDENCY_TYPE_FF;
296   }
297
298   if (fromSide === 'start' && toSide === 'end') {
299     return DEPENDENCY_TYPE_SF;
300   }
301
302   return null;
303 }

```

Применение цветовой палитры к интерфейсу через CSS-переменные корневого элемента документа.

ThemeContext.jsx

```

305 useEffect(() => {
306   const root = document.documentElement;
307
308   THEME_COLOR_FIELDS.forEach((field) => {
309     root.style.setProperty(field.cssVariable, resolvedColors[field.key]);
310   });
311   LEGACY_CSS_VARIABLES.forEach((field) => {
312     root.style.setProperty(field.cssVariable, resolvedColors[field.key]);
313   });
314
315   root.dataset.themeMode = previewSettings.mode;
316   root.dataset.resolvedTheme = resolvedThemeMode;
317   root.style.colorScheme = resolvedThemeMode === THEME_MODE_DARK ? 'dark' :
    'light';
318 }, [resolvedColors, resolvedThemeMode, previewSettings.mode]);

```



Построение маршрута стрелки зависимости между двумя полосами задач на диаграмме Ганта: при пересечении предпочтительного пути с другими полосами выбирается обходной маршрут.

GanttChart.jsx

```
775 function buildArrowPath(fromX, fromY, fromSide, toX, toY, toSide, taskRects =  
    []) {  
776   const preferred = buildPreferredArrowPoints(fromX, fromY, fromSide, toX,  
       toY, toSide);  
777   const fromDirX = fromSide === 'end' ? 1 : -1;  
778   const toDirX = toSide === 'end' ? 1 : -1;  
779   const fromOut = { x: fromX + fromDirX * ARROW_PADDING_PX, y: fromY };  
780   const toIn = { x: toX + toDirX * ARROW_PADDING_PX, y: toY };  
781   const routeRects = findRelevantRouteRects(taskRects, fromOut, toIn);  
782   const isSameSideTooLong = fromSide === toSide && Math.abs(toX - fromOut.x)  
       > SAME_SIDE_DIRECT_MAX_PX;  
783  
784   if (!isSameSideTooLong && !pathIntersectsTasks(preferred, routeRects)) {  
785     return arrowPointsToPath(preferred);  
786   }  
787  
788   const route = findBestRoutePoints(fromOut, toIn, routeRects);  
789  
790   if (!route) {  
791     return arrowPointsToPath(preferred);  
792   }  
793  
794   return arrowPointsToPath([  
795     { x: fromX, y: fromY },  
796     ...route,  
797     { x: toX, y: toY }  
798   ]);  
799 }
```